

Università degli Studi di Trento

Dipartimento di Ingegneria e Scienza dell'Informazione

Appunti di Basi di dati

Lezioni, e Simulazioni d'esame

A cura di:

Mirco Negri

Appunti redatti per gli studenti del DISI.
Eventuali errori possono essere segnalati su GitHub.

novembre 2026

Indice

1 Le Basi di Dati e i loro Utenti **6**
- 1.1 Cos'è una Base di Dati . . . 6
- 1.2 DBMS e Database System . . . 6
- 1.3 Architettura di un Database System . . . 6
- 1.4 Cosa Deve Fornire un DBMS . . . 7
- 1.5 Interazione tra Applicazioni e Basi di Dati . . . 7
- 1.6 Le Origini del Modello Relazionale . . . 7
- 1.7 Indipendenza dei Dati . . . 7
  - 1.7.1 Indipendenza Logica dei Dati . . . 8
  - 1.7.2 Indipendenza Fisica dei Dati . . . 8
- 1.8 Livelli di Astrazione dei Dati . . . 8
- 1.9 Perché Usare una Base di Dati . . . 9
  - 1.9.1 Condivisione dei Dati e Transazioni Multi-utente . . . 9
- 1.10 Digressione: OLTP vs. OLAP . . . 9
- 1.11 Utenti di una Base di Dati . . . 10
  - 1.11.1 Tipi di Utilizzatori . . . 10
- 1.12 Quando è Meglio NON Usare un DBMS . . . 10

2 Il Modello Relazionale **11**
- 2.1 Concetti Generali . . . 11
- 2.2 Cos'è una Relazione . . . 11
  - 2.2.1 Definizione Intensionale . . . 11
  - 2.2.2 Definizione Estensionale . . . 12
- 2.3 Stato di una Relazione . . . 12
- 2.4 Chiave di una Relazione . . . 12
- 2.5 Terminologia . . . 12
- 2.6 Vincoli . . . 12
- 2.7 Vincolo di Chiave . . . 13
- 2.8 Vincolo di Integrità dell'Entità (Entity Integrity) . . . 13
- 2.9 Vincoli di Integrità Referenziale . . . 14
- 2.10 Schema di un Database Relazionale . . . 15
- 2.11 Altri Tipi di Vincolo . . . 15
- 2.12 Stato di una Base di Dati Relazionale . . . 15
- 2.13 Operazioni di Aggiornamento e Violazione dei Vincoli . . . 15
  - 2.13.1 Violazioni per Operazione INSERT . . . 16
  - 2.13.2 Violazioni per Operazione DELETE . . . 16
  - 2.13.3 Violazioni per Operazione UPDATE . . . 16
- 2.14 Come Preservare l'Integrità Referenziale . . . 16

3 Algebra Relazionale **17**
- 3.1 Introduzione . . . 17
- 3.2 Schema e Dati di Riferimento . . . 18
- 3.3 Selezione (SELECT, σ) . . . 18
- 3.4 Proiezione (PROJECT, π) . . . 19
- 3.5 Unione, Differenza e Intersezione . . . 19
- 3.6 Prodotto Cartesiano ($\times$) . . . 19
- 3.7 Ridenominazione (RENAME, ρ) . . . 19

3.8 Join . . . 19
3.8.1 Theta-Join . . . 19
3.8.2 Join Naturale . . . 20
3.8.3 Equijoin . . . 20
3.9 Divisione (÷) . . . 20
3.10 Outer Join . . . 20
3.11 Espressioni ed Alberi di Query . . . 21

4 SQL **21**
4.1 Cos'è SQL . . . 21
4.2 Istanza di Riferimento . . . 21
4.3 Struttura Base di una Query . . . 21
4.3.1 Semantica Valutativa (concettuale) . . . 22
4.4 Operatori di Confronto . . . 22
4.5 Operatori di Aggregazione . . . 23
4.6 GROUP BY e HAVING . . . 23
4.6.1 Esempio (Q32) . . . 24
4.7 Interrogazioni Annidate (Nested Queries) . . . 24
4.7.1 Query Indipendenti e Correlate . . . 24
4.7.2 Casi d'Uso . . . 24
4.7.3 Annidamento a Più Livelli e Operatori ANY/ALL . . . 25
4.8 Operatori Insiemistici . . . 25
4.9 JOIN . . . 25
4.9.1 INNER JOIN vs. Prodotto Cartesiano + WHERE . . . 25
4.9.2 Esempio di LEFT JOIN . . . 26
4.9.3 ANTI JOIN . . . 26
4.10 Linee Guida Pratiche per Scrivere Query SQL . . . 26

5 Progettazione Concettuale: il Modello Entity-Relationship (ER) **27**
5.1 La Progettazione di una Base di Dati . . . 27
5.2 Concetti Fondamentali: Entità e Attributi . . . 28
5.2.1 Tipi di Attributo . . . 28
5.2.2 Attributi Chiave . . . 28
5.2.3 Rappresentazione Grafica . . . 28
5.3 Relazioni (Relationships) . . . 28
5.3.1 Grado di una Relazione . . . 28
5.3.2 Attributi della Relazione . . . 29
5.3.3 Relationship Type vs. Relationship Set . . . 29
5.4 Vincoli sulle Relazioni . . . 29
5.4.1 Vincolo di Chiave (key constraint) . . . 29
5.4.2 Vincoli di Cardinalità (cardinality ratio) . . . 29
5.4.3 Vincolo di Partecipazione (participation constraint) . . . 29
5.4.4 Notazione (min, max) . . . 30
5.5 Relazioni Ricorsive . . . 30
5.6 Entity Types Deboli . . . 30
5.7 Gerarchie di Classi (ISA) . . . 30
5.7.1 Vincoli: Overlap e Coverage . . . 31
5.8 Aggregazione . . . 31
5.9 Dai Requisiti al Modello: Regole Empiriche . . . 31

5.10 Scelte di Progettazione Concettuale con ER 32
5.10.1 Entità vs. Attributo 32
5.10.2 Entità vs. Relazione 32
5.10.3 Relazioni Binarie o Ternarie 32
5.10.4 Aggregazione o Relazione Ternaria? 33

6 Mapping dal Modello ER al Modello Relazionale 33
6.1 Obiettivi del Mapping 33
6.2 Da Entity Type a Tabelle 33
6.3 Mapping delle Relazioni (Relationship Types) 34
6.3.1 Caso Base (senza vincoli) 34
6.3.2 Relazioni Ricorsive 34
6.3.3 Vincolo di Chiave 34
6.3.4 Vincolo di Partecipazione 35
6.4 Weak Entities 36
6.5 Gerarchie IS-A 37
6.5.1 Metodo Generale 37
6.5.2 Metodo di Riduzione 37
6.5.3 Confronto tra i Due Metodi 38
6.6 Aggregazioni 38

7 Dipendenze Funzionali e Chiavi 38
7.1 Motivazione: Anomalie nella Base di Dati 38
7.2 Definizione Formale di Dipendenza Funzionale 39
7.3 Come si Identificano le DF 39
7.4 DF Banali 39
7.5 DF e Chiavi 39
7.6 DF e Anomalie 40
7.7 Chiusura di un Insieme di Dipendenze Funzionali 40
7.7.1 Assiomi di Armstrong 40
7.8 Chiusura di un Insieme di Attributi 40
7.9 Individuazione delle Superchiavi 41
7.9.1 Algoritmo per le Chiavi Candidate 41
7.10 Copertura Minimale 42

8 Normalizzazione 43
8.1 Normalizzazione via Decomposizione 43
8.2 Proprietà Desiderabili di una Decomposizione 43
8.2.1 Perdita di Informazione: Esempio 43
8.2.2 Condizione Formale per la Decomposizione Senza Perdita 43
8.2.3 Conservazione delle DF 43
8.3 Prima Forma Normale (1NF) 44
8.4 Richiami: Attributi Primi e Non-Primi 44
8.5 Seconda Forma Normale (2NF) 44
8.6 Terza Forma Normale (3NF) 45
8.7 3NF e Ridondanza Residua 45
8.8 Forma Normale di Boyce-Codd (BCNF) 45
8.8.1 Procedura di Decomposizione in BCNF 45
8.8.2 Quando la BCNF Non È Raggiungibile Preservando le DF 45

8.9 Verificare una DF su un'Istanza . . . 46
8.10 Esempio Completo: dalle Chiavi alla BCNF . . . 47

9 Metodi di Storage e Modello di Costo **47**
9.1 Introduzione . . . 47
9.2 Record e Blocchi del Disco . . . 48
9.3 Operazioni su File . . . 48
9.4 Tuple, Tabelle, Pagine . . . 48
9.5 Il Modello di Costo . . . 48
9.6 Metodi di Memorizzazione di una Relazione . . . 49
9.7 Heap File: Costo delle Operazioni Principali . . . 49
9.7.1 Stimare $|R_{A=c}|$: il Selectivity Factor . . . 49
9.8 Sorted File: Costo delle Operazioni Principali . . . 50

10 Indici **50**
10.1 Indicizzazione dei File . . . 50
10.1.1 Indici Primari, Secondari, Unici . . . 50
10.2 B+-Trees . . . 51
10.2.1 Costo del Lookup . . . 51
10.2.2 Costo delle Operazioni . . . 51
10.3 Hash Indexes . . . 52
10.3.1 Costo delle Operazioni . . . 52
10.4 Index Matching . . . 53
10.5 Tabella Riassuntiva dei Costi . . . 53

11 Algoritmi di JOIN e Costi **55**
11.1 Il JOIN . . . 55
11.2 Nested Loop Join (NLJ) . . . 55
11.3 (Sort-)Merge Join . . . 55
11.3.1 Costo, se R e S *non* sono già ordinate . . . 56
11.3.2 Algoritmi di External Merge Sort . . . 56
11.4 Hash Join . . . 56
11.5 Index Nested Loop Join . . . 56
11.6 Aggiustamento degli Indici . . . 57
11.7 Decisioni Progettuali a Livello Fisico . . . 57

12 Piani di Esecuzione delle Query **57**
12.1 Esecuzione di Query SQL . . . 57
12.2 Cosa Serve per il Calcolo . . . 58
12.3 Albero di Esecuzione della Query . . . 58
12.3.1 Esempio di Piano Fisico (illustrativo) . . . 58
12.4 Esempio Completo: Cardinalità e Costo . . . 58
12.4.1 Calcolo della Cardinalità dell'Output . . . 58
12.4.2 Calcolo del Costo (I/O) . . . 59
12.4.3 Variante con Indice . . . 59

13 Esercizi Supplementari da Appelli Precedenti **59**
13.1 Dipendenze Funzionali e Normalizzazione . . . 60
13.2 Vincoli di Integrità e Chiavi Esterne . . . 61

13.3 SQL . . . 62
13.4 Indici . . . 62
13.5 Ottimizzazione delle Query . . . 62
13.6 Algebra Relazionale . . . 62
13.7 Modello ER . . . 63
13.8 Viste . . . 63
13.9 Vincoli di Integrità e Chiavi . . . 64
13.10SQL . . . 64
13.11Dipendenze Funzionali e Normalizzazione (cont.) . . . 65
13.12Indici e Costi . . . 66
13.13Ottimizzazione delle Query (cont.) . . . 66
13.14Algebra Relazionale (cont.) . . . 67
13.15Note e Chiarimenti da Spiegazioni Tutor (2023) . . . 68

14 II Esercitazione sull'Algebra Relazionale 68
14.1 Schema di riferimento e convenzioni . . . 68
14.2 Richiami teorici sugli operatori . . . 69
14.3 Esercizio 1: formulare le interrogazioni in algebra relazionale . . . 70
14.3.1 Selezione, proiezione e join lungo le chiavi esterne (1.1–1.6) . . . 70
14.3.2 Confronto tra tuple e unione (1.7–1.8) . . . 73
14.3.3 Quantificazioni e negazione (1.9–1.14) . . . 75
14.4 Esercizio 2: descrivere in linguaggio naturale . . . 79
14.5 Riepilogo dei pattern ricorrenti . . . 81

15 Esercizi SQL: GROUP BY e HAVING su Marinai, Barche e Prenotazioni 82
15.1 Schema di riferimento, istanza e convenzioni . . . 82
15.2 Richiami teorici: aggregazione, GROUP BY e HAVING . . . 83
15.3 Quesiti . . . 84
15.4 Riepilogo dei pattern ricorrenti . . . 91

1 Le Basi di Dati e i loro Utenti

1.1 Cos'è una Base di Dati

Una **base di dati** (BdD) è una raccolta organizzata (strutturata) di informazioni che devono essere gestite per un periodo di tempo lungo, anche di molti anni. Più precisamente, una base di dati è una raccolta di informazioni gestita da un **database management system (DBMS)**.

Esempi di Basi di Dati:

- Dati relativi a studenti, corsi ed esami di un'università
- Il catalogo dei prodotti di un marketplace elettronico
- I *Points of Interest* (POI) su una mappa (es. Google Maps)
- Il Pubblico Registro Automobilistico (PRA) dell'Automobile Club Italiano (ACI)
- I pagamenti delle tasse dei contribuenti di un Comune
- Gli orari dei voli e le prenotazioni dei passeggeri di una compagnia aerea

1.2 DBMS e Database System

- **Database Management System (DBMS)**: pacchetto software progettato per gestire basi di dati informatizzate.
- **Database System**: l'insieme costituito dal software del DBMS insieme ai dati memorizzati nella base di dati. Talvolta nella definizione sono incluse anche le applicazioni software esterne che interagiscono con il DBMS per eseguire i propri compiti.

1.3 Architettura di un Database System

- Gli **utenti non esperti** (*unsophisticated*: clienti, agenti di viaggio, ...) interagiscono tramite *Web Forms* o *Application Front Ends*; gli **utenti esperti** (*sophisticated*: programmatori, amministratori) interagiscono tramite un'*SQL Interface*. Entrambi generano comandi SQL.
- Il **Query Evaluation Engine** riceve i comandi ed è composto da *Parser*, *Optimizer*, *Plan Executor* e *Operator Evaluator*.
- Il livello sottostante gestisce *Files and Access Methods*, il *Buffer Manager* e il *Recovery Manager*, coordinati dal blocco di **Concurrency Control** (*Transaction Manager* e *Lock Manager*).
- Alla base, il *Disk Space Manager* gestisce fisicamente gli **Index Files**, i **Data Files** e il **System Catalog** che compongono il database.

1.4 Cosa Deve Fornire un DBMS

Un DBMS deve:

- supportare la memorizzazione di grandi quantità di dati;
- gestire la robustezza dei dati e il loro recupero in caso di fallimenti, errori o azioni malevole;
- controllare l'accesso ai dati di più utenti contemporaneamente (**concorrenza**), evitando interazioni indesiderate tra utenti (*isolation*) e modifiche incomplete dei dati (*atomicity*).

Per la gestione dei dati sono previsti diversi linguaggi specializzati:

- **DDL** (*Data Definition Language*): definisce gli schemi per strutturare i dati;
- **DML** (*Data Manipulation Language*, o *query language*): modifica i dati e li estrae dalla base di dati;
- **DCL** (*Data Control Language*): gestisce l'accesso ai dati (`grant`, `revoke`, ...);
- **TCL** (*Transaction Control Language*): gestisce la sicurezza delle transazioni (`commit`, `rollback`, ...).

1.5 Interazione tra Applicazioni e Basi di Dati

I programmi applicativi interagiscono con la base di dati generando:

- **Interrogazioni** (*queries*): accesso a differenti porzioni di dati e formulazione della risposta;
- **Transazioni** (*transactions*): sequenze atomiche di azioni (letture/scritture) sulla base di dati.

Le applicazioni non devono consentire l'accesso ai dati ad utenti non autorizzati.

1.6 Le Origini del Modello Relazionale

Un enorme impulso allo sviluppo delle basi di dati è venuto dal lavoro di **Ted Codd** (1970), che ha introdotto l'idea di mostrare agli utenti una **visione logica dei dati**, indipendente dalla struttura fisica. Le interrogazioni vengono così espresse attraverso un linguaggio di alto livello, che utilizza il modello logico e non quello fisico dei dati.

1.7 Indipendenza dei Dati

La separazione tra programmi e dati è spesso chiamata **indipendenza programmi-dati**: permette di modificare le strutture dati e l'organizzazione fisica dei dati senza dover cambiare l'accesso dei programmi alla base di dati via DBMS.

Nei sistemi basati su file, invece, la struttura dei file di dati è contenuta nei programmi che devono accedervi: se cambia la struttura del file, è probabile che debbano essere modificati anche i programmi che vi accedono, rendendo difficile e costosa la manutenzione del software nel tempo.

1.7.1 Indipendenza Logica dei Dati

- **Definizione**: capacità di modificare lo schema logico del database senza dover alterare lo schema esterno o le applicazioni che lo utilizzano.
- **Vantaggi**: consente di aggiungere, rimuovere o modificare campi e tabelle senza influire sulle applicazioni esistenti; migliora la flessibilità e la gestione dell'evoluzione del database.
- **Esempio**: aggiungere un nuovo attributo a una tabella senza modificare il codice delle applicazioni che vi accedono.

1.7.2 Indipendenza Fisica dei Dati

- **Definizione**: capacità di modificare lo schema fisico del database senza influenzare lo schema logico o le applicazioni.
- **Vantaggi**: permette di ottimizzare le prestazioni (es. cambiando la struttura di memorizzazione o gli indici) senza alterare la logica delle applicazioni; migliora la scalabilità e la gestione delle risorse hardware.
- **Esempio**: spostare un database su un nuovo hardware, o ristrutturarne l'archiviazione, senza cambiare la modalità di accesso ai dati per l'utente finale.

1.8 Livelli di Astrazione dei Dati

- **Schema logico**: modello che rappresenta i dati nei termini del modello proprio del DBMS (es. il modello relazionale nei RDBMS);
- **Schema fisico**: dettagli su come le relazioni descritte dallo schema logico sono memorizzate su memoria secondaria (es. file di record ordinati/non ordinati, tipo di indici creati, ecc.);
- **Schema esterno**: una o più **viste** specifiche per velocizzare/facilitare/autorizzare l'accesso a insiemi di dati a specifici utenti o gruppi di utenti.

I tre livelli seguono il flusso: *livello fisico* $\rightarrow$ *livello logico* $\rightarrow$ *livello esterno (viste).*

Esempio. Schema logico per un DB universitario:

```
Students(sid: string, name: string, login: string, age: integer, gpa: real)
Faculty(fid: string, fname: string, sal: real)
Courses(cid: string, cname: string, credits: integer)
Rooms(rno: integer, address: string, capacity: integer)
Enrolled(sid: string, cid: string, grade: string)
Teaches(fid: string, cid: string)
Meets_In(cid: string, rno: integer, time: string)
```

- **Schema fisico**: file non ordinato di record, con un indice B+Tree sulla prima colonna di `STUDENTS` e `COURSES`.
- **Schema esterno**: una relazione che mostri tutti i corsi con relativo docente e numero di iscritti: `Courseinfo(cid: string, fname: string, enrollment: integer)`.

1.9 Perché Usare una Base di Dati

- Indipendenza logica e fisica dei dati (vista sopra);
- Accesso concorrente ai dati;
- Crash recovery;
- Integrità dei dati e sicurezza;
- Amministrazione dei dati;
- Riduzione dei tempi di sviluppo di applicazioni.

1.9.1 Condivisione dei Dati e Transazioni Multi-utente

La condivisione dei dati permette a gruppi di utenti di leggere e scrivere contemporaneamente (*concurrent users*) sulla stessa base di dati:

- il **controllo della concorrenza** nel DBMS garantisce che ogni transazione sia eseguita correttamente o annullata;
- il **sottosistema di recovery** assicura che ogni transazione correttamente completata sia memorizzata in modo permanente.

Questo permette di eseguire centinaia di transazioni concorrenti al secondo mantenendo validità e coerenza dei dati: si parla di **OLTP** (*Online Transaction Processing*).

1.10 Digressione: OLTP vs. OLAP

	OLTP	**OLAP**
Finalità	Gestire le transazioni quotidiane di un'azienda (es. ordini, pagamenti)	Supportare l'analisi complessa dei dati per BI e reporting
Operazioni	Alta frequenza, operazioni brevi e semplici (`insert`, `update`, `delete`)	Bassa frequenza, query complesse e di lunga durata
Priorità	Integrità/coerenza dei dati; risposta rapida alle query	Query ad-hoc, analisi multidimensionale
Schema	Normalizzato, per minimizzare la ridondanza	Denormalizzato (es. *star schema*), per le performance
Esempi	E-commerce, banche, punti vendita	Reporting aziendale, dashboard di BI

Tabella 1: Confronto tra sistemi OLTP e OLAP

Sistemi per OLTP:

- **Database Relazionali (RDBMS)**: MySQL, MariaDB, PostgreSQL, Microsoft SQL Server, IBM DB2, Oracle Database;

- **Software ERP**: SAP, Oracle, Panthera, TeamSystem, ...

Sistemi per OLAP:

- **Data Warehousing Platforms**: Amazon Redshift, Google BigQuery, Snowflake, Microsoft Azure Synapse Analytics;
- **Business Intelligence Tools**: Tableau, Power BI, Qlik Sense, SAP BusinessObjects.

1.11 Utenti di una Base di Dati

Gli utenti si dividono in due gruppi principali:

- chi utilizza e controlla il contenuto della base di dati, insieme a chi progetta e sviluppa applicazioni su di essa (**utilizzatori**);
- chi progetta e sviluppa il software del DBMS con tutti i suoi sottosistemi (**sviluppatori**).

1.11.1 Tipi di Utilizzatori

- **Database Administrator** (Amministratore): responsabile dell'autorizzazione di accesso alla base di dati, del coordinamento e monitoraggio del suo utilizzo, dell'acquisto di risorse software/hardware, del monitoraggio dell'efficienza operativa del sistema.
- **Database Designer** (Progettista): responsabile della definizione del contenuto, dello schema, dei vincoli e delle transazioni eseguibili sulla BD; comunica con gli utenti per raccoglierne le necessità.
- **End-users** (Utenti finali): utilizzano i dati mediante query e report; alcuni possono aggiornare i contenuti. Si suddividono in:
  - *Casual* (Occasionali): accedono alla base di dati solo occasionalmente, quando necessario (es. top manager che leggono dati fortemente aggregati);
  - *Naïve o Parametric* (non-esperti): la maggioranza degli utenti finali; usano funzioni predefinite (le *canned transactions*) per interagire con la BD (es. cassieri di banca, impiegati di segreteria, utenti di social media o applicazioni web).

1.12 Quando è Meglio NON Usare un DBMS

Cosa può inibirne l'adozione:

- alti investimenti iniziali e necessità di hardware dedicato;
- costi aggiuntivi per sicurezza, controllo di concorrenza, recovery, funzioni di integrità.

Quando un DBMS potrebbe non essere necessario:

- se la base di dati e l'applicazione sono molto semplici, ben definite e non soggette a modifiche continue;
- se non è richiesto l'accesso di più utenti.

Quando adottarlo può essere impossibile o sconsigliato:

- in molti *embedded systems*, dove mancano adeguate capacità di memorizzazione per un DBMS *general purpose*;
- con requisiti di tempo reale molto stringenti (es. sistemi di switch telefonico);
- quando il modello dati del DBMS non è abbastanza espressivo per la complessità dei dati (es. basi di dati genomiche/proteiche);
- se servono operazioni speciali non supportate dallo specifico DBMS (es. GIS e sistemi basati sulla localizzazione).

2 Il Modello Relazionale

2.1 Concetti Generali

Il modello relazionale fu proposto da **E.F. Codd** (IBM Research) nel 1970, nel famoso articolo *"A Relational Model for Large Shared Data Banks"* (Communications of the ACM, giugno 1970). Il modello si basa sul concetto matematico di **relazione**, fondato sulla teoria degli insiemi. Questo lavoro fu alla base di una profonda rivoluzione nel campo delle basi di dati, che portò Codd a vincere il prestigioso **ACM Turing Award**.

2.2 Cos'è una Relazione

Informalmente, una **relazione** può essere vista come una tabella con un insieme di valori su ogni riga. Due livelli la definiscono:

- lo **schema** della relazione (livello *intensionale*);
- le **istanze** della relazione (livello *estensionale*).

2.2.1 Definizione Intensionale

Lo schema di una relazione definisce:

- il nome della relazione (es. STUDENTI);
- il nome di ogni attributo (es. SID, NOME, LOGIN, ...);
- il dominio di ogni attributo (es. INTEGER, STRING, ...), ovvero l'insieme dei valori che quell'attributo può assumere.

L'ordine degli attributi nello schema non ha alcun significato specifico. Notazione standard:

```
Students(sid: string, name: string, login: string, age: integer, gpa: real)
```

Il numero di attributi definisce il **grado** (o *arietà*) della relazione (5 nell'esempio sopra).

2.2.2 Definizione Estensionale

Un'**istanza** di uno schema di relazione è un insieme di tuple (o record), ognuna con lo stesso numero di campi dello schema:

- essendo un insieme, **non possono esserci duplicati**;
- l'ordine delle tuple non ha alcun valore specifico.

Il numero di istanze (tuple) definisce la **cardinalità** della relazione.

2.3 Stato di una Relazione

Lo stato di una relazione è un sottoinsieme del prodotto cartesiano dei domini dei suoi attributi. Data una relazione $R(A_1, A_2, \ldots, A_n)$, uno specifico stato di R è definito come:

$$r(R) \subseteq dom(A_1) \times dom(A_2) \times \ldots \times dom(A_n)$$

ovvero $r(R) = \{t_1, t_2, \ldots, t_n\}$ dove ogni t_i è una tupla, $t_i = \langle v_1, v_2, \ldots, v_n \rangle$ e ogni v_j è un elemento di $dom(A_j)$. Nello stato di una relazione l'ordine delle tuple non conta (è un insieme!).

Esempio. Sia $R(A_1, A_2)$ con $dom(A_1) = \{0, 1\}$ e $dom(A_2) = \{a, b, c\}$. Il prodotto $dom(A_1) \times dom(A_2)$ è l'insieme di tutte le coppie ordinate possibili:

$$\{\langle 0, a\rangle, \langle 0, b\rangle, \langle 0, c\rangle, \langle 1, a\rangle, \langle 1, b\rangle, \langle 1, c\rangle\}$$

Un possibile stato $r(R)$ è $\{\langle 0, a\rangle, \langle 0, b\rangle, \langle 1, c\rangle\}$, con cardinalità 3.

2.4 Chiave di una Relazione

Ogni riga di una relazione ha un campo (o un insieme di campi) il cui valore identifica univocamente quella riga: si parla di **chiave** (*key*) della relazione. Talvolta si usano valori convenzionali per identificare una riga: si parla in questo caso di **chiavi artificiali** (*artificial key*) o **chiavi surrogate** (*surrogate key*).

2.5 Terminologia

Informale	**Formale**
Tabella	Relazione
Intestazione colonna	Attributo
Tutti i possibili valori di una colonna	Dominio
Riga della tabella	Tupla
Definizione della tabella	Schema della relazione
Tabella popolata	Stato della Relazione

Tabella 2: Corrispondenza tra terminologia informale e formale

2.6 Vincoli

I vincoli determinano quali stati di una relazione sono ammissibili. Sono di tre tipi:

1. **Vincoli impliciti**: dipendono dal data model stesso (es. il modello relazionale non ammette liste come valore di un attributo);
2. **Vincoli basati sullo schema** (o espliciti): definiti nello schema con gli strumenti forniti dal modello (es. un vincolo di integrità referenziale);
3. **Vincoli applicativi o semantici**: vanno oltre il potere espressivo del modello e devono essere imposti a livello di programma applicativo (es. un libro deve essere restituito entro 30 giorni dal prestito).

I principali vincoli espliciti nel modello relazionale sono:

- vincolo di dominio;
- vincolo di chiave;
- vincolo di integrità delle entità;
- vincolo di integrità referenziale.

2.7 Vincolo di Chiave

- **Superchiave** di R: insieme di attributi SK tale che non esistono due tuple distinte t_1, t_2 di $r(R)$ con $t_1[SK] = t_2[SK]$; la condizione deve valere in ogni stato valido di R.
- **Superchiave minimale** (o **chiave candidata**): una superchiave tale che rimuovendo un qualsiasi attributo si ottiene un insieme che non è più superchiave.
- Una chiave è sempre una superchiave, ma non viceversa.

Esempio. La relazione STUDENT ha due chiavi candidate: $\{sid\}$ e $\{login\}$. L'insieme $\{sid, name\}$ è una superchiave ma *non* una chiave (contiene una chiave, ma non è minimale). Ogni insieme di attributi che contiene una chiave è ovviamente una superchiave.

Se una relazione ha più chiavi candidate, una viene scelta (tipicamente dal DBA) come **chiave primaria**: in genere la più piccola per numero di attributi; quando ciò non è possibile, possono intervenire fattori soggettivi. I valori della chiave primaria identificano univocamente ogni tupla e possono essere usati per riferirsi ad essa da un'altra relazione.

2.8 Vincolo di Integrità dell'Entità (Entity Integrity)

Nessuno degli attributi che compongono la chiave primaria PK di una relazione R può avere valore `NULL` in alcuna tupla di $r(R)$:

$$t[PK] \neq null \quad \forall t \in r(R)$$

Se PK ha più attributi, `NULL` non è ammesso per nessuno di essi. È comunque possibile imporre che anche altri attributi (non chiave primaria) non possano assumere valore `NULL`.

2.9 Vincoli di Integrità Referenziale

A differenza degli altri, un vincolo di integrità referenziale coinvolge **due relazioni**: una relazione **referenziante** R_1 e una **referenziata** R_2. In R_1 un insieme di attributi FK (**chiave esterna**, *Foreign Key*) fa riferimento alla chiave primaria PK di R_2: una tupla $t_1 \in R_1$ referenzia $t_2 \in R_2$ se $t_1[FK] = t_2[PK]$.

Esempio. `Enrolled` è la relazione referenziante, `Students` quella referenziata: `Enrolled.sid` è la chiave esterna che referenzia `Students.sid`.

I valori della FK possono essere:

1. uno dei valori del corrispondente attributo PK in R_2, oppure
2. il valore `NULL` (ma solo se la FK non fa parte della chiave primaria di R_1).

Osservazioni:

- la FK può fare riferimento alla *stessa* relazione a cui appartiene la PK;
- gli attributi della FK non devono necessariamente avere lo stesso nome dei corrispondenti attributi della PK.

Esempio: database COMPANY. Vincoli di integrità referenziale tipici in uno schema aziendale:

- `EMPLOYEE.Dno` → `DEPARTMENT.Dnumber`
- `EMPLOYEE.Super_ssn` → `EMPLOYEE.Ssn` *(la FK referenzia la stessa relazione della PK — vedi Osservazione 1)*
- `DEPARTMENT.Mgr_ssn` → `EMPLOYEE.Ssn`
- `DEPT_LOCATIONS.Dnumber` → `DEPARTMENT.Dnumber`
- `PROJECT.Dnum` → `DEPARTMENT.Dnumber`
- `WORKS_ON.Essn` → `EMPLOYEE.Ssn`; `WORKS_ON.Pno` → `PROJECT.Pnumber`
- `DEPENDENT.Essn` → `EMPLOYEE.Ssn`

Esercizio 1

Dati gli schemi $R(A, B)$, $S(B, C)$ con `Primary Key (A)` e `Foreign Key (B) references S(B)`, cosa si può concludere?

a) A non può essere `NULL` ma B sì

b) A e B possono essere `NULL`

c) La primary key è parte della dichiarazione di R, la foreign key è nella dichiarazione di S

d) B è una chiave primaria o una chiave candidata per S

Soluzione: d) — per definizione, una chiave esterna deve fare riferimento a una chiave primaria (o comunque candidata) della relazione referenziata, altrimenti il vincolo di integrità referenziale non sarebbe ben definito.

2.10 Schema di un Database Relazionale

I vincoli di integrità referenziale coinvolgono più di una relazione: non basta quindi considerare lo schema di una singola relazione. Uno **schema di database relazionale** è un insieme S di schemi di relazione appartenenti alla stessa base di dati:

$$S = \langle \{R_1, R_2, \ldots, R_n\}, IC \rangle$$

dove $R_1, \ldots, R_n$ sono i nomi degli schemi di relazione e IC è l'insieme dei vincoli di integrità.

2.11 Altri Tipi di Vincolo

I **vincoli di integrità semantica** si riferiscono al significato dell'applicazione e non possono essere espressi nel modello in sé (es. "il numero massimo di ore che un dipendente può lavorare su tutti i progetti è di 46 ore per settimana"). Serve quindi un linguaggio per la specifica di vincoli: SQL-99 introduce *trigger* e *asserzioni*, non implementati in modo uniforme da tutti i DBMS. In MySQL si può usare `CHECK`:

```
CREATE TABLE Person (...,
    Age int CHECK (Age >= 18));
```

2.12 Stato di una Base di Dati Relazionale

Uno stato di una base di dati relazionale con schema S è un insieme di stati di relazione $\{r_1, r_2, \ldots, r_m\}$ tale che ogni r_i è uno stato di R_i e soddisfa i vincoli di integrità in IC. Uno stato di una base di dati è talvolta chiamato **istantanea** (*snapshot*) o **istanza** (*instance*). Uno stato che non rispetta i vincoli in IC è uno **stato non valido**.

2.13 Operazioni di Aggiornamento e Violazione dei Vincoli

Le principali operazioni per cambiare lo stato di una base di dati sono:

- **INSERT**: inserimento di una nuova tupla;
- **DELETE**: cancellazione di una tupla;
- **MODIFY**: modifica di un attributo di una tupla.

Queste operazioni non devono mai portare alla violazione di un vincolo di integrità: garantire questa condizione può richiedere di propagare automaticamente gli aggiornamenti. Se si verifica una violazione, le opzioni possibili sono:

- **RESTRICT** (*NO ACTION*, *REJECT*): impedire l'operazione;
- **CASCADE**, **SET NULL**, **SET DEFAULT**: avviare altri aggiornamenti per eliminare la violazione;
- eseguire routine specifiche definite dall'utente.

2.13.1 Violazioni per Operazione INSERT

L'INSERT può violare **tutti** i vincoli:

- **dominio**: il valore di un attributo non appartiene al dominio specificato;
- **chiave**: inserimento di una tupla il cui valore di chiave esiste già;
- **integrità referenziale**: valore di FK che non esiste come valore di PK nella relazione referenziata;
- **integrità dell'entità**: valore della chiave primaria della nuova tupla è `NULL`.

Esercizio 2

Un'operazione di `INSERT` in una relazione può produrre la violazione di quale vincolo? (Dominio / Chiave / Integrità referenziale / Integrità delle entità / Nessuna / Tutte)

Soluzione: Tutte — come visto sopra, l'INSERT è l'unica operazione che può potenzialmente violare ciascuno dei quattro vincoli contemporaneamente.

2.13.2 Violazioni per Operazione DELETE

Il DELETE può violare **solo** vincoli di integrità referenziale, se si rimuove una tupla la cui chiave primaria è referenziata da altre tuple.

Esercizio 3

Dato lo schema `Enrolled(sid, cid, grade)` e `Students(sid, name, login, age, gpa)`, con `sid` chiave esterna di `Enrolled` verso `Students(sid)`: la cancellazione di una qualsiasi tupla nella tabella `ENROLLED` non può produrre la violazione di alcun vincolo. Vero o Falso?

Soluzione: Vero — `Enrolled` è la relazione "figlia" (referenziante): eliminare un suo record non viola mai l'integrità referenziale. La violazione scatterebbe solo cancellando un record "padre" in `Students` ancora referenziato da altre tuple.

2.13.3 Violazioni per Operazione UPDATE

L'UPDATE può violare tutti i vincoli, a seconda dell'attributo modificato:

- update della **chiave primaria**: integrità referenziale;
- update di una **chiave esterna**: integrità referenziale;
- update di un attributo ordinario: vincoli di dominio, o vincoli `UNIQUE/NOT NULL`.

2.14 Come Preservare l'Integrità Referenziale

- **RESTRICT** (*NO ACTION*): rifiuta l'operazione;
- **CASCADE**: cancella tutte le tuple che referenziavano la PK della tupla cancellata/modificata;

- **SET NULL**: assegna `NULL` alla FK delle tuple che referenziavano la tupla cancellata/modificata;
- **SET DEFAULT**: assegna un valore di default alla FK delle tuple che referenziavano la tupla cancellata/modificata.

Attenzione: le opzioni `SET NULL` e `SET DEFAULT` *non* sono disponibili se la chiave esterna include attributi che sono anche chiave primaria della tabella referenziante (altrimenti si violerebbe l'integrità dell'entità).

Esempio in SQL.

```
CREATE TABLE Enrolled (
    studid  CHAR(20),
    cid     CHAR(20),
    grade   CHAR(10),
    PRIMARY KEY (studid, cid),
    FOREIGN KEY (studid) REFERENCES Students
        ON DELETE CASCADE
        ON UPDATE NO ACTION
);
```

3 Algebra Relazionale

3.1 Introduzione

L'**algebra relazionale** è un linguaggio di interrogazione formale e **procedurale**: ogni operatore prende in input una o due relazioni e produce in output una relazione (proprietà di **chiusura**). Questo permette di comporre e annidare espressioni, applicando un operatore al risultato di un altro. L'algebra relazionale è alla base dei linguaggi pratici come SQL.

Operatori di base (un insieme completo, da cui derivano tutti gli altri):

- Selezione (σ), Proiezione (π);
- Unione ($\cup$), Differenza ($-$);
- Prodotto Cartesiano ($\times$);
- Ridenominazione (ρ).

Operatori derivati (esprimibili in funzione dei precedenti, ma usati per comodità):

- Intersezione ($\cap$);
- Join ($\bowtie$): theta-join, equijoin, join naturale;
- Divisione ($\div$).

3.2 Schema e Dati di Riferimento

Negli esempi che seguono si userà questo schema:

```
Sailors(sid: integer, sname: string, rating: integer, age: real)
Boats(bid: integer, bname: string, color: string)
Reserves(sid: integer, bid: integer, day: date)
```

con le seguenti istanze di esempio:

S1 (istanza di Sailors)

sid	sname	rating	age
22	dustin	7	45.0
31	lubber	8	55.5
58	rusty	10	35.0

S2 (istanza di Sailors)

sid	sname	rating	age
28	yuppy	9	35.0
31	lubber	8	55.5
44	guppy	5	35.0
58	rusty	10	35.0

R1 (istanza di Reserves)

sid	bid	day
22	101	10/10/96
58	103	11/12/96

Tabella 3: Istanze di esempio usate negli operatori seguenti

3.3 Selezione (SELECT, σ)

L'operatore di **selezione** σ permette di selezionare un sottoinsieme delle tuple di una relazione, applicando a ciascuna di esse una formula booleana F.

Espressione: $\sigma_F(R)$	
Schema	$R(X) \to X$
Istanza	$r \to \sigma_F(r) = \{t \mid t \in r \text{ AND } F(t) = \text{vero}\}$

Tabella 4: Definizione formale di SELECT

F si compone di predicati connessi da AND ($\wedge$), OR ($\vee$) e NOT ($\neg$). Ogni predicato è del tipo $A\,\theta\,c$ oppure $A\,\theta\,B$, dove:

- A e B sono attributi in X;
- $c \in dom(A)$ è una costante;
- θ è un operatore di confronto, $\theta \in \{=, \neq, <, >, \leq, \geq\}$.

Esempio. Trovare in S2 tutti i sailor che hanno giudizio superiore a 8: $\sigma_{rating>8}(S2)$.

Risultato: $\{\langle 28, \text{yuppy}, 9, 35.0\rangle, \langle 58, \text{rusty}, 10, 35.0\rangle\}$ — stesso numero di attributi, selezionate le due (sole) tuple che soddisfano *rating > 8*.

3.4 Proiezione (PROJECT, π)

L'operatore di **proiezione** π estrae, da ogni tupla della relazione di input, solo gli attributi specificati, eliminando le colonne non richieste. Essendo il risultato un insieme, eventuali **tuple duplicate vengono eliminate**: per questo il numero di tuple del risultato può essere inferiore a quello della relazione di partenza.

Esempio. Trovare nome e giudizio di tutti i sailor (istanza S2): $\pi_{sname,rating}(S2)$.
Risultato: $\{\langle \text{yuppy}, 9\rangle, \langle \text{lubber}, 8\rangle, \langle \text{guppy}, 5\rangle, \langle \text{rusty}, 10\rangle\}$.

3.5 Unione, Differenza e Intersezione

Questi operatori insiemistici richiedono che le due relazioni operande siano **union-compatibili**: stesso numero di attributi, con domini corrispondenti compatibili.

- $R \cup S$: tutte le tuple che appartengono a R o a S (o entrambe);
- $R - S$: le tuple che appartengono a R ma non a S;
- $R \cap S$ (*derivato*): le tuple che appartengono sia a R sia a S; $R \cap S = R - (R - S)$.

$\cup$ e $\cap$ sono **commutativi**; $-$ **non** lo è.

3.6 Prodotto Cartesiano ($\times$)

$R \times S$ combina **ogni** tupla di R con **ogni** tupla di S: lo schema del risultato è la concatenazione degli attributi di R e di S (con $|R| \cdot |S|$ tuple in output). Se R e S hanno un attributo con lo stesso nome, questo compare due volte nello schema risultato, creando ambiguità.

Esempio. $S1 \times R1$: 6 tuple (3×2), con la colonna `sid` duplicata (una da S1, una da R1) — l'ambiguità che motiva l'operatore di ridenominazione.

3.7 Ridenominazione (RENAME, ρ)

L'operatore ρ rinomina la relazione e/o i suoi attributi, senza modificare i dati, per risolvere ambiguità come quella del prodotto cartesiano.

Esempio. $\rho(C(1 \to sid1,\ 5 \to sid2),\ S1 \times R1)$: rinomina il 1° attributo (sid da S1) in `sid1` e il 5° (sid da R1) in `sid2`.

3.8 Join

3.8.1 Theta-Join

Il **theta-join** $R \bowtie_c S$ equivale a $\sigma_c(R \times S)$: un prodotto cartesiano seguito da una selezione secondo una condizione c qualsiasi (non solo uguaglianze). Lo schema del risultato mantiene **entrambi** gli attributi coinvolti nella condizione, anche se omonimi.

Esempio. $S1 \bowtie_{S1.sid<R1.sid} R1$: delle 6 combinazioni possibili, solo 2 soddisfano la condizione $sid_{S1} < sid_{R1}$: $\langle 22, \text{dustin}, 7, 45.0, 58, 103, 11/12/96\rangle$ e $\langle 31, \text{lubber}, 8, 55.5, 58, 103, 11/12/96\rangle$.

3.8.2 Join Naturale

	Espressione: $R_1 \bowtie R_2$
Schema	$R_1(X_1), R_2(X_2) \to X_1 \cup X_2$
Istanza	$r_1, r_2 \to r_1 \bowtie r_2 = \{t \mid t[X_1] \in r_1 \text{ AND } t[X_2] \in r_2\}$

Tabella 5: Definizione formale del join naturale

Ogni tupla del risultato combina (*match*) una tupla di r_1 con una di r_2 sulla base dell'**uguaglianza** dei valori degli attributi comuni ($X_1 \cap X_2$); questi attributi compaiono **una sola volta** nello schema risultato.

Osservazioni:

- una tupla che non trova corrispondenza nell'altra relazione si dice **dangling**;
- la cardinalità del join è compresa tra 0 e $|r_1| \cdot |r_2|$;
- se il join è eseguito su una superchiave di $R_1(X_1)$, allora $|r_1 \bowtie r_2| \leq |r_2|$;
- se $X_1 \cap X_2$ è chiave primaria di $R_1(X_1)$ e chiave esterna in $R_2(X_2)$ (vincolo di integrità referenziale), allora $|r_1 \bowtie r_2| = |r_2|$.

3.8.3 Equijoin

È la forma di join più comunemente utilizzata: la condizione c include **soltanto** confronti di uguaglianza (=).

Esempio. $S1 \bowtie_{sid=sid} R1$: risultato $\{\langle 22, \text{dustin}, 7, 45.0, 101, 10/10/96\rangle, \langle 58, \text{rusty}, 10, 35.0, 103, 11/12/$
(la tupla con $sid = 31$ è *dangling*: nessuna corrispondenza in R1).

3.9 Divisione (÷)

Dati $A(x, y)$ e $B(y)$, $A \div B$ restituisce l'insieme delle tuple x tali che, per **ogni** tupla y in B, la coppia (x, y) compare in A. Caso d'uso tipico: "trovare i sailor che hanno prenotato **tutte** le barche in un certo insieme".

Esempio. Dato $A(sno, pno)$ con le coppie $\{s_1p_1, s_1p_2, s_1p_3, s_1p_4, s_2p_1, s_2p_2, s_3p_2, s_4p_2, s_4p_4\}$:

- $B_1 = \{p_2\} \Rightarrow A/B_1 = \{s_1, s_2, s_3, s_4\}$ (tutti prenotano p_2);
- $B_2 = \{p_2, p_4\} \Rightarrow A/B_2 = \{s_1, s_4\}$ (solo loro prenotano **sia** p_2 **sia** p_4);
- $B_3 = \{p_1, p_2, p_4\} \Rightarrow A/B_3 = \{s_1\}$ (solo s_1 prenota tutte e tre).

3.10 Outer Join

A differenza del join (naturale o theta), l'**outer join** conserva anche le tuple *dangling*, riempiendo con `NULL` gli attributi mancanti:

- **LEFT OUTER JOIN**: conserva le tuple dangling della relazione di sinistra;
- **RIGHT OUTER JOIN**: conserva le tuple dangling della relazione di destra;
- **FULL OUTER JOIN**: conserva le tuple dangling di entrambe le relazioni.

3.11 Espressioni ed Alberi di Query

Un'espressione di algebra relazionale può essere rappresentata come un **albero di query**: le foglie sono le relazioni di base, i nodi interni sono gli operatori, e la valutazione procede dal basso verso l'alto. Espressioni algebricamente equivalenti (stesso risultato per ogni istanza valida) possono corrispondere ad alberi diversi con costi di esecuzione molto differenti: è la base per l'**ottimizzazione delle query**, approfondita quando si tratterà l'analisi dei costi.

4 SQL

4.1 Cos'è SQL

SQL (*Structured Query Language*) è un linguaggio specializzato per la gestione di basi di dati relazionali: standardizzato e ampiamente utilizzato, permette la creazione, modifica e interrogazione dei dati. Si divide in:

- **DDL** (*Data Definition Language*): definisce la struttura della base di dati (creazione tabelle, modifiche allo schema, definizione di vincoli);
- **DML** (*Data Manipulation Language*): manipola i dati (inserimento, aggiornamento, cancellazione, interrogazione).

4.2 Istanza di Riferimento

Negli esempi che seguono si usa questa istanza (tratta dal libro di testo):

Sailors (istanza S3)

sid	sname	rating	age
22	Dustin	7	45.0
29	Brutus	1	33.0
31	Lubber	8	55.5
32	Andy	8	25.5
58	Rusty	10	35.0
64	Horatio	7	35.0
71	Zorba	10	16.0
74	Horatio	9	35.0
85	Art	3	25.5
95	Bob	3	63.5

Boats (istanza B1)

bid	bname	color
101	Interlake	blue
102	Interlake	red
103	Clipper	green
104	Marine	red

Tabella 6: Istanze di riferimento per gli esempi SQL (`Reserves`, istanza R2, omessa qui: 10 tuple, 4 per sid 22, 3 per sid 31, 2 per sid 64, 1 per sid 74, verificabili dal risultato della query in Sez. 4.9.2)

4.3 Struttura Base di una Query

```
SELECT   [DISTINCT] target-list
FROM     relation-list
WHERE    qualification
```

- **relation-list**: lista di nomi di relazioni (talvolta con *variabili di range*) — da 1 a n;
- **target-list**: lista di attributi che appartengono alle relazioni nella relation-list;
- **qualification**: predicati di confronto, eventualmente combinati con AND, OR o NOT;
- **DISTINCT**: chiave opzionale per eliminare i duplicati dal risultato (il **default è che NON vengono eliminati**).

4.3.1 Semantica Valutativa (concettuale)

1. si calcola il prodotto cartesiano delle tabelle nella relation-list;
2. si scartano le tuple che non soddisfano la qualification;
3. si eliminano le colonne degli attributi che non appartengono alla target-list;
4. se presente DISTINCT, si eliminano i duplicati.

Nota: questa è una strategia concettuale, non quella realmente usata dal DBMS — è molto inefficiente; il modulo di *query optimization* costruisce piani di esecuzione più efficienti ma semanticamente equivalenti.

Esempio.

```
-- Q0: sname ed eta' di tutti i marinai
select s.sname, s.age
from sailors s;

-- Q0.1: sname ed eta' dei marinai con piu' di 50 anni
select s.sname, s.age
from sailors s
where s.age > 50;
```

Risultato di Q0.1 sull'istanza S3: $\{\langle \text{Lubber}, 55.5\rangle, \langle \text{Bob}, 63.5\rangle\}$ — le uniche due tuple con *age* > 50.

4.4 Operatori di Confronto

- aritmetici: `=`, `<`, `>`, `>=`, `<=`, `<>`;
- `BETWEEN`: valore compreso in un intervallo;
- `IN`: valore compreso in una lista;
- `LIKE`: confronto fra stringhe con caratteri jolly (`%` e `_`);
- `IS [NOT] NULL`: verifica se un valore ha (o non ha) valore `NULL`.

Attenzione: `WHERE x = NULL` è **sempre sbagliato** — va usato `WHERE x IS NULL`.

Esercizio 1

Quale dei seguenti operatori è usato per verificare se il valore di un attributo è uguale a `NULL` in SQL? (nessuna delle opzioni / `IS` / `IN` / `=`)

Soluzione: `IS` (cioè `IS NULL`) — l'operatore `=` non funziona mai con `NULL`, per via della logica a tre valori di SQL (vedi Esercizio 2).

4.5 Operatori di Aggregazione

SQL supporta 5 operatori di aggregazione, che vanno oltre gli operatori dell'algebra relazionale:

- `count(*)`: numero di tuple restituite da una query;
- `count([DISTINCT] A)`: numero di valori (unici) di A;
- `sum([DISTINCT] A)`: somma dei valori (unici) di A;
- `avg([DISTINCT] A)`: media dei valori (unici) di A;
- `MAX(A)` / `MIN(A)`: valore più alto / più basso della colonna A.

Nota: gli operatori di aggregazione ignorano i valori `NULL` nella colonna aggregata (eccetto `count(*)`, che conta le tuple indipendentemente dai NULL).

4.6 GROUP BY e HAVING

Quando serve applicare l'aggregazione non a tutte le tuple ma a **gruppi** di tuple (es. "quante volte è stata noleggiata ogni barca", "età media dei marinai per ogni rating"), si estende la struttura base:

```
SELECT   [DISTINCT] select-list
FROM     from-list
WHERE    qualification
GROUP BY grouping-list
HAVING   group-qualification
```

- la **select-list** è composta da (1) nomi di colonne e (2) termini `agg-op(colonna) AS nuovo_nome`;
- ogni colonna in (1) **deve** comparire anche nella grouping-list (altrimenti potrebbe avere valori diversi nello stesso gruppo);
- le espressioni nella group-qualification devono avere un solo valore per gruppo;
- senza GROUP BY, l'intera tabella è considerata un singolo gruppo.

Valutazione (concettuale):

1. prodotto cartesiano delle tabelle nel FROM;

2. applicazione del WHERE;
3. eliminazione delle colonne non richieste (non in select-list/grouping-list/group-qualification);
4. riordino in base al GROUP BY, per identificare i gruppi;
5. applicazione della group-qualification nella clausola HAVING;
6. generazione di una riga di risposta per ogni gruppo rimanente.

4.6.1 Esempio (Q32)

Trovare, per ogni livello di rating con almeno due marinai maggiorenni (age $\geq$ 18), l'età del più giovane tra questi.

```
SELECT   S.rating, MIN(S.age) AS minage
FROM     Sailors S
WHERE    S.age >= 18
GROUP BY S.rating
HAVING   COUNT(*) > 1
```

Sull'istanza S3: il WHERE esclude Zorba (16.0, minorenne). Raggruppando per rating: rating 1 (Brutus, 1 sailor), rating 3 (Art 25.5, Bob 63.5 — 2 sailor), rating 7 (Dustin 45.0, Horatio 35.0 — 2 sailor), rating 8 (Lubber 55.5, Andy 25.5 — 2 sailor), rating 9 (Horatio 35.0, 1 sailor), rating 10 (Rusty 35.0, 1 sailor). L'HAVING scarta i gruppi con un solo sailor. **Risultato**: $\{\langle 3, 25.5\rangle, \langle 7, 35.0\rangle, \langle 8, 25.5\rangle\}$.

4.7 Interrogazioni Annidate (Nested Queries)

Una **query annidata** (o *nested*) è una query (*interna*) che compare dentro un'altra (*esterna*). Semanticamente è simile a un *nested loop*: per ogni tupla della query esterna, si valuta il WHERE eseguendo la query interna. Una query annidata può restituire:

- un **singolo valore**, da confrontare con un attributo della query esterna;
- un **insieme di valori**, tipicamente con `IN`, `EXISTS`, `ANY` o `ALL`.

4.7.1 Query Indipendenti e Correlate

- **Indipendente**: eseguibile da sola, senza riferimenti a colonne/valori della query esterna; viene eseguita **una sola volta**.
- **Correlata**: la query interna fa riferimento a colonne della query esterna; deve essere **rieseguita per ogni riga** della query esterna.

4.7.2 Casi d'Uso

- filtrare con un valore calcolato dinamicamente (es. marinai con rating minore del rating medio);
- confrontare con un risultato derivato (es. il marinaio con il rating più alto);

- verificare l'esistenza (o assenza) di certi valori (`EXISTS`; es. marinai che non hanno mai prenotato);
- aggregazione condizionale (es. rating dei marinai con meno di 2 prenotazioni);
- unire il risultato di una subquery con altre tabelle via JOIN.

4.7.3 Annidamento a Più Livelli e Operatori ANY/ALL

È possibile annidare più di un livello. Con `ANY`: la condizione è vera se vale per **almeno una** tupla del risultato della subquery (se la subquery è vuota, `ANY` restituisce FALSE). Con `ALL`: la condizione deve valere per **tutte** le tuple (se la subquery è vuota, `ALL` restituisce TRUE — vacuous truth).

4.8 Operatori Insiemistici

- `UNION` (o `UNION ALL`): unione di due insiemi di tuple compatibili; `UNION` elimina i duplicati, `UNION ALL` no;
- `INTERSECT`: tuple comuni a entrambi gli insiemi;
- `EXCEPT`: tuple del primo insieme che non compaiono nel secondo (differenza insiemistica).

Le subquery possono comparire in diversi punti di una query: nel `FROM` (per costruire una tabella temporanea), o nel `WHERE` (per calcolare al volo un valore o un insieme con cui confrontare).

4.9 JOIN

Le 4 forme principali:

1. **(INNER) JOIN**: solo le tuple che soddisfano la condizione in **entrambe** le tabelle;
2. **LEFT (OUTER) JOIN**: tutte le tuple della tabella di sinistra, più quelle di destra che soddisfano la condizione (NULL altrimenti);
3. **RIGHT (OUTER) JOIN**: simmetrico, tutte le tuple della tabella di destra;
4. **FULL (OUTER) JOIN**: tutte le tuple di entrambe le tabelle a fronte di un match in almeno una[1].

4.9.1 INNER JOIN vs. Prodotto Cartesiano + WHERE

Un INNER JOIN si può scrivere in due modi equivalenti: con l'operatore esplicito `JOIN ... ON`, oppure mettendo le tabelle nel FROM e la condizione nel WHERE — restituiscono **esattamente lo stesso risultato**. La forma `JOIN...ON` è raccomandata per chiarezza (separa la condizione di join dagli altri filtri), ma nella pratica sono intercambiabili. **Non** lo sono, invece, quando serve un OUTER JOIN.

[1] MariaDB (come molti altri DBMS) **non supporta** il FULL OUTER JOIN.

4.9.2 Esempio di LEFT JOIN

Quante prenotazioni ha fatto ogni marinaio (inclusi quelli con zero prenotazioni)?

```
select s.sid, count(r.sid)
from sailors s left join reserves r
on s.sid = r.sid
group by s.sid;
```

Con un INNER JOIN o un prodotto cartesiano, i marinai senza prenotazioni (dangling) sparirebbero dal risultato: serve il LEFT JOIN per conservarli con conteggio 0. **Risultato** su S3/R2: $\langle 22, 4\rangle, \langle 29, 0\rangle, \langle 31, 3\rangle, \langle 32, 0\rangle, \langle 58, 0\rangle, \langle 64, 2\rangle, \langle 71, 0\rangle, \langle 74, 1\rangle, \langle 85, 0\rangle, \langle 95, 0\rangle$.

4.9.3 ANTI JOIN

I casi che restituiscono solo le tuple **senza** corrispondenza nell'altra relazione si chiamano LEFT/RIGHT/FULL **ANTI JOIN**. Poco usati nello sviluppo applicativo, ma utili in business intelligence/data science (es. "tutti i marinai che non hanno mai noleggiato una barca").

4.10 Linee Guida Pratiche per Scrivere Query SQL

1. **Studia lo schema**: comprendi tabelle/attributi, domini, vincoli di integrità referenziale (serviranno per i JOIN);
2. **Leggi bene la richiesta**: individua gli attributi in output ($\rightarrow$ target-list), la tabella di provenienza di ciascuno ($\rightarrow$ relation-list), i filtri necessari;
3. **Cerca le aggregazioni**: parole come "valore medio" ($\rightarrow$ AVG), "il totale" ($\rightarrow$ SUM), "il numero di" ($\rightarrow$ COUNT) suggeriscono operatori di aggregazione, eventualmente con GROUP BY/HAVING;
4. **Comprendi i vincoli di integrità referenziale**: individua gli attributi per il JOIN (tipicamente PK$\rightarrow$FK) e il tipo di JOIN semanticamente corretto (INNER, OUTER, ANTI, ...) — ricorda che il WHERE opera tupla per tupla;
5. **Cerca le query annidate**: confronto con valore dinamico, appartenenza a un insieme ($\rightarrow$ IN), confronto con insieme di valori ($\rightarrow$ ANY/ALL), verifica di esistenza ($\rightarrow$ EXISTS);
6. **Individua eventuali operazioni insiemistiche**: unione di tuple con stesso schema da tabelle diverse ($\rightarrow$ UNION), tuple comuni ($\rightarrow$ INTERSECT), tuple in un insieme ma non nell'altro ($\rightarrow$ EXCEPT);
7. **Verifica la coerenza semantica del risultato**: scomponi la query in parti più semplici, controlla filtri/cardinalità/colonne restituite, non dimenticare DISTINCT o l'ordinamento se richiesti.

Esercizio 2

Si assuma che la relazione $R(A, B)$ abbia le dichiarazioni `PRIMARY KEY (A)` e `UNIQUE (B)`. Quale delle seguenti affermazioni è vera? (A può essere NULL, B no / Sia A che B possono essere NULL / A e B non possono essere NULL / A non può essere NULL, B sì)

Soluzione: A non può essere NULL, B sì — il vincolo `PRIMARY KEY` implica sempre `NOT NULL` (integrità dell'entità), mentre `UNIQUE` vieta solo i duplicati tra valori non-NULL: la maggior parte dei DBMS ammette (tipicamente) un valore NULL in una colonna UNIQUE, proprio perché NULL non è mai considerato uguale a un altro NULL.

Esercizio 3

Data la tabella R:

A	B	C
1	2	6
NULL	NULL	3
4	NULL	NULL
5	6	NULL
NULL	NULL	NULL

Qual è il risultato di `select C from R where A=B`? (l'insieme vuoto / NULL / 3, NULL / 3)

Soluzione: l'insieme vuoto. Per la logica a tre valori di SQL, `A=B` è vero solo se **entrambi** non sono NULL **e** uguali. Riga 1: $1 = 2$ falso. Righe 2 e 5: NULL=NULL $\rightarrow$ `UNKNOWN` (non vero). Riga 3: 4 =NULL $\rightarrow$ `UNKNOWN`. Riga 4: $5 = 6$ falso. Nessuna riga soddisfa la condizione: il WHERE non include mai le righe con esito `UNKNOWN`, solo quelle con esito `TRUE`.

5 Progettazione Concettuale: il Modello Entity-Relationship (ER)

5.1 La Progettazione di una Base di Dati

Le fasi principali della progettazione di una BdD sono:

1. Analisi dei requisiti;
2. Progettazione concettuale;
3. Progettazione logica;
4. Raffinamento dello schema logico (**normalizzazione**);
5. Progettazione del livello fisico;
6. Progettazione applicativa e sicurezza.

Per le fasi 1-3 ci si chiede *quali dati* devono essere raccolti; per le fasi 4-6, *quali applicazioni* useranno questi dati e quali saranno le operazioni più frequenti/costose.

Un **modello** è una rappresentazione semplificata di una porzione di mondo (*mini-mondo*); è *epistemologicamente adeguato* se permette di esprimere concretamente i fatti noti del mini-mondo di interesse (McCarthy & Hayes, 1969). Per le basi di dati, il modello più usato per la progettazione concettuale è il modello **Entity-Relationship (ER)**.

5.2 Concetti Fondamentali: Entità e Attributi

- **Entità** (*entity*): un oggetto del mondo reale (o del mini-mondo), distinguibile dagli altri, che vogliamo rappresentare nella BD (es. il dipendente John Smith).
- **Insieme di entità** (*entity set*): a livello estensionale, un insieme di entità simili (stessi attributi).
- **Tipo di entità** (*entity type*): la definizione a livello intensionale delle entità che possono appartenere a un entity set di quel tipo.
- **Attributi** (*attributes*): le proprietà usate per descrivere un'entità (es. nome, SSN, indirizzo). Ogni attributo è associato a un dominio (*value set*).

5.2.1 Tipi di Attributo

- **Semplice** (*simple*): un singolo valore atomico (es. `'M'` per Sesso);
- **Composto** (*composite*): più componenti (es. Indirizzo → Via, Città, CAP, ...); un componente di un attributo composto può a sua volta essere composto;
- **Multi-valore** (*multi-valued*): un'entità può avere più valori per lo stesso attributo (es. CERTIFICAZIONI di un dipendente).

5.2.2 Attributi Chiave

Ogni entity type ha un attributo (o insieme di attributi) che identifica univocamente ogni entità: l'**attributo chiave**. Può essere:

- **composto** (es. per un'AZIENDA: RAGIONE_SOCIALE + PROVINCIA);
- **non unico**: un entity type può avere più **chiavi candidate** (es. AUTOMOBILE: NumeroTelaio/VIN oppure Targa); una viene scelta come **chiave primaria**.

In ER, la chiave primaria è sempre sottolineata.

5.2.3 Rappresentazione Grafica

- un **entity type** è un **rettangolo** con il nome del tipo;
- gli **attributi** sono **ovali**, collegati al loro entity type; gli attributi chiave sono sottolineati.

5.3 Relazioni (Relationships)

Una **relazione** (*relationship*) è un'associazione tra due o più entità (es. la relazione tra un IMPIEGATO e il DIPARTIMENTO per cui lavora). Relazioni dello stesso tipo sono raggruppate in un **relationship type**.

5.3.1 Grado di una Relazione

Il numero di entity types che partecipano a un relationship type è il suo **grado**: una relazione tra 2 entity types è **binaria** (grado 2); tra 3, **ternaria** (grado 3) — es. IMPIEGATO-DIPARTIMENTO-SEDE, quando un impiegato lavora per un dipartimento che ha più sedi.

5.3.2 Attributi della Relazione

Una relazione può avere attributi descrittivi propri (es. `since` su LAVORA_PER), che descrivono la relazione stessa, non le entità partecipanti. **Attenzione**: una relazione è identificata unicamente dalle entità che vi partecipano, **non** dai suoi attributi descrittivi — quindi per ogni coppia $\langle ssn, did \rangle$ di WORKS_IN non può esistere più di un valore per `since` (se serve registrare più periodi, serve un modello diverso — vedi Sez. 5.10.2).

5.3.3 Relationship Type vs. Relationship Set

- **Relationship Type**: lo schema che descrive la relazione — nome, entity types partecipanti, vincoli;
- **Relationship Set**: l'insieme di istanze della relazione rappresentate nella BD in un certo istante (lo stato attuale del relationship type).

5.4 Vincoli sulle Relazioni

Tre tipi principali: vincoli di **chiave**, di **cardinalità**, di **partecipazione (o esistenza)**.

5.4.1 Vincolo di Chiave (key constraint)

Esprime che un'entità può partecipare al **massimo una volta** in una relazione (es. un Dipartimento può avere al massimo un manager, ma non viceversa). Si generalizza a relazioni di grado > 2: una certa entità non può comparire più di una volta nel relationship set. **Notazione**: freccia semplice dall'entità vincolata verso la relazione.

5.4.2 Vincoli di Cardinalità (cardinality ratio)

Specificano il numero massimo di volte in cui un'entità può comparire in un relationship set:

- **One-to-one (1:1)** — es. PERSONA *commissario_tecnico* NAZIONALE;
- **One-to-many / Many-to-one (1:N / N:1)** — es. STUDENTE *iscritto* SCUOLA_DI_DOTTORATO;
- **Many-to-many (M:N)** — es. STUDENTE *frequenta* CORSO.

5.4.3 Vincolo di Partecipazione (participation constraint)

Specifica quante volte un'entità deve comparire **al minimo** in una relazione (da cui anche il nome *Existence Dependency Constraint*):

- **partecipazione parziale**: l'entità può non comparire → **linea semplice**;
- **partecipazione totale**: l'entità deve comparire almeno una volta → **linea in grassetto**.

Esempio: DEPARTMENTS ha partecipazione totale in MANAGES (ogni dipartimento ha necessariamente un manager); DEPARTMENTS ed EMPLOYEES hanno entrambi partecipazione totale in WORKS_IN (ogni impiegato afferisce ad almeno un dipartimento, e viceversa).

5.4.4 Notazione (min, max)

Alternativa più precisa: ogni entità in E partecipa in almeno `min` e al massimo `max` istanze di R. Il default (assenza di vincoli espliciti) è `min=0, max=N` e non va indicato; si usa la notazione esplicita quando `min`>1 o `max` è diverso da 1 o N (es. 5).

5.5 Relazioni Ricorsive

Una relazione può legare entità dello **stesso tipo** ma in **ruoli distinti**: es. REPORTS_TO, dove EMPLOYEE partecipa due volte con ruoli diversi (Supervisore e Subordinato).

5.6 Entity Types Deboli

Un'**entità debole** non ha un proprio attributo chiave sufficiente all'identificazione univoca, e dipende da un'altra entità (**identificante**) tramite una **relazione identificante**. È identificata dalla combinazione di:

- una **chiave parziale** (propria dell'entità debole);
- la chiave dell'entità identificante.

Esempio 1. DEPENDENT (familiare a carico) è identificato dal proprio nome (chiave parziale) + l'EMPLOYEE a cui è associato tramite la relazione identificante POLICY.

Esempio 2. ORDER_DETAILS (righe di un ordine, con prodotto e quantità) è identificato da `OrderID` (dell'ORDER a cui appartiene, tramite la relazione identificante INCLUDE) + `ProductID`.

5.7 Gerarchie di Classi (ISA)

Talvolta è utile organizzare entity types in gerarchie, per:

- **Generalizzazione**: due entity types vengono ricompresi in uno più generale;
- **Specializzazione**: un entity type viene suddiviso in sottoinsiemi con caratteristiche comuni (può avvenire su più dimensioni, creando gerarchie multilivello).

Esempio: EMPLOYEE specializzato in HOURLY_EMPS e CONTRACT_EMPS (generalizzati in EMPLOYEES).

Perché introdurre gerarchie ISA:

- per aggiungere attributi specifici di una sottoclasse (es. costo orario per impiegati a ore);
- per identificare il sottoinsieme di entità che partecipa a una certa relazione (es. solo i dipendenti a contratto in un circolo ricreativo aziendale).

5.7.1 Vincoli: Overlap e Coverage

- **Overlap**: possono due sottoclassi avere entità in comune? Es. CONTRACT_EMPS e HOURLY_EMPS **non** possono sovrapporsi; CONTRACT_EMPS e SENIOR_EMPS sì (si scrive `CONTRACT_EMPS OVERLAPS SENIOR_EMPS`). **Default: nessun overlap**.
- **Coverage** (completezza): le sottoclassi coprono tutta la superclasse? Es. TIGRI e PANTERE **non** coprono tutti i FELINI; JUNIOR_EMPS e SENIOR_EMPS presumibilmente coprono EMPLOYEES (si scrive `JUNIOR_EMPS AND SENIOR_EMPS COVER EMPLOYEES`). **Default: coverage non vale**.

Si hanno quindi 4 combinazioni: (no overlap, copertura totale), (no overlap, copertura parziale), (overlap, copertura totale), (overlap, copertura parziale). **Nota**: la generalizzazione è di solito (ma non necessariamente) totale, poiché le superclassi sono derivate dalle sottoclassi.

5.8 Aggregazione

Un relationship set normalmente associa entity set. L'**aggregazione** permette di associare un entity set con un **intero relationship set** (che a sua volta associa 2 o più entity set), trattandolo come un'entità unica ai fini di un'ulteriore relazione.

Esempio. Un impiegato (EMPLOYEE) può MONITORARE un PROGETTO (PROJECT) che è SPONSORIZZATO da uno o più DIPARTIMENTI (DEPARTMENT): MONITORS aggrega la relazione SPONSORS (tra PROJECT e DEPARTMENT).

5.9 Dai Requisiti al Modello: Regole Empiriche

1. **Analisi del linguaggio naturale**: nomi → entità candidate; verbi → relazioni candidate; aggettivi/frasi descrittive → attributi candidati;
2. **Chiedere chiarimenti al cliente**: **non inventare** — se un attributo o vincolo non è esplicitato, va discusso col cliente prima di aggiungerlo;
3. **Identificazione delle entità**: nomi concreti (Cliente, Prodotto, Ordine); generalizzare sostantivi simili/correlati in una categoria comune;
4. **Individuazione delle relazioni**: frasi come "un cliente effettua un ordine"; la **cardinalità** emerge da frasi come "ogni cliente può effettuare più ordini"; la **partecipazione** da frasi come "ogni dipartimento deve avere un manager" (totale);
5. **Identificazione degli attributi**: descrittori delle entità; le chiavi primarie da frasi come "ogni cliente è identificato con un ID", o dedotte dalla semantica;
6. **Validazione**: completezza (tutte le entità/relazioni richieste sono presenti) e assenza di **ridondanza** ingiustificata (es. duplicare il nome del cliente sia nell'anagrafica sia negli ordini);
7. **Iterazione e raffinamento**: presentare il diagramma ER al cliente, raccogliere feedback, iterare.

5.10 Scelte di Progettazione Concettuale con ER

Definire un modello ER non è un processo lineare: lo stesso insieme di requisiti può essere modellato in modi diversi, e non tutti i vincoli di dominio sono esprimibili in ER (limiti di espressività).

5.10.1 Entità vs. Attributo

Quando la scelta non è vincolante, si preferisce l'attributo per semplicità. Serve un **entity type** quando:

1. potremmo dover registrare **più valori** per la stessa entità (es. più indirizzi per lo stesso dipendente);
2. vogliamo **scomporre** ulteriormente l'informazione (es. indirizzo in via/comune/provincia per query mirate).

Stesso ragionamento per un attributo su una *relazione*: se `from/to` sono attributi di WORKS_IN, si può registrare un solo periodo per coppia impiegato-dipartimento; per registrarne più di uno serve un nuovo entity type.

5.10.2 Entità vs. Relazione

Esempio: un impiegato che gestisce un dipartimento ha anche un budget. Se il budget è associato alla relazione MANAGES, e un impiegato può gestire più dipartimenti con un budget **complessivo** (non per dipartimento), si presentano due problemi:

- **ridondanza**: il valore di DBUDGET si ripeterebbe identico per ogni istanza che lega l'impiegato ai vari dipartimenti (si riprenderà nella normalizzazione);
- **semantica scorretta**: si suggerisce che il budget appartenga alla relazione, mentre appartiene al manager.

Soluzione: specializzare EMPLOYEES con una sottoclasse MANAGERS, e assegnare a quest'ultima gli attributi SINCE e DBUDGET (se SINCE deve invece variare per dipartimento gestito, va lasciato sulla relazione).

5.10.3 Relazioni Binarie o Ternarie

Quando 3 entity types sono coinvolti in un requisito, bisogna chiedersi se si tratta davvero di un'unica relazione a 3 vie o di due relazioni binarie distinte.

Esempio. EMPLOYEE può possedere più POLICY (polizze), ognuna posseduta da un solo EMPLOYEE; ogni DEPENDENT può avere più POLICY. Vincoli aggiuntivi: una polizza non può essere posseduta congiuntamente da più impiegati; ogni polizza deve essere posseduta da un impiegato.

- **Ipotesi 1 (relazione ternaria con vincoli)**: un vincolo di chiave da POLICIES verso la relazione ternaria implicherebbe (erroneamente) che una polizza copra un solo DEPENDENT; un vincolo di partecipazione totale su POLICY funzionerebbe solo se ogni polizza copre almeno un DEPENDENT.

- **Ipotesi 2 (due relazioni binarie)**: soluzione preferibile — evidenzia le due relazioni distinte a cui partecipa POLICIES, e usa una relazione binaria come identificante per l'entità debole (richiesto in alcune varianti di ER).

Non c'è una risposta univoca: dipende dalla semantica dei requisiti (nell'esempio dei manager con budget visto sopra, invece, la scelta corretta era la relazione ternaria).

5.10.4 Aggregazione o Relazione Ternaria?

Riprendendo l'esempio MONITORS/SPONSORS: una relazione ternaria unica (SPONSORS2, tra EMPLOYEE-PROJECT-DEPARTMENT) sarebbe stata sufficiente solo se non fosse stato necessario un attributo `until` specifico su MONITORS; inoltre, non avrebbe permesso di esprimere facilmente il vincolo "ogni sponsorizzazione è monitorata al massimo da un impiegato", che con l'aggregazione (vincolo di chiave da MONITORS verso SPONSORS) si esprime naturalmente.

Esercizio 1

In ER un'aggregazione è definita come: (un operatore per aggregare entità in relazione gerarchica IS-A / un attributo dinamico ottenuto da operatori di aggregazione come somma o media / un modo per rappresentare una relazione tra un'entità e un gruppo di entità correlate, trattando questo gruppo come un'unica entità / nessuna delle opzioni)

Soluzione: un modo per rappresentare una relazione tra un'entità e un gruppo di entità correlate, trattando questo gruppo come un'unica entità — è la definizione esatta vista sopra (associare un entity set a un intero relationship set).

6 Mapping dal Modello ER al Modello Relazionale

6.1 Obiettivi del Mapping

- preservare il più possibile la conoscenza rappresentata nel modello concettuale;
- mantenere i vincoli, per quanto possibile: **alcuni vincoli non sono esprimibili** nel modello relazionale — cardinalità diverse da 1 e N, vincoli di partecipazione per relazioni di grado > 2;
- minimizzare i valori `NULL`.

Le procedure qui presentate sono quelle implementate nella maggior parte dei tool commerciali.

6.2 Da Entity Type a Tabelle

Per ogni entity type E dello schema ER:

1. creare una tabella R con **tutti** gli attributi di E;
2. porre uno degli attributi chiave di E come chiave primaria di R (se la chiave di E è composta, la chiave di R è l'insieme dei corrispondenti attributi).

Esempio.

```
CREATE TABLE Employees (
    ssn   CHAR(11),
    name  CHAR(30),
    lot   INTEGER,
    PRIMARY KEY (ssn)
);
```

6.3 Mapping delle Relazioni (Relationship Types)

6.3.1 Caso Base (senza vincoli)

Data una relazione ER tra due entity types, si crea una nuova tabella con:

- la **chiave primaria di ciascuna entità partecipante** (come chiave esterna);
- gli **attributi descrittivi** della relazione.

Gli attributi non descrittivi di questa nuova tabella formano una **superchiave** (o **chiave candidata**, se non ci sono vincoli di chiave in ER).

Esempio. Employees – Works_In2 (con attributo `since`) – Departments, dove Works_In2 coinvolge anche una terza entità Locations:

```
CREATE TABLE Works_In2 (
    ssn      CHAR(11),
    did      INTEGER,
    address  CHAR(20),
    since    DATE,
    PRIMARY KEY (ssn, did, address),
    FOREIGN KEY (address) REFERENCES Locations,
    FOREIGN KEY (did) REFERENCES Departments
);
```

6.3.2 Relazioni Ricorsive

```
CREATE TABLE Reports_To (
    supervisor_ssn  CHAR(11),
    subordinate_ssn CHAR(11),
    PRIMARY KEY (supervisor_ssn, subordinate_ssn),
    FOREIGN KEY (supervisor_ssn)  REFERENCES Employees(ssn),
    FOREIGN KEY (subordinate_ssn) REFERENCES Employees(ssn)
);
```

6.3.3 Vincolo di Chiave

Se su una delle due entità (es. E_2) esiste un vincolo di chiave, il mapping di base non va bene: la coppia $\langle PK_1, PK_2 \rangle$ sarebbe una superchiave ma **non** una chiave candidata (solo PK_2 non può ripetersi). Due soluzioni:

A. nuova relazione R_A con PK_1, PK_2 come chiavi esterne, ma **solo** PK_2 come chiave primaria, più eventuali attributi descrittivi;

B. aggiungere a E_2 un nuovo attributo (chiave esterna) che referenzia PK_1 — sfruttando il fatto che PK_2 non può comunque ripetersi, per il vincolo di chiave — più eventuali attributi descrittivi.

Esempio (Employees –Manages–> Departments, vincolo di chiave su Departments).

```
-- Soluzione A
CREATE TABLE Manages (
    ssn    CHAR(11),
    did    INTEGER,
    since  DATE,
    PRIMARY KEY (did),
    FOREIGN KEY (ssn) REFERENCES Employees,
    FOREIGN KEY (did) REFERENCES Departments
);

-- Soluzione B
CREATE TABLE Dept_Mgr (
    did     INTEGER,
    dname   CHAR(20),
    budget  REAL,
    ssn     CHAR(11),
    since   DATE,
    PRIMARY KEY (did),
    FOREIGN KEY (ssn) REFERENCES Employees
);
```

Confronto:

- **B**: *pro* — evita una tabella MANAGES a sé stante, associa direttamente un dipartimento al suo manager senza join; *contro* — molti `NULL` in `ssn` se ci sono molti dipartimenti senza manager.
- **A**: *pro* — elimina il problema dei NULL; *contro* — molte operazioni richiedono un join tra le due tabelle.

Entrambi gli approcci si estendono a relazioni di grado > 2.

6.3.4 Vincolo di Partecipazione

- **partecipazione parziale**: non richiede alcuna specifica particolare — corrisponde all'assenza di vincoli;
- **partecipazione totale**: **non** rappresentabile in modo completo senza perdita di informazione rispetto al modello ER; si può catturare solo combinando specifiche del modello relazionale con strumenti SQL aggiuntivi (*table constraints*, *assertions* — approfonditi con SQL).

Esempio parziale: aggiungere `NOT NULL` su `ssn` in `Dept_Mgr` implica che non possa esistere un dipartimento senza manager specificato — cattura **un aspetto** della partecipazione totale di Departments in Manages, non l'intero vincolo.

Esercizio 1

Nel mapping tra modello ER e modello relazionale, il vincolo di partecipazione parziale non richiede alcuna specifica particolare e corrisponde all'assenza di vincoli. (Falso / È falso, va rappresentato con una linea in grassetto che connette l'entità con partecipazione totale con la relazione / Vero / È vero solo nel caso in cui per l'entità sull'altro lato della relazione viene specificato un vincolo di chiave)

Soluzione: Vero — è esattamente la definizione data sopra: la partecipazione parziale è il caso di default, senza bisogno di alcuna specifica aggiuntiva nel modello relazionale (a differenza della partecipazione totale, che invece richiede accorgimenti extra e comunque non è pienamente rappresentabile).

6.4 Weak Entities

Le entità deboli: partecipano sempre a una relazione **1:N** con l'entità identificante; hanno un **vincolo di chiave**; hanno **partecipazione totale**. Due caratteristiche aggiuntive da gestire nel mapping:

- hanno solo una **chiave parziale**;
- devono essere **cancellate** se viene cancellata l'entità da cui dipendono (`ON DELETE CASCADE`).

Esempio. Employees –Policy–> Dependents (entità debole, chiave parziale `pname`):

```
CREATE TABLE Dep_Policy (
    pname  CHAR(20),
    age    INTEGER,
    cost   REAL,
    ssn    CHAR(11) NOT NULL,
    PRIMARY KEY (pname, ssn),
    FOREIGN KEY (ssn) REFERENCES Employees,
        ON DELETE CASCADE
);
```

La chiave primaria combina la chiave parziale dell'entità debole con la chiave dell'entità identificante; `NOT NULL` rinforza la partecipazione totale (vedi sopra).

Esercizio 2

Data la specifica in ER dell'entità debole Dependents (chiave parziale `pname`, entità identificante Employees con chiave `ssn`, relazione identificante Policy), qual è il suo corretto mapping nel modello relazionale?

a) `CREATE TABLE Dependents (pname CHAR(20), age INT, cost REAL, PRIMARY KEY (pname), FOREIGN KEY pname REFERENCES Employees (ssn) ON DELETE CASCADE)`

b) `CREATE TABLE Dependents (pname CHAR(20), age INT, cost REAL, ssn CHAR(11), PRIMARY KEY (ssn, pname), FOREIGN KEY (ssn) REFERENCES Employees(ssn) ON DELETE CASCADE)`

c) Tutte le opzioni sono valide

Soluzione: b) — coerente con quanto appena visto: la chiave primaria dev'essere `(ssn, pname)`, non la sola chiave parziale `pname` (opzione a), che da sola non identifica univocamente un Dependent tra impiegati diversi.

6.5 Gerarchie IS-A

Due approcci principali per mappare una gerarchia (es. Employee → ISA → Hourly_Emps, Contract_Emps):

6.5.1 Metodo Generale

Si crea una tabella per **ogni** classe della gerarchia, incluse le sottoclassi con solo la chiave (ereditata) più gli attributi specifici:

```
CREATE TABLE Employees (ssn CHAR(11), name CHAR(30), lot INTEGER,
    PRIMARY KEY (ssn));

CREATE TABLE Hourly_Emps (
    ssn CHAR(11), hourly_wages REAL, hours_worked INTEGER,
    PRIMARY KEY (ssn),
    FOREIGN KEY (ssn) REFERENCES Employees ON DELETE CASCADE
);

CREATE TABLE Contract_Emps (
    ssn CHAR(11), contract_id INTEGER,
    PRIMARY KEY (ssn),
    FOREIGN KEY (ssn) REFERENCES Employees ON DELETE CASCADE
);
```

`ON DELETE CASCADE` assicura che le tuple che referenziano un dipendente vengano cancellate insieme al dipendente stesso.

6.5.2 Metodo di Riduzione

Si eliminano la classe generale e si mappano **solo** le sottoclassi, ciascuna con **tutti** gli attributi ereditati:

```
Hourly_Emps(ssn, name, lot, hourly_wages, hours_worked)
Contract_Emps(ssn, name, lot, contract_id)
```

Richiede che **ogni** impiegato appartenga ad almeno una delle sottoclassi (copertura totale).

6.5.3 Confronto tra i Due Metodi

- **Metodo generale**: si applica sempre; le query sugli attributi di Employees non richiedono di esaminare le altre tabelle; le query su attributi specifici di una sottoclasse possono richiedere un join.
- **Metodo di riduzione**: **non** si applica se esistono impiegati che non appartengono a nessuna sottoclasse; se c'è **overlap**, le informazioni ereditate (name, lot) sono **duplicate** tra le tabelle; le query su **tutti** gli impiegati devono esaminare entrambe le tabelle.

La scelta dipende dalla semantica dei dati e dalla frequenza con cui vengono eseguite specifiche query.

6.6 Aggregazioni

Esempio. Employees –Monitors(`until`)–> [Projects –Sponsors(`since`)–> Departments], dove Monitors aggrega Sponsors.

Approccio generale: Employees, Projects, Departments mappate come di consueto; Sponsors mappata come una relazione normale; per Monitors si crea una nuova tabella con le chiavi delle entità coinvolte + gli attributi descrittivi:

```
CREATE TABLE Monitors (
    ssn CHAR(11), did INTEGER, pid INTEGER, until DATE,
    PRIMARY KEY (ssn, did, pid),
    FOREIGN KEY (ssn) REFERENCES Employees,
    FOREIGN KEY (did, pid) REFERENCES Sponsors(did, pid)
);
```

Caso speciale (semplificazione): se **entrambe** le condizioni seguenti valgono:

1. la relazione esterna all'aggregazione (Sponsors) **non ha attributi descrittivi**;
2. l'aggregazione ha **partecipazione totale** in Monitors (ogni coppia progetto-dipartimento deve avere un impiegato che la monitora);

allora **Sponsors può essere rimossa** dal mapping: tutte le sue istanze sono ricavabili dalle colonne $\langle pid, did \rangle$ di Monitors.

7 Dipendenze Funzionali e Chiavi

7.1 Motivazione: Anomalie nella Base di Dati

Una relazione mal progettata può presentare **anomalie**. Esempio: una tabella che unisce impiegati, progetti a cui partecipano (con relativa funzione) e informazioni di stipendio/bilancio, dove per ogni impiegato lo stipendio è unico (indipendente dai progetti) e per ogni progetto il bilancio è unico (indipendente dagli impiegati). Questo causa:

- **Ridondanza**: lo stipendio di un impiegato si ripete in ogni tupla in cui l'impiegato compare;
- **Anomalia di aggiornamento**: se cambia lo stipendio, vanno aggiornate tutte le tuple in cui compare quell'impiegato;
- **Anomalia di inserimento**: non si può inserire un nuovo impiegato senza assegnargli anche un progetto;
- **Anomalia di cancellazione**: se un impiegato smette di partecipare a qualsiasi progetto, va cancellato dalla tabella insieme alle sue informazioni.

Causa: la tabella mescola informazioni eterogenee (impiegati e stipendi; progetti e bilanci; il legame impiegato-progetto-funzione) in un'unica relazione. Per studiare sistematicamente il problema si introduce il concetto di **dipendenza funzionale** (DF).

7.2 Definizione Formale di Dipendenza Funzionale

Sia r una relazione sullo schema $R(X)$, e siano Y, Z due sottoinsiemi non vuoti di X. Z **dipende funzionalmente** da Y ($Y \to Z$) se, per ogni coppia di tuple $t_1, t_2 \in r$:

$$t_1[Y] = t_2[Y] \implies t_1[Z] = t_2[Z]$$

(cioè: se t_1 e t_2 concordano su Y, allora concordano anche su Z).

7.3 Come si Identificano le DF

Le DF derivano in genere da vincoli del mondo reale sul significato degli attributi, **non** solo dai dati:

- da un'**istanza** di R si può solo **inferire** che una DF *potrebbe* sussistere (potrebbe non valere su un'altra istanza);
- viceversa, se un'istanza contiene tuple che **violano** una DF, si può concludere con **certezza** che quella DF **non** sussiste.

7.4 DF Banali

Una DF $Y \to A$ è **banale** se $A \in Y$ (es. Impiegato, Stipendio $\to$ Stipendio è sempre soddisfatta). Una DF $Y \to Z$ è **non banale** se nessun attributo di Z appartiene a Y. D'ora in poi si considerano solo DF non banali.

7.5 DF e Chiavi

Data una chiave K di R, esiste sempre una DF tra K e ogni altro attributo di R (per definizione di chiave, non possono esistere due tuple con lo stesso valore su K). Il concetto di DF **generalizza** quindi il vincolo di chiave: una DF $Y \to Z$ su $R(X)$ **degenera in un vincolo di chiave** se $Y \cup Z = X$ (in tal caso Y è una (super)chiave per $R(X)$).

7.6 DF e Anomalie

Nell'esempio iniziale, Impiegato $\rightarrow$ Stipendio e Progetto $\rightarrow$ Bilancio **non** contengono una chiave nella parte sinistra (la chiave è $\langle$Impiegato, Progetto$\rangle$), mentre Impiegato, Progetto $\rightarrow$ Funzione sì. **Una DF che contiene una chiave a sinistra non può generare ridondanza** per definizione: le anomalie sono causate esclusivamente dalle DF $Y \rightarrow Z$ in cui Y **non** contiene una chiave.

7.7 Chiusura di un Insieme di Dipendenze Funzionali

Dato un insieme F di DF, è possibile derivare altre DF logicamente implicate da F (es. se $X \rightarrow Y$ e $Y \rightarrow Z$, allora $X \rightarrow Z$). F implica logicamente $X \rightarrow Y$ se ogni istanza che soddisfa F soddisfa anche $X \rightarrow Y$. L'insieme di **tutte** le DF logicamente implicate da F è la **chiusura** di F, indicata F^+.

7.7.1 Assiomi di Armstrong

F^+ si calcola con gli **Assiomi di Armstrong (AA)**:

- **Riflessività**: se $Y \subseteq X$, allora $X \rightarrow Y$;
- **Aumento**: se $X \rightarrow Y$, allora $XZ \rightarrow YZ$;
- **Transitività**: se $X \rightarrow Y$ e $Y \rightarrow Z$, allora $X \rightarrow Z$.

Regole derivate: **Unione** (se $X \rightarrow Y$ e $X \rightarrow Z$, allora $X \rightarrow YZ$); **Scomposizione** (se $X \rightarrow YZ$, allora $X \rightarrow Y$ e $X \rightarrow Z$).

Gli AA sono **corretti** (*sound*: ogni DF derivata è logicamente implicata da F) e **completi** (*complete*: ogni DF logicamente implicata da F può essere derivata) — generano quindi **tutte e sole** le DF di F^+.

Esempio. $R(A, B, C, G, H, I)$, $F = \{A \rightarrow B, A \rightarrow C, CG \rightarrow H, CG \rightarrow I, B \rightarrow H\}$. Alcuni membri di F^+:

- $A \rightarrow H$: per transitività da $A \rightarrow B$ e $B \rightarrow H$;
- $AG \rightarrow I$: aumentando $A \rightarrow C$ con G ($AG \rightarrow CG$), poi transitività con $CG \rightarrow I$;
- $CG \rightarrow HI$: aumentando $CG \rightarrow I$ con CG si ottiene $CG \rightarrow CGI$; aumentando $CG \rightarrow H$ con I si ottiene $CGI \rightarrow HI$; infine transitività tra le due.

7.8 Chiusura di un Insieme di Attributi

Calcolare F^+ per intero è dispendioso (può richiedere un numero esponenziale di DF). Per verificare se $X \rightarrow Y$, basta invece calcolare la **chiusura di un insieme di attributi** X, indicata $(X)^+$:

1. si pone $(X)^+ = X$;
2. se esiste una DF $Z \rightarrow W$ in F con $Z \subseteq (X)^+$, si aggiunge W a $(X)^+$;
3. si ripete finché non è più possibile aggiungere nulla.

Esempio. $F = \{A \to B,\ B \to C,\ AC \to D\}$. Chiusura di A: $(A)^+ = A \to AB$ (usando $A \to B$) $\to ABC$ (usando $B \to C$) $\to ABCD$ (usando $AC \to D$). Quindi $(A)^+ = ABCD$.

7.9 Individuazione delle Superchiavi

Dato $\langle R(U), F\rangle$ e $X \subseteq U$: si calcola $(X)^+$; se $(X)^+ = U$ (cioè $X \to U$), allora X è una **superchiave** di R, altrimenti no. **Attenzione**: X potrebbe essere una superchiave non minimale (non una chiave).

Esempio. $R(A, B, C, D)$, $F = \{A \to B,\ B \to C,\ AC \to D\}$: $(A)^+ = ABCD \Rightarrow A$ è superchiave (ed è una chiave, essendo un singolo attributo); $(AC)^+ = ACBD = ABCD$ $\Rightarrow AC$ è superchiave ma **non** è minimale (quindi non è una chiave, basta A); $(B)^+ = BC$ $\Rightarrow B$ **non** è superchiave.

7.9.1 Algoritmo per le Chiavi Candidate

1. **Isolati** (I): attributi né a sinistra né a destra di alcuna DF;
2. **Left** (L): attributi solo a sinistra di qualche DF;
3. **Right** (R): attributi solo a destra di qualche DF;
4. si combinano $I \cup L$;
5. si calcola la chiusura di $I \cup L$: se coincide con U, è l'**unica** chiave candidata e l'algoritmo termina;
6. altrimenti, si individuano gli attributi non ancora considerati (quelli che compaiono sia a sinistra sia a destra, più quelli solo a destra);
7. si combina la chiusura di $I \cup L$ con ciascun sottoinsieme degli attributi dello Step 6, uno alla volta, cercando quali chiusure danno U: sono le chiavi candidate (aggiungere ulteriori attributi produce solo superchiavi non minimali).

Per aiutarsi si può disegnare il **grafo delle dipendenze** (nodi = attributi, archi = DF): un nodo senza archi entranti (isolato incluso) appartiene necessariamente a **ogni** chiave candidata.

Esempio. $R(A, B, C, D, E)$, $F = \{A \to B,\ BC \to E,\ ED \to A\}$.

Step 1 (Isolati)	$\{\}$
Step 2 (Left)	$\{C, D\}$
Step 3 (Right)	$\{\}$
Step 4 (unione I, L)	$\{C, D\}$
Step 5 (chiusura)	$(CD)^+ = CD$ (non è chiave)
Step 6 (restanti)	$\{A, B, E\}$
Step 7	$(CDA)^+$=$(CDB)^+$=$(CDE)^+$=$ABCDE$

Aggiungere ulteriori attributi produce solo superchiavi: **ACD, BCD e CDE** sono le sole chiavi candidate di R (C e D non hanno archi entranti nel grafo delle dipendenze, quindi appartengono a ogni chiave candidata).

Esercizio

Dato $\langle R(A,B,C,D), \{AB \rightarrow C,\ C \rightarrow D,\ C \rightarrow A\}\rangle$, elencare tutte le chiavi candidate.

Soluzione: applicando l'algoritmo — Isolati= {}, Left= $\{B\}$, Right= $\{D\}$; $(B)^+ = B$ (non basta). Restanti = $\{A, C\}$: $(AB)^+$: $AB \rightarrow C$ (ho A,B) dà ABC; $C \rightarrow D$ dà $ABCD$ $\Rightarrow$ **AB è chiave candidata**. $(BC)^+$: $C \rightarrow D$ dà BCD; $C \rightarrow A$ dà $ABCD \Rightarrow$ **BC è chiave candidata**. Chiavi candidate: **AB, BC**.

7.10 Copertura Minimale

Una **copertura minimale** F_{min} di F è un insieme di DF logicamente equivalente a F ($F^+_{min} = F^+$) tale che: (1) ogni DF in F_{min} ha un **solo** attributo a destra; (2) rimuovendo una DF o un attributo dal lato sinistro di una DF in F_{min}, l'insieme non è più equivalente a F. **Nota**: può esistere più di una copertura minimale per F. È dimostrabile che uno schema è in BCNF se e solo se lo è rispetto a F_{min}: per questo ci si può limitare a considerare F_{min}.

Calcolo (3 step):

1. **Normalizzazione**: ogni DF con più attributi a destra ($X \rightarrow ABC\ldots$) si scompone in DF con un solo attributo a destra ($X \rightarrow A$, $X \rightarrow B$, $X \rightarrow C$, $\ldots$);
2. **Attributi estranei**: per ogni DF $XA \rightarrow B$, si verifica se $B \in (X)^+$ (calcolata SENZA usare A): se sì, A è ridondante e si sostituisce $XA \rightarrow B$ con $X \rightarrow B$;
3. **DF ridondanti**: per ogni DF $X \rightarrow A$, si rimuove temporaneamente da M ottenendo M'; se $A \in (X)^+$ calcolata su M', la DF è ridondante e resta rimossa.

Esempio. $R(A,B,C,D,E)$, $F = \{A \rightarrow BCE,\ CDB \rightarrow A,\ CD \rightarrow E,\ E \rightarrow B\}$.

1. Normalizzazione: $A \rightarrow BCE$ diventa $A \rightarrow B, A \rightarrow C, A \rightarrow E$. Insieme: $\{A \rightarrow B, A \rightarrow C, A \rightarrow E, CDB \rightarrow A, CD \rightarrow E, E \rightarrow B\}$;
2. Attributi estranei: $(CD)^+ = CDAEB$ (usando le altre DF) $\Rightarrow$ B è estraneo in $CDB \rightarrow A$, che si riduce a $CD \rightarrow A$. Verificando C e D singolarmente: $(C)^+ = C$, $(D)^+ = D$ — nessuno dei due è ridondante, $CD \rightarrow A$ resta;
3. DF ridondanti: da A si può derivare B senza usare $A \rightarrow B$ (via $A \rightarrow E$, $E \rightarrow B$) $\Rightarrow$ $A \rightarrow B$ è ridondante; da CD si può derivare E senza usare $CD \rightarrow E$ (via $CD \rightarrow A$, $A \rightarrow E$) $\Rightarrow$ $CD \rightarrow E$ è ridondante.

$$F_{min} = \{A \rightarrow C,\ A \rightarrow E,\ CD \rightarrow A,\ E \rightarrow B\}$$

8 Normalizzazione

8.1 Normalizzazione via Decomposizione

Le **forme normali** sono condizioni che garantiscono che certe anomalie non possano presentarsi in una relazione. Il processo con cui una relazione che non soddisfa una forma normale viene scomposta in due o più relazioni si chiama **normalizzazione**. L'intuizione: le anomalie si risolvono **decomponendo** la relazione originale in relazioni più piccole. R_1 e R_2 sono una decomposizione di R se:

- R_1 e R_2 contengono ciascuna un sottoinsieme degli attributi di R;
- $R_1 \cup R_2$ contiene tutti gli attributi di R.

In generale, una decomposizione rappresenta l'informazione di R tramite due **proiezioni** di R.

8.2 Proprietà Desiderabili di una Decomposizione

- **Decomposizione senza perdita di informazione** (*lossless join*): ricomponendo con un join la decomposizione, non si devono ottenere tuple **spurie** (non appartenenti alla relazione iniziale) — questa proprietà **deve sempre** essere garantita;
- **Conservazione delle DF**: la decomposizione deve preservare le DF della relazione originale — questa proprietà, in certi casi, può essere **sacrificata**.

8.2.1 Perdita di Informazione: Esempio

$R(\text{Impiegato}, \text{Progetto}, \text{Sede})$ decomposta in $R_1(\text{Impiegato}, \text{Sede})$ e $R_2(\text{Progetto}, \text{Sede})$: il natural join su `Sede` produce tuple **spurie** (es. un impiegato associato a un progetto a cui non ha mai partecipato, solo perché condividono la sede). **Perché**: `Sede` è l'unico attributo comune, ma non è (e non può essere) chiave di nessuna delle due relazioni — più impiegati e più progetti possono condividere la stessa sede — quindi il join genera tutte le combinazioni possibili, perdendo l'associazione originale impiegato-progetto.

8.2.2 Condizione Formale per la Decomposizione Senza Perdita

Sia R su X, con $X = X_1 \cup X_2$ e $X_0 = X_1 \cap X_2$. R si decompone **senza perdita** su X_1, X_2 se soddisfa $X_0 \to X_1$ **oppure** $X_0 \to X_2$. In altre parole: **gli attributi comuni tra le relazioni decomposte devono contenere una (super)chiave di almeno una delle due**. Nell'esempio sopra: `Sede` non contiene una chiave né per R_1 né per $R_2 \Rightarrow$ la decomposizione **non** è senza perdita.

8.2.3 Conservazione delle DF

Condizione: ciascuna DF dello schema originario deve coinvolgere attributi che compaiono tutti insieme in **uno** degli schemi decomposti (le DF originali devono essere ottenibili per proiezione dagli schemi decomposti).

Esempio. Le DF di R(Impiegato, Progetto, Sede) sono Impiegato $\rightarrow$ Sede e Progetto $\rightarrow$ Sede. La decomposizione R_1(Impiegato, Sede), R_2(Impiegato, Progetto) **non** conserva Progetto $\rightarrow$ Sede: si potrebbero inserire $\langle$Neri,Milano$\rangle$ in R_1 e $\langle$Neri,Marte$\rangle$ in R_2 senza violare alcuna DF nelle tabelle decomposte, pur violando concettualmente Progetto $\rightarrow$ Sede nella relazione originale.

Esercizio 1

Una decomposizione di $R(A, B, C)$ nelle relazioni $S(A, B)$ e $T(B, C)$ è senza perdita di informazione se: (tutte le tuple di R sono nel join di S e T / il join di S e T contiene tutte e sole le tuple che sono nella relazione originale R / se soddisfa la DF A$\rightarrow$B / la DF C$\rightarrow$A è soddisfatta)

Soluzione: il join di S e T contiene tutte e sole le tuple che sono nella relazione originale R — è la definizione formale di decomposizione lossless: $R \bowtie S = R$ (né tuple mancanti né tuple spurie).

8.3 Prima Forma Normale (1NF)

Una relazione è in 1NF se **non** ha: tuple ripetute (deve esistere una chiave); attributi composti e/o multi-valore. Molti DBMS non accettano nemmeno relazioni che violano la 1NF.

Esempio. `DEPARTMENT(Dname, Dnumber, Dmgr_ssn, Dlocations)` non è in 1NF perché `Dlocations` ha più valori per la stessa entità (es. Research: {Bellaire, Sugarland, Houston}). Si normalizza ripetendo una riga per ogni località:

```
Research        5   333445555   Bellaire
Research        5   333445555   Sugarland
Research        5   333445555   Houston
Administration  4   987654321   Stafford
Headquarters    1   888665555   Houston
```

8.4 Richiami: Attributi Primi e Non-Primi

- **attributo primo** (*prime*): appartiene ad **almeno una** chiave candidata;
- **attributo non-primo** (*non-prime*): non appartiene ad **alcuna** chiave candidata.

8.5 Seconda Forma Normale (2NF)

R è in 2NF se è in 1NF e **ogni attributo non-primo dipende dall'intera chiave primaria** (nessuna dipendenza parziale da un sottoinsieme proprio della chiave). **Osservazione**: una relazione con chiave primaria **non composta** non può mai violare la 2NF (non può esistere una dipendenza da "parte" di un solo attributo). **Violare la 2NF** causa principalmente **ridondanza** (e le conseguenti anomalie di inserimento/cancellazione/aggiornamento).

8.6 Terza Forma Normale (3NF)

R è in 3NF se è in 2NF e **nessun attributo non-primo dipende transitivamente da una chiave candidata** (non deve valere $X \to Y$, $Y \to Z \Rightarrow X \to Z$ con X chiave candidata, a meno che Y non sia esso stesso una chiave candidata). **Formulazione equivalente**: per ogni DF non banale $X \to A$, vale almeno una delle:

- X è una superchiave di R, oppure
- A è un attributo primo di R.

Esempio. `(CodImpiegato, Nome, Reparto, TelReparto)`, chiave `CodImpiegato`, DF: CodImpiegato → Reparto, Reparto → TelReparto. Anomalie: il telefono del reparto si ripete per ogni impiegato (ridondanza); un reparto senza impiegati non può avere un telefono registrato. Si normalizza in `(CodImpiegato,Nome,Reparto)` + `(Reparto,TelReparto)`: entrambe in 3NF, DF conservate, **nessuna perdita** (Reparto, l'attributo comune, determina funzionalmente TelReparto nella seconda tabella).

8.7 3NF e Ridondanza Residua

La 3NF non elimina **tutte** le ridondanze. Esempio: `(Prefisso, Numero, Località, Abbonato, Indirizzo)` con ⟨Prefisso, Numero⟩ chiave e Località → Prefisso (dove Prefisso è un attributo **primo**, essendo parte della chiave): la relazione È in 3NF (la condizione "A è primo" è soddisfatta), ma il prefisso si ripete comunque per ogni abbonato della stessa località — **ridondanza**. Per escludere anche $Y \to Z$ con Y non-primo e Z primo, si definisce una forma normale più restrittiva.

8.8 Forma Normale di Boyce-Codd (BCNF)

R è in BCNF se è in 2NF e, per **ogni** DF non banale $X \to Y$ su R, X è una **superchiave** di R. Intuitivamente: ogni attributo descrive un'entità "identificata dalla chiave, dall'intera chiave, e da nient'altro che dalla chiave" (W. Kent, 1978). **È più restrittiva della 3NF** ed è teoricamente la forma migliore, ma **non sempre raggiungibile** preservando tutte le DF — in tal caso ci si "accontenta" della 3NF (sempre ottenibile).

8.8.1 Procedura di Decomposizione in BCNF

Data $R(U)$ e una DF $X \to A$ che viola la BCNF (X non superchiave): decomporre in $R_1(U - A)$ e $R_2(XA)$; ripetere ricorsivamente su R_1/R_2 finché non sono entrambe in BCNF.

Esempio. `(Prefisso,Numero,Località,Abbonato,Indirizzo)`: Località → Prefisso viola la BCNF (Località non è chiave). Decomposizione in BCNF: `(Numero,Località,Abbonato,Indiriz` + `(Prefisso,Località)`.

8.8.2 Quando la BCNF Non È Raggiungibile Preservando le DF

Esempio. DF: Dirigente → Sede (ogni dirigente opera in una sola sede); Progetto, Sede → Dirigente (per ogni sede, un progetto ha un solo dirigente). La relazione **non** è in BCNF

(nella prima DF, Dirigente non è superchiave). **Nessuna decomposizione può preservare entrambe le DF** (la seconda coinvolge tutti gli attributi): non esiste modo di raggiungere la BCNF per questo schema — ci si accontenta della 3NF.

Esercizio 2

INSEGNA(STUDENTE, INSEGNAMENTO, DOCENTE) con DF: $(\text{STUDENTE}, \text{INSEGNAMENTO}) \to \text{DOCENTE}$, $\text{DOCENTE} \to \text{INSEGNAMENTO}$. Decomposizione: $\text{INSEGNA}_1(\text{DOCENTE}, \text{INSEGNAMENTO})$, $\text{INSEGNA}_2(\text{DOCENTE}, \text{STUDENTE})$. Quale affermazione è vera? (non esiste modo di decomporre INSEGNA in BCNF / la decomposizione non è in BCNF / la decomposizione è in BCNF / la decomposizione è in BCNF e preserva tutte le DF originali)

Soluzione: la decomposizione è in BCNF (**ma non** preserva tutte le DF) — è esattamente il fenomeno appena visto: in INSEGNA_1 la chiave è DOCENTE (che determina tutto); in INSEGNA_2 la chiave è DOCENTE (idem). Entrambe in BCNF, ma la decomposizione perde $(\text{STUDENTE}, \text{INSEGNAMENTO}) \to \text{DOCENTE}$.

Esercizio 3

$\langle R(A, B, C, D, E, F), F = \{AB \to CD, C \to EF\}\rangle$, decomposta in $R_1(A, B, C, D)$ e $R_2(C, E, F)$. Quale affermazione è vera? (in BCNF e conserva tutte le DF / in 3NF ma non in BCNF / in BCNF ma non conserva le DF)

Soluzione: in BCNF e conserva tutte le DF — in R_1 la chiave è AB (che con $AB \to CD$ determina tutto lo schema R_1); in R_2 la chiave è C (che con $C \to EF$ determina tutto R_2). Entrambe in BCNF, e le DF di partenza sono interamente coperte dai due nuovi schemi.

Esercizio 4

$R(A, B, C, D)$ è in BCNF. Sapendo che su R sussiste la DF $D \to B$, è vero che anche $D \to C$ sussiste? (No / Solo se anche B→D sussiste / Sì / Dipende, non è possibile saperlo con certezza)

Soluzione: Sì — poiché R è in BCNF, ogni DF non banale come $D \to B$ implica che il determinante D sia una **superchiave**. Se D è superchiave, determina **tutti** gli altri attributi di R, incluso C.

8.9 Verificare una DF su un'Istanza

Data un'istanza r di R, **è possibile** stabilire con certezza che r **viola** una DF $A \to B$ (basta trovare due tuple con stesso A e diverso B); non è invece possibile dimostrare che una DF **vale**, dato che una violazione potrebbe comparire in un'istanza futura.

Esercizio 5

Data un'istanza di una relazione S, è possibile determinare se S **non** soddisfa una DF analizzando le tuple presenti nell'istanza. (Vero / Falso)

Soluzione: Vero — se si trovano due tuple con gli stessi valori per gli attributi determinanti ma valori diversi per gli attributi determinati, si ha la prova che la DF è violata.

Esercizio 6

Sia data $R(A, B, C, D)$. La seguente query verifica se la DF $A \to B$ è soddisfatta da una qualunque istanza di R (0 righe = nessuna violazione; altrimenti una o più tuple violano la DF)?

```
SELECT COUNT(*)
FROM R AS R1, R AS R2
WHERE (R1.B != R2.B) AND (R1.A = R2.A)
```

(Vero / Falso)

Soluzione: Vero — la query cerca esattamente le violazioni della definizione di DF: coppie di tuple con lo stesso valore di A ma valori diversi di B.

8.10 Esempio Completo: dalle Chiavi alla BCNF

(Ramakrishnan & Gehrke, Exercise 19.2 — stesso schema usato per l'algoritmo delle chiavi candidate nella sezione precedente.)

Esercizio 7

$R(A, B, C, D, E)$ con $F = \{A \to B,\ BC \to E,\ ED \to A\}$. (1) Elencare tutte le chiavi di R. (2) R è in 3NF? (3) R è in BCNF?

Soluzione: (1) Chiavi candidate: **ACD, BCD, CDE** (già calcolate con l'algoritmo delle chiavi candidate — Sez. precedente). Dato che la loro unione copre $\{A, B, C, D, E\}$, **ogni** attributo di R è primo — non esistono attributi non-primi. (2) **È in 3NF**: per ogni DF non banale $X \to A$ basta che A sia primo, condizione qui sempre soddisfatta. (3) **Non è in BCNF**: la DF $A \to B$ viola la condizione, perché $(A)^+ = AB \neq ABCDE$ — A non è una superchiave.

9 Metodi di Storage e Modello di Costo

9.1 Introduzione

Le basi di dati sono normalmente memorizzate su **dischi magnetici**. Gerarchia di memorizzazione:

- **primaria**: CPU, memoria principale, cache;
- **secondaria**: dischi magnetici, memoria flash, dischi a stato solido;

- **terziaria**: media rimovibili.

L'**organizzazione dei file** determina come le tuple sono fisicamente memorizzate sul disco e come vi si accede.

9.2 Record e Blocchi del Disco

Un **record** è una collezione di valori collegati tra loro (numerico, stringa, booleano, data/ora, o BLOB — *Binary Large Object*, per dati non strutturati). La lunghezza di un record può variare (campi opzionali o di lunghezza variabile).

I file sono allocati in **blocchi** del disco (dette **pagine** in ambito DB):

- **record esteso** (*spanned*): più grande di un singolo blocco — un puntatore alla fine del primo blocco rimanda al blocco col resto del record;
- **record non esteso** (*unspanned*): non supera le dimensioni di una pagina.

9.3 Operazioni su File

- di **sola lettura** (R): `Open, Find, Read, FindNext, Close, Scan` — nessun cambiamento ai dati;
- di **aggiornamento** (U): `Delete, Insert` — modificano il file.

Tipicamente i record sono selezionati mediante condizioni di selezione.

9.4 Tuple, Tabelle, Pagine

id	name	city	age
1	Marco	Trento	33
2	Andrea	Cosenza	24
3	Giuseppe	Milano	27
4	Federica	Trento	20
5	Rossana	Cosenza	18
6	Laura	Trento	25

Tabella 7: Relazione R usata come riferimento per gli esempi di costo

La prima tupla occupa 19 byte ($4 + 5 + 6 + 4$). Per semplicità si assume che **ogni tupla occupi lo stesso spazio** (quello della tupla più lunga); si indica con t_R la dimensione di una tupla di R. Il DBMS raggruppa più settori del disco in una **pagina** (es. 16KB di default in MySQL); per ogni operazione (lettura o scrittura), il DBMS carica sempre pagine intere.

9.5 Il Modello di Costo

Si introduce un modello di costo basato sul **numero di pagine** che il DBMS deve leggere/scrivere per eseguire un'operazione. Le operazioni in memoria centrale sono molto più veloci di quelle su disco: il modello **ignora** il costo in memoria centrale e si concentra solo sulla memoria secondaria. Si indica con P_R il numero di pagine che contengono le tuple di R.

9.6 Metodi di Memorizzazione di una Relazione

- **Heap file** (o *pile*): le tuple sono scritte una dopo l'altra nell'ordine di inserimento;
- **Sorted file**: le tuple sono ordinate in base al valore di un attributo (o insieme di attributi);
- **Indexed file**: un indice separato accompagna il file che contiene le tuple (vedi prossima sezione).

9.7 Heap File: Costo delle Operazioni Principali

Inserimento: molto facile e veloce — le tuple si memorizzano nell'ordine di arrivo; quando una pagina è piena si passa alla successiva (eventualmente lasciando spazio vuoto in fondo alla pagina precedente, per evitare di dover leggere due pagine per una singola tupla a cavallo).

- **SCAN** (`SELECT * FROM R;`): $Costo_{Scan} = P_R$ — bisogna leggere tutte le pagine che contengono le tuple di R.
- **EQUALITY SEARCH** (`SELECT * FROM R WHERE city = 'Trento';`): $Costo_{EqSearch} = P_R$ — stesso costo dello scan: senza indice né ordinamento, bisogna leggere **tutte** le pagine per essere sicuri di aver trovato tutte le tuple che soddisfano il filtro.
- **INSERT**: $Costo_{Insert} = 2$ — si legge l'ultima pagina del file (creandone una nuova se non c'è spazio) e si riscrive la pagina modificata. *Nota*: per trovare l'ultima pagina basta $P_R = S/P$ (dimensione del file S diviso dimensione di una pagina P) e leggere la pagina $P_R - 1$.
- **DELETE** (`DELETE FROM R WHERE city='Trento';`): se $|R_{A=c}|$ è il numero di tuple in cui l'attributo A vale c:

$$Costo_{Delete} = P_R + |R_{A=c}|$$

9.7.1 Stimare $|R_{A=c}|$: il Selectivity Factor

Se $|R_{A=c}|$ non è noto a priori, si approssima assumendo una **distribuzione uniforme** dei valori: sia $|R|$ il numero di tuple e $|R.A|$ il numero di valori **distinti** dell'attributo A,

$$|R_{A=c}| \approx \left\lceil \frac{|R|}{|R.A|} \right\rceil$$

Il rapporto $f = \dfrac{1}{|R.A|}$ è il **selectivity factor** di A; quindi:

$$Costo_{Delete} = P_R + |R_{A=c}| = P_R + \lceil f \cdot |R| \rceil$$

Attenzione: è solo un'approssimazione — assume una distribuzione uniforme dei valori, che non sempre è garantita.

Esempio. Sulla tabella R sopra: $|R.city| = 3$ (Trento, Cosenza, Milano sono i 3 valori distinti). $\dfrac{|R|}{|R.city|} = \dfrac{6}{3} = 2$: ci si aspettano cioè 2 tuple per ogni valore di `city` (in questo caso, coincide esattamente col dato reale: 3 tuple con `city='Trento'`).

9.8 Sorted File: Costo delle Operazioni Principali

- **SCAN**: identico all'heap file, $Costo_{Scan} = P_R$.
- **EQUALITY SEARCH**: si usa la **ricerca binaria** (si legge la pagina centrale; se il valore cercato segue/precede alfabeticamente, si esclude metà delle pagine rimanenti, e si ripete):
$$Costo_{EqSearch} = \lceil \log_2(P_R) \rceil + \left\lceil \frac{|R_{city='Trento'}|}{\lceil P/t_R \rceil} \right\rceil$$
Il primo termine è il costo della ricerca binaria per trovare la prima pagina con `'Trento'`; il secondo è il numero di pagine aggiuntive da leggere per ottenere tutte le tuple corrispondenti (P/t_R = tuple per pagina).
- **RICERCA PER INTERVALLO** (es. `WHERE age BETWEEN 18 AND 30`): stessa idea — ricerca binaria per il primo valore, poi si leggono tutte le pagine successive contenenti valori nell'intervallo. La formula non cambia concettualmente, una volta note le tuple/pagine coinvolte.
- **DELETE**: ricerca binaria (costo $\lceil \log_2(P_R) \rceil$) + **riscrittura** delle pagine modificate (si trascura il costo di "compattare" lo spazio liberato):
$$Costo_{Delete} = \lceil \log_2(P_R) \rceil + 2 \cdot \left\lceil \frac{|R_{city='Trento'}|}{\lceil P/t_R \rceil} \right\rceil$$
(il fattore 2 è perché le pagine coinvolte vanno sia **lette** sia **riscritte**).
- **INSERT**: si cerca il punto d'inserimento con ricerca binaria, poi si riscrive la pagina modificata:
$$Costo_{Insert} = \lceil \log_2(P_R) \rceil + 1$$
Nel **caso peggiore** (pagina piena, serve spostare tutte le tuple successive) l'INSERT può richiedere di leggere e riscrivere tutte le P_R pagine. In ogni caso, **l'INSERT in un sorted file è sempre più costoso** che in un heap file.

10 Indici

10.1 Indicizzazione dei File

Un **indice** è una struttura ausiliaria che rende più efficienti le operazioni su un file di dati, indipendentemente dal fatto che quest'ultimo sia ordinato o meno:

- **data file**: il file che contiene i dati veri e propri;
- **index file**: il file che contiene l'indice (può essercene più di uno per relazione).

I due tipi più usati dai DBMS più comuni: **B+-tree** e **Hash**.

10.1.1 Indici Primari, Secondari, Unici

- **Primario**: contiene la chiave primaria della relazione — nessun duplicato;
- **Secondario**: la chiave di ricerca è diversa dalla chiave primaria — può contenere duplicati;
- **Unico**: la chiave di ricerca è una chiave candidata — nessun duplicato.

10.2 B+-Trees

Indice dinamico ad **albero**, dove i puntatori ai dati sono memorizzati **solo nei nodi foglia**:

- i **nodi interni** contengono *index entries* e guidano la ricerca;
- i **nodi foglia** contengono i puntatori ai dati nel data file;
- i nodi foglia sono collegati tra loro (*sequence set*) — permette scansioni ordinate e ricerche per intervallo.

Proprietà principali:

- inserimento/cancellazione mantengono l'albero **bilanciato** (tutte le foglie alla stessa profondità);
- occupazione garantita di **almeno il 50%** di ogni nodo (eccetto la radice);
- la ricerca richiede solo di **discendere** l'albero fino al nodo foglia appropriato;
- l'**altezza** (lunghezza di un qualunque cammino radice-foglia) raramente supera 3 o 4.

10.2.1 Costo del Lookup

Il **costo di lookup** è il numero di pagine da aprire per trovare il nodo foglia con i puntatori cercati. Sia B il numero massimo di figli di un nodo interno e $|R.A|$ il numero di valori distinti della chiave di ricerca A: a ogni passo si divide per B il numero di elementi rimanenti, quindi:

$$L_b = \lceil \log_B(|R.A|) \rceil$$

Se B e $|R.A|$ non sono noti, si assume per convenzione un costo di lookup di **default pari a 3**.

Esempio (B+tree vs. sorted file). Con $P_R = 1.000.000/100 = 10.000$: $\log_2(10.000) \approx 13$ (ricerca binaria su sorted file); con $B = 100$ e $|R.A| = 400$: $\log_{100}(400) \approx 1{,}3$ (lookup B+tree) — molto più efficiente, perché (a) non ci sono i costi di mantenimento dell'ordinamento del file dati, e (b) il logaritmo è in base B (tipicamente grande) invece che in base 2.

10.2.2 Costo delle Operazioni

- **SCAN**: $Costo_{Scan} = P_R$ (uguale a heap/sorted file — si aprono comunque tutte le pagine del data file);
- **EQUALITY SEARCH, file <u>unclustered</u>** (dati non ordinati secondo la chiave indicizzata):

  $$Costo_{EqSearch} = L_b + |R_{A=c}|$$

  un accesso a disco **per ogni** tupla trovata, dato che le tuple corrispondenti possono trovarsi su pagine qualsiasi;

- **EQUALITY SEARCH, file clustered** (dati fisicamente ordinati secondo la chiave indicizzata):
$$Costo_{EqSearch} = L_b + \left\lceil \frac{|R_{A=c}|}{\lceil P/t_R \rceil} \right\rceil$$
le tuple corrispondenti sono **fisicamente vicine**, quindi bastano poche pagine aggiuntive (tuple-per-pagina) invece di una per tupla;

- **INSERT**: $Costo_{Insert} = L_b + 1 + 1$ (lookup, più eventuale creazione di una nuova pagina, più scrittura della pagina modificata — **non** si considera qui il costo di aggiornare l'indice stesso).

10.3 Hash Indexes

Si sceglie una **funzione di hash** h (es. per interi, $h(x) = x \bmod N$) e si allocano i valori dell'attributo indicizzato in **bucket** in base al valore di h. Se un bucket riceve troppi puntatori, il DBMS crea una **pagina di overflow** collegata al bucket principale.

Per trovare una tupla con `age`$= x$: si applica h a x (costo 0, in memoria centrale) per trovare il bucket, poi si legge la pagina (o le pagine) di quel bucket nell'index file.

10.3.1 Costo delle Operazioni

Sia L_h il costo di lookup nel file di indice (il calcolo di h è gratuito; il costo è nella lettura della pagina del bucket — comprese eventuali pagine di overflow, per cui si assume di default un valore medio di 1,2):

- **EQUALITY SEARCH**: $Costo_{EqSearch} = L_h + |R_{A=c}|$ — **molto efficiente**;

- **RICERCA PER INTERVALLI**: **non efficiente** — non è garantito che valori vicini finiscano nello stesso bucket, quindi serve un lookup per ciascun valore dell'intervallo; un hash index non permette di leggere le tuple in ordine rispetto alla chiave;

- **INSERT**: $Costo_{Insert} = L_h + 1 + 1$ (si apre l'ultima pagina del file dati e si appende la tupla, costo 1+1; più il lookup per trovare il bucket in cui inserire il puntatore, costo L_h — anche qui si ignora il costo di aggiornare l'indice);

- **DELETE**: $Costo_{Delete} = L_h + 2 \cdot |R_{A=c}|$ (lookup per trovare il bucket, poi si apre una alla volta ogni pagina con tuple da cancellare, le si modifica e riscrive: fattore 2 per lettura+scrittura);

- **SCAN**: $Costo_{Scan} = P_R$ — si ignora l'index file e si accede direttamente al data file.

Considerazioni generali: l'hash index è molto efficiente, tranne che per ricerche per intervallo. Nel tempo i bucket accumulano sempre più pagine di overflow (aumentando il costo di lookup): il DBMS deve periodicamente riorganizzare l'index file con più bucket (**Extendible Hash Index**), spostando i puntatori esistenti nei nuovi bucket.

10.4 Index Matching

Con più condizioni nella clausola WHERE (es. `city='Trento' AND age=30`), un indice **soddisfa** (*matcha*) un predicato di selezione se può essere usato per valutarlo.

- **Indice hash** su $(A, B, \ldots)$: soddisfa `pred1 AND pred2 AND ...` se e solo se **tutti** gli attributi della chiave di ricerca compaiono in un predicato di **uguaglianza** ($=$). Es. con indice hash su (A, B): `WHERE A=10 AND B=5` $\rightarrow$ MATCH; `WHERE A=5` (da solo) $\rightarrow$ NO MATCH (manca B).
- **Indice B+tree** su $(A, B, \ldots)$: soddisfa la selezione se e solo se gli attributi nei predicati costituiscono un **prefisso** della chiave di ricerca (i predicati possono usare un operatore qualsiasi: $=, <, >, \ldots$, non solo l'uguaglianza).

Selezioni che matchano più indici (es. hash su A, B+tree su (B, C), selezione $A{=}7 \wedge B{=}5 \wedge C{=}4$): possibilità — (a) usare l'hash su A e poi filtrare B, C sulle tuple recuperate; (b) usare il B+tree su (B, C) e poi filtrare A; (c) usare **entrambi**, fare l'intersezione dei RID risultanti, e solo dopo recuperare le tuple.

Selezioni con disgiunzione (`OR`): se **nessun** indice copre **tutti** i predicati in OR, l'unica soluzione è lo **SCAN** completo (es. hash su A e su B, selezione $A{=}7 \vee B{>}5$: l'hash su B non aiuta con $>$, quindi solo $A{=}7$ matcha — non basta, serve SCAN). Se invece **ogni** predicato in OR ha un indice che lo copre (es. hash su A, B+tree su B, stessa selezione $A{=}7 \vee B{>}5$): si può fare SCAN, **oppure** usare i due indici separatamente e fare l'**unione** dei RID.

10.5 Tabella Riassuntiva dei Costi

	Scan	**EqSearch**	**Insert**	**Delete**
Heap file	P_R	P_R	$1+1$	$P_R + \lvert R_{A=c}\rvert$ [ma vedi nota]
Sorted file	P_R	$\lceil \log_2 P_R \rceil + \lceil \frac{\lvert R_{A=c}\rvert}{\lvert P/t_R \rvert} \rceil$	$\lceil \log_2 P_R \rceil + 1$	$\lceil \log_2 P_R \rceil + 2\lceil \frac{\lvert R_{A=c}\rvert}{\lvert P/t_R \rvert} \rceil$
B+tree unclustered	P_R	$L_b + \lvert R_{A=c}\rvert$	$L_b + 1 + 1$	$L_b + 2\lvert R_{A=c}\rvert$
B+tree clustered	P_R	$L_b + \lceil \frac{\lvert R_{A=c}\rvert}{\lvert P/t_R \rvert} \rceil$	$L_b + 1 + 1$	$L_b + 2\lceil \frac{\lvert R_{A=c}\rvert}{\lvert P/t_R \rvert} \rceil$
Hash index	P_R	$L_h + \lvert R_{A=c}\rvert$	$L_h + 1 + 1$	$L_h + 2\lvert R_{A=c}\rvert$

Tabella 8: Costi delle operazioni su singola relazione

Nota sulla discrepanza: la tabella riassuntiva di queste slide riporta il DELETE su heap file come $P_R + 2 \cdot |R_{A=c}|$ (stesso fattore 2 di sorted file/B+tree/hash); la derivazione esplicita nel deck sul modello di costo (Argomento 9) dà invece $P_R + |R_{A=c}|$, **senza** il fattore 2. Le due fonti dello stesso corso si contraddicono su questa singola cella — vale la pena chiarirlo con il materiale del docente prima di usarla in sede d'esame.

Algoritmo	**Costo di** $R \bowtie_{R.B=S.C} S$
Nested loop join	$P_R + P_R \cdot P_S$
Sort-merge join	$Costo_{OrdR} + Costo_{OrdS} + P_R + P_S$
Hash join	$(P_R + 2\|R\|) + (P_S + 2\|S\|) + (P_R + P_S)$
Index Nested Loop (indice su S.C)	$P_R + \|R\| \cdot Costo_{EqSearchS}$

Tabella 9: Costi degli algoritmi di join tra $R(A, B)$ e $S(C, D)$ — riportata così com'è nelle slide, senza derivazione: la trattazione dei costi di join non è coperta da nessuno slide-deck caricato finora

Esercizio 1

Indice B+-tree **unclustered** sull'attributo CITY di una tabella PERSON, costo di lookup = 3, selectivity factor = 0,1, 1000 tuple nella tabella (ogni tupla 500KB, ogni pagina 600KB su disco). Qual è il costo di trovare, accedere e mostrare le tuple con `CITY='Trento'`?
(a) $(3 + 1) \cdot 0{,}1 \cdot 1000$ (b) $3 \cdot 0{,}1 \cdot 1000$ (c) $3 + 0{,}1 \cdot 1000$ (d) $3 \cdot 0{,}1 + 1000$ (e) $3 \cdot (1 + 0{,}1 \cdot 1000)$

Soluzione: (c) $3 + 0{,}1 \cdot 1000$ — applicazione diretta della formula per B+tree **unclustered**: $Costo_{EqSearch} = L_b + |R_{A=c}|$. Il costo di lookup $L_b = 3$ è dato direttamente. $|R_{A=c}| = f \cdot |R| = 0{,}1 \cdot 1000 = 100$ (selectivity factor $\times$ tuple totali). Essendo l'indice **unclustered** e ogni tupla grande quasi quanto una pagina intera (500KB su 600KB), ogni tupla trovata richiede un accesso a disco separato — da qui il $+|R_{A=c}|$ invece della versione "clustered" con la divisione per tuple-per-pagina.

Esercizio 2

Tabella `Train(UserID, Gender, productID)`, 7.150.884 righe, senza indice, non ordinata. `SELECT * FROM Train WHERE productID LIKE '%7085%';` richiede tempo T_1. Creando un indice B+tree su `productID`, la stessa query richiede tempo T_2. Cosa si può concludere? ($T_2 > T_1$ / non si può concludere nulla / $T_1 > T_2$ / $T_1 = T_2$)

Soluzione: $T_1 = T_2$ — per la regola di **index matching** dei B+tree, l'indice è utilizzabile solo se il predicato costituisce un **prefisso** della chiave di ricerca. Un `LIKE '%7085%'` con wildcard **iniziale** non fissa alcun prefisso (il valore cercato può comparire ovunque nella stringa): il B+tree, essendo ordinato per prefisso, non può restringere la ricerca. Il DBMS deve eseguire comunque una scansione completa (SCAN) in entrambi i casi.

Esercizio 3

$R[A, B]$ con P_R pagine e $|R|$ tuple; $S[B, C]$ con P_S pagine e $|S|$ tuple; **hash index** su $S.B$ con costo di lookup = 2. Costo di `update R,S set R.A=10 where R.A=S.B`?
(a) $P_R + P_S + 2|R|$ (b) $P_R + P_R + 2|R|$ (c) $P_R + P_S + 2|S|$ (d) $P_R + |R| \cdot P_S \cdot 2$ (e) $P_S + |S| \cdot P_R \cdot 2$

Soluzione: (b) $P_R + P_R + 2|R|$ — si leggono tutte le pagine di R (P_R, obbligatorio per esaminare ogni tupla); per **ogni** tupla di R si esegue un lookup sull'hash index di $S.B$ per trovare le tuple di S che soddisfano la condizione di join ($|R| \cdot L_h = |R| \cdot 2$); infine si

riscrivono le pagine di R modificate (caso peggiore, un altro P_R). Si noti che $L_h = 2$ qui gioca lo stesso ruolo del termine di lookup nella formula generale di EqSearch vista sopra, applicato tupla per tupla nel join.

11 Algoritmi di JOIN e Costi

11.1 Il JOIN

Finora si sono considerate solo operazioni su una **singola** tabella; una delle operazioni più frequenti (e costose) in SQL è però il **JOIN** tra tabelle. Riferimento: $R(A, B) \bowtie S(C, D)$ con condizione $B = C$, dove R è la **outer table** e S la **inner table**.

Assunzioni: il costo del JOIN è il numero di pagine che l'algoritmo deve leggere e/o scrivere; le tuple prodotte vengono mostrate a video via via che sono generate (non riscritte su disco) — quindi non c'è costo di scrittura per il risultato.

11.2 Nested Loop Join (NLJ)

L'algoritmo più semplice:

```
foreach tuple t in R do:
    foreach tuple t' in S do:
        if t.B = t'.C then output the tuple (t, t')
```

(versione a tuple: $|R| + |R| \cdot |S|$ tuple lette). La versione **realistica** opera a livello di **pagine**: per ogni pagina di R, si scandisce ogni pagina di S, confrontando tutte le coppie di tuple al loro interno.

$$Costo_{NestedLoopJoin} = P_R + P_R \cdot P_S$$

L'ordine delle tabelle conta: conviene sempre usare come *outer table* quella con **cardinalità (in pagine) minore**. Il costo del NLJ ingenuo è molto elevato: con $P_R = P_S = 1000$, il costo è 1.001.000 operazioni di I/O.

11.3 (Sort-)Merge Join

Assume che R e S siano già **ordinate** rispetto agli attributi di join (B e C):

1. si prende il primo valore di B in R;
2. si scorrono i valori di C in S: se si trova lo stesso valore (anche più volte), si genera la tupla di output; se si trova un valore **più alto**, si ripete il processo invertendo i ruoli di R e S (si scarta il valore più basso e si riparte da lì).

Nessuna tupla viene mai riletta: si esaminano solo $|R| + |S|$ tuple in totale, cioè $P_R + P_S$ pagine — con $P_R = P_S = 1000$: solo 2000 pagine, contro le 1.001.000 del NLJ!

11.3.1 Costo, se R e S *non* sono già ordinate

Bisogna aggiungere il costo di ordinarle:

$$Costo_{SortMergeJoin} = Costo_{OrdR} + Costo_{OrdS} + P_R + P_S$$

Nota: in alcuni casi il NLJ resta preferibile — se S è abbastanza piccola da stare tutta in RAM, il DBMS la mantiene in memoria e il costo del NLJ scende a $P_R + P_S$ (uguale al merge join, ma senza il costo di ordinamento).

11.3.2 Algoritmi di External Merge Sort

Si applicano quando i dati da ordinare non entrano in RAM e serve appoggiarsi a memoria esterna (disco). Sia P_R il numero di pagine di R e B il numero di pagine caricabili in RAM (tipicamente centinaia): il costo di ordinare R è

$$Costo_{OrdR} = 2 \cdot P_R \cdot \left(\left\lceil \log_{B-1} \left\lceil \frac{P_R}{B} \right\rceil \right\rceil + 1 \right)$$

11.4 Hash Join

Usa tabelle hash per velocizzare il JOIN: una funzione di hash (es. $h(x) = x \bmod N$, con N = numero di bucket) divide le tuple di R e S in bucket in base al valore dell'attributo di join. Se t e t' possono andare in join, allora $t.B = t'.C$, quindi hanno necessariamente lo **stesso hash** (il viceversa **non** vale: stesso hash non implica valori uguali).

Algoritmo: il DBMS crea una copia di R e S con le tuple riorganizzate per bucket; poi, bucket per bucket: legge e carica in memoria il bucket i di R e di S, calcola in memoria il join tra i due, passa al bucket successivo. Le tuple in un bucket di R possono fare match **solo** con tuple nel bucket corrispondente di S.

$$Costo_{HashJoin} = (P_R + 2|R|) + (P_S + 2|S|) + (P_R + P_S)$$

(costo di creare le due copie ripartizionate, più il costo di leggere i bucket per calcolare il join). Se le tuple sono **già** distribuite per bucket secondo la funzione di hash scelta, il costo si riduce a $P_R + P_S$. La scelta tra NLJ, merge join e hash join (in base a $P_R, P_S, |R|, |S|$) è compito dell'ottimizzatore del DBMS.

11.5 Index Nested Loop Join

Variante del NLJ che assume un **indice sulla inner table**: si leggono tutte le pagine della outer table (R) e, per ciascuna tupla, si cercano tramite indice le tuple di S che soddisfano la condizione di join:

$$Costo_{IndexNestedLoopJoin} = P_R + |R| \cdot Costo_{EqSearchS}$$

dove $Costo_{EqSearchS}$ dipende dal tipo di indice presente su S (le formule di EqSearch viste per B+tree/hash — Argomento 10). **Attenzione**: il vantaggio si ottiene solo se l'indice è sull'attributo della **inner table** — l'indice non aiuta lo scan della outer table.

11.6 Aggiustamento degli Indici

Obiettivi: valutare dinamicamente i requisiti e riorganizzare gli indici per la miglior performance. Motivi per rivedere le scelte iniziali:

- alcune query richiedono troppo tempo per mancanza di un indice adatto;
- alcuni indici risultano poco utilizzati;
- alcuni indici sono soggetti ad aggiornamenti troppo frequenti (costo di mantenimento elevato).

11.7 Decisioni Progettuali a Livello Fisico

- **Se indicizzare** un attributo: es. è chiave, oppure è usato in una o più query frequenti;
- **Quali attributi** indicizzare: singolo attributo o combinazione (indice multi-colonna);
- **Se prevedere un clustered index**: al **massimo uno** per tabella (i dati possono essere fisicamente ordinati in un solo modo);
- **Hash o B+tree**: gli indici hash **non** supportano ricerche per intervallo;
- **Hashing dinamico o meno**: l'*Extendible Hash Index* è adatto quando i dati sono molto volatili (inserimenti/cancellazioni frequenti).

12 Piani di Esecuzione delle Query

12.1 Esecuzione di Query SQL

Per ogni query esistono almeno due livelli di pianificazione:

- **pianificazione logica**: decidere **cosa** fare per produrre il risultato (selezione, proiezione, join, unione, ...);
- **pianificazione fisica**: decidere **come** eseguire i singoli passi del piano logico (file scan, nested loop join, hash join, ...).

Piani diversi per la **stessa** query possono avere costi molto differenti. Qui ci si limita a due aspetti della stima del costo:

- la stima della **cardinalità** di ogni passo intermedio;
- il costo di ogni singolo passo in termini di **I/O** (visto nelle lezioni precedenti).

(Argomenti più avanzati non trattati: lo spazio di ricerca dei piani possibili, e gli algoritmi per esplorarlo — competenza dell'*ottimizzatore* del DBMS.)

12.2 Cosa Serve per il Calcolo

- la **cardinalità** delle relazioni coinvolte;
- la presenza (e il tipo) di **indici**, e il numero di chiavi nell'indice;
- il numero di **pagine** usate per ogni relazione;
- informazioni **statistiche** sugli attributi (valore minimo/massimo, numero di valori distinti, ...).

12.3 Albero di Esecuzione della Query

Rappresentazione tipica dei passi di esecuzione:

- le **foglie** sono le relazioni di partenza;
- i **nodi intermedi** sono operazioni algebriche (con un costo associato a ciascuno);
- la **radice** è l'output della query (anch'esso, concettualmente, una relazione).

12.3.1 Esempio di Piano Fisico (illustrativo)

(Notazione ripresa da un corso esterno, solo per mostrare come si annota un piano fisico — non fa parte dell'esempio numerico principale che segue.) Per la query *"nomi dei marinai che hanno prenotato la barca di nome '100'"*, un piano fisico possibile:

- π_{sname} [**on-the-fly**, cioè calcolato al volo senza materializzare un risultato intermedio su disco]
  - **Nested Loop Join** tra Sailors [**table scan**] e:
    - **Nested Loop Join** tra Reserves [**table scan**] e $\sigma_{bname=100}$(Boats) [**table scan**]

12.4 Esempio Completo: Cardinalità e Costo

Query: `SELECT S.sid, R.rname FROM Sailors S, Boats B, Reserves R WHERE S.sid=R.sid AND B.bid=R.bid AND B.color=red`

Albero logico: Output $\leftarrow \sigma_{color=red} \leftarrow (\bowtie_{B.bid=R.bid}) \leftarrow [(\bowtie_{S.sid=R.sid} (\text{Sailors}, \text{Reserves})),\ \text{Boats}]$

Assunzioni: $|S|$=12 (Sailors), $|B|$=4 (Boats), $|R|$=10 (Reserves); f(color)=1/3, $f(B.bid{=}R.bid)$=1/4, $f(S.sid{=}R.sid)$=1/12; $t_S{=}t_B{=}t_R$=25 byte, P=50 byte ($\Rightarrow P/t_x = 2$ tuple/pagina); P_S=6, P_B=2, P_R=5.

12.4.1 Calcolo della Cardinalità dell'Output

La cardinalità si approssima **bottom-up** nell'albero. Limite superiore: $|Res| \leq |R| \cdot |S| \cdot |B| \leq 10 \cdot 12 \cdot 4 \leq 480$, ridotto progressivamente da ciascun *selectivity factor* f:

- $S.sid{=}R.sid$: $12 \cdot 10 \cdot \frac{1}{12} = 10$;
- $B.bid{=}R.bid$: $4 \cdot 10 \cdot \frac{1}{4} = 10$;
- $color{=}'red'$: $10 \cdot \frac{1}{3} = 3{,}4$.

(Il selectivity factor è comunque sempre una **stima**, non un valore esatto.)

12.4.2 Calcolo del Costo (I/O)

- **Primo JOIN** (Reserves $\bowtie$ Sailors, Nested Loop con Reserves come outer — ha meno pagine): $Costo = P_R + P_S \cdot P_R = 5 + 6 \cdot 5 = 35$;
- **Secondo JOIN** (risultato $\bowtie$ Boats): $Costo = P_B = 2$ — l'output del primo join è **già in memoria** (on-the-fly), quindi basta caricare Boats e verificare al volo la condizione;
- **Selezione** *color=red*: costo **nullo** — eseguita in memoria sull'output delle operazioni precedenti, senza ulteriore I/O.

$$\textbf{Costo totale} = 35 + 2 + 0 = 37$$

12.4.3 Variante con Indice

Supponendo un **hash index** su `Sailors.sid` (con L_h=1): il costo del primo JOIN diventa un **Index Nested Loop Join** (Sez. Argomento 11):

$$Costo = P_R + |R| \cdot (L_h + |S_{S.sid=R.sid}|) = 5 + 10 \cdot (1 + 1) = 25$$

(contro i 35 della versione senza indice) — si scandisce Reserves (P_R=5) e, per ciascuna delle $|R|$=10 tuple, si fa un lookup sull'indice hash di Sailors (L_h=1) più l'accesso a ogni tupla trovata (qui, in media 1 per valore di sid, essendo sid la chiave primaria di Sailors).

Esercizio

Relazioni $R(\underline{A}, B)$, $S(\underline{B}, C)$, $U(\underline{G}, H)$ (chiavi primarie sottolineate). Su $S.B$ esiste un B+-tree con costo di lookup $L_b = 3$. Calcolare il costo di esecuzione della query secondo il piano fisico raffigurato nella slide (foglie $R(A, B)$ e $S(B, C)$ unite con un **Index Nested Loop Join** sfruttando l'indice su $S.B$; il risultato combinato ulteriormente con $U(G, H)$ e con una selezione intermedia).

Nota mia: l'albero esatto (quali attributi nella selezione intermedia, come esattamente si combina con $U(G, H)$) è disegnato a mano nella slide originale e non sono riuscito a trascriverlo con sicurezza sufficiente da qui — piuttosto che inventare condizioni plausibili, ti rimando alla slide stessa per completare l'esercizio. Lo schema generale però è chiaro: il join $R \bowtie S$ va calcolato come Index Nested Loop ($Costo = P_R + |R| \cdot (L_b + |S_{B=c}|)$, con R come outer table e l'indice su $S.B$ usato per l'inner), esattamente come nella variante indicizzata appena vista sopra.

13 Esercizi Supplementari da Appelli Precedenti

Provenienza: appelli del 20/01/2023, in lingua originale inglese, di un'edizione del corso con un docente diverso da quello attuale. Tradotti qui in italiano; usare con cautela dove la terminologia o le convenzioni differiscono da quelle di Bouquet (segnalato caso per caso).

13.1 Dipendenze Funzionali e Normalizzazione

Esercizio 1

Dato $\langle R(A,B,C,D), \{AB \to C,\ C \to D,\ C \to A\}\rangle$ — lo stesso schema usato per l'esercizio sulle chiavi candidate (Argomento 7: chiavi candidate AB, BC) — dire se R è in 3NF, e in caso negativo fornire una decomposizione lossless in 3NF.

Soluzione: non è in 3NF. Attributi primi: A, B, C (appartengono alle chiavi AB/BC); D è l'unico non-primo. La DF $C \to D$ viola la 3NF: C non è superchiave ($(C)^+ = ACD \neq ABCD$) e D non è primo — nessuna delle due condizioni della 3NF è soddisfatta. Decomposizione: $R_1(A,B,C)$ con $\{AB \to C, C \to A\}$ (chiave AB, in 3NF) e $R_2(C,D)$ con $\{C \to D\}$ (chiave C, in 3NF). **Lossless**: l'attributo comune C determina interamente R_2 ($C \to D$).

Esercizio 2

Due insiemi di DF F_1 e F_2 su uno schema S sono equivalenti se: (applicando la chiusura transitiva a entrambi si ottiene lo stesso insieme di DF / sono esattamente lo stesso insieme di DF / hanno la stessa chiusura / non abbiamo abbastanza informazioni)

Soluzione: hanno la stessa chiusura ($F_1^+ = F_2^+$) — è la definizione di equivalenza logica tra insiemi di DF; non serve che siano letteralmente lo stesso insieme, basta che implichino le stesse DF.

Esercizio 3

Dato $R(A,B,C,D)$, quale dei seguenti insiemi di DF è una copertura minimale? ($AB \to C, B \to D, B \to A$ / $AB \to C, D \to BC$ / $AB \to C, C \to D, D \to C$ / $AB \to A, C \to BD$)

Soluzione: $AB \to C,\ D \to BC$ — *nota sulla notazione*: qui $D \to BC$ è scritto in forma compatta invece che come $D \to B, D \to C$ (la forma a singolo attributo richiesta esplicitamente da Bouquet); sono equivalenti, ma se ti viene chiesto di scrivere una copertura minimale all'esame di Bouquet, scomponi comunque in DF a un solo attributo a destra.

Esercizio 4

Dato $\langle R(A,B,C,D), \{AB \to C, C \to D, C \to A\}\rangle$ e la decomposizione $R_1(A,B,C)$, $R_2(C,D)$: è in 3NF? (non ci sono abbastanza informazioni / Sì / No)

Soluzione: Sì — è esattamente la decomposizione corretta ricavata nell'Esercizio 1 sopra: entrambi gli schemi risultanti sono in 3NF.

Esercizio 5

L'insieme di DF $\{AB \to C, C \to D, C \to A\}$ implica logicamente: ($AC \to D, C \to B$ / $AB \to D, C \to B$ / nessuna delle altre risposte è corretta / $AC \to D, AB \to D$)

Soluzione: $AC \to D$, $AB \to D$ — verifica diretta: $(AC)^+ = ACD \ni D$ (via $C \to D$) $\Rightarrow AC \to D$; $(AB)^+ = ABCD \ni D$ (via $AB \to C$, poi $C \to D$) $\Rightarrow AB \to D$. Le altre opzioni includono $C \to B$, falsa: $(C)^+ = ACD$ non contiene B.

13.2 Vincoli di Integrità e Chiavi Esterne

Esercizio 6

Cos'è una chiave esterna (*foreign key*)? (un vincolo di integrità / una query SQL / un trigger / nessuna delle altre opzioni)

Soluzione: un vincolo di integrità.

Esercizio 7

`Student(Sid,Name,Login,Age,GPA)`: (53666,Kayne,A@ca,28,4.2), (53655,Tupac,B@ca,26,3.5), (53688,Bieber,C@ca,22,3.9). `Enrolled.Sid` è chiave esterna verso `Student.Sid`. Quali dei seguenti INSERT in `ENROLLED(Sid,Cid,Grade)` vanno a buon fine?

1. `INSERT INTO ENROLLED VALUES(53667, '15-420', 'C');`
2. `INSERT INTO ENROLLED VALUES(53666, '15-421', 'C');`
3. `INSERT INTO ENROLLED VALUES(53667, '15-415', 'C');`
4. `INSERT INTO ENROLLED VALUES(53666, '15-415', 'C');`

(solo 1 e 3 / solo 2 e 4 / solo 3 / solo 4)

Soluzione: solo 2 e 4 — entrambi usano `Sid=53666`, presente in `Student`; gli inserimenti 1 e 3 usano `Sid=53667`, che **non** esiste in `Student` — violano l'integrità referenziale.

Esercizio 8

Date le tabelle $S(A, B, C)$ (4 righe, tutte $\langle 1, a, e\rangle$), $T(D, E, F)$ (righe $\langle a, e, 1\rangle, \langle a, e, 2\rangle, \langle a, e, 3\rangle, \langle a, e, 4\rangle$) e $R(A, B, C)$ (righe con B variabile, incluso un `NULL`): senza altre assunzioni, si può dire che (B, C, A) è chiave esterna di S verso T? (potrebbe esserlo, ma non possiamo saperlo con certezza / no, non può esserlo / sì, di sicuro)

Soluzione: sì, di sicuro — ogni riga di S proiettata su (B, C, A) dà $\langle a, e, 1\rangle$, che compare come riga di T (posizionalmente su D, E, F): il vincolo di integrità referenziale è soddisfatto da **tutte** le righe mostrate di S.

13.3 SQL

Esercizio 9

Dati $R(A, B)$ e $S(B, C)$, quale delle seguenti è SQL legale? (`select A, AVG(C) as AV from R,S where R.B=S.B group by C` / `... group by A` / `... group by B`)

Soluzione: `... group by A` — regola generale (Argomento SQL): ogni colonna nella `select-list` che non sia un'aggregazione deve comparire anche nella `GROUP BY`. Qui l'unica colonna non aggregata nel `SELECT` è A: deve quindi essere lei l'oggetto del `GROUP BY` (raggruppare per B o per C lascerebbe A senza un valore univoco per gruppo).

13.4 Indici

Esercizio 10

Qual è lo scopo di un indice in un motore di basi di dati SQL? (solo velocizzare le ricerche / solo migliorare le performance delle query / tutte le ragioni menzionate / solo fornire un indice a un record)

Soluzione: tutte le ragioni menzionate.

13.5 Ottimizzazione delle Query

Esercizio 11

Il costo di $\sigma_{A=1}(R \bowtie S)$ è sempre uguale al costo di $\sigma_{A=1}(R) \bowtie S$? (No / Sì / vero solo se A è chiave esterna verso S / solo se A è chiave in R o S / vero solo se A è attributo di entrambe)

Soluzione: No — è un classico esempio di **selection pushdown**: anticipare la selezione prima del join (filtrando R per $A = 1$ quando possibile) riduce drasticamente il numero di tuple coinvolte nel join, abbassando il costo; le due espressioni sono equivalenti dal punto di vista del **risultato** (per questo l'ottimizzatore le considera intercambiabili), ma **non** del costo — è esattamente il tipo di riscrittura che un ottimizzatore di query applica automaticamente.

13.6 Algebra Relazionale

Esercizio 12

Due relazioni A e B con lo stesso schema, chiave C. In un diagramma di Venn con i due cerchi A e B che si sovrappongono, quale query restituisce solo la parte blu (la porzione di A che **non** si sovrappone a B)? ($B - A$ / $A - (A \cup B)$ / $A - (A \bowtie B)$ / nessuna è corretta / $A - (A \cap B)$)

Soluzione: $A - (A \bowtie B)$ — dato che A e B hanno lo **stesso schema** (inclusa la chiave C), il join naturale $A \bowtie B$ coincide con l'intersezione $A \cap B$ (fa match su tutti gli attributi in comune, cioè tutti); quindi $A - (A \bowtie B)$ equivale a $A - (A \cap B)$, le tuple di A che non

sono anche in B — esattamente la porzione di A non sovrapposta. (Nota: $A - (A \cap B)$, l'ultima opzione, sarebbe equivalente ma **non** è quella segnata come corretta nella griglia originale — probabilmente perché la domanda mette alla prova proprio l'equivalenza $A \bowtie B \equiv A \cap B$ su schemi identici.)

Esercizio 13

Dati $R(A, B)$ e $S(B)$, la divisione R/S in algebra relazionale equivale a: $(\pi_B(R) - \pi_B(S - (\pi_B(R) \times S))$ / $\pi_A(R) - \pi_A(R - (\pi_A(R) \times S))$ / $\pi_A(R) - \pi_A((\pi_A(R) \times S) - R)$ / $\pi_B(R) - \pi_A((\pi_B(R) \times S) - R))$

Soluzione: $\pi_A(R) - \pi_A((\pi_A(R) \times S) - R)$ — è la formula standard per esprimere la divisione tramite gli operatori base: si parte da tutti i possibili valori di A ($\pi_A(R)$), si sottraggono quelli per cui **esiste almeno un** valore di B in S non accoppiato in R (cioè quelli che **non** soddisfano la divisione), ottenendo solo i valori di A associati a **tutti** i valori di B in S.

13.7 Modello ER

Esercizio 14

Un'entità debole... (non partecipa a nessuna relazione / ha la chiave composta da attributi di un'altra entità / è collegata da una relazione ISA a un'altra entità / non ha una chiave)

Soluzione: ha la chiave composta da attributi di un'altra entità — coerente con quanto visto nel Mapping ER→relazionale: la chiave di un'entità debole combina la propria chiave parziale con la chiave dell'entità identificante.

13.8 Viste

Esercizio 15

Quale delle seguenti affermazioni sulle viste è **sbagliata**? Se sono tutte corrette, scegliere "sono tutte corrette".

A. Le viste possono rappresentare un sottoinsieme dei dati di una tabella, limitando l'esposizione delle tabelle di base (un utente può avere accesso alla vista ma non alla tabella).

B. Le viste possono unire (join) e semplificare più tabelle in un'unica tabella virtuale.

C. Le viste possono aggregare dati (SUM, AVG, ...) e presentare i risultati calcolati come parte dei dati.

D. Le viste possono nascondere la complessità dei dati (es. Vendite2000/Vendite2001 come partizioni trasparenti di una tabella).

E. Sono tutte corrette.

Soluzione: E — A, B, C, D descrivono tutte proprietà reali e comuni delle viste (controllo degli accessi, semplificazione via join, aggregazione, trasparenza); nessuna è falsa.

13.9 Vincoli di Integrità e Chiavi

Esercizio 16

Quale delle seguenti **non** è una proprietà di un DBMS relazionale? (ogni colonna ha un nome univoco / le relazioni possono essere vuote / tutti i valori di una colonna sono dello stesso tipo / gli attributi di chiave e chiave esterna devono avere lo stesso nome / l'ordine delle righe non conta / l'ordine delle colonne non conta)

Soluzione: gli attributi di chiave e chiave esterna devono avere lo stesso nome — **falso**: come visto nel Modello Relazionale, "gli attributi appartenenti alla FK non necessariamente hanno lo stesso nome dei corrispondenti attributi della PK".

Esercizio 17

Quale affermazione su chiavi/chiavi esterne **non** è vera? (la PK identifica univocamente le righe / la PK è unica e non può essere NULL / una tabella può avere una sola PK / la superchiave minimale è una chiave candidata / la chiave candidata è la chiave primaria)

Soluzione: "la chiave candidata è la chiave primaria" — falso: una relazione può avere **più** chiavi candidate (es. STUDENT aveva sia {sid} sia {login} — vedi Modello Relazionale), ma **una sola** viene scelta come primaria; le altre restano candidate ma non sono "la" primaria.

Esercizio 18 (ripetuta identica in entrambi gli appelli)

Date le tabelle $S(A, B, C)$ (4 righe $\langle 1, a, e\rangle$), $T(D, E, F)$ (righe $\langle a, e, 1..4\rangle$): (B, C, A) è chiave esterna di S verso T?

Soluzione: Sì, di sicuro — vedi Esercizio 8, identico.

13.10 SQL

Esercizio 19

Quale dei seguenti comandi **non** fa parte del DDL SQL? (CREATE / SELECT / DROP / ALTER)

Soluzione: SELECT — è DML (interrogazione), non DDL (definizione dello schema).

Esercizio 20

Qual è la differenza tra HAVING e WHERE?

Soluzione: WHERE si applica a ogni tupla dopo il prodotto cartesiano (il FROM); HAVING si applica a ogni **gruppo**, dopo l'aggregazione. Se non ci sono gruppi (niente GROUP BY), WHERE e HAVING si comportano allo stesso modo.

Esercizio 21

La normalizzazione è il processo di... (definire le tabelle / rendere le tuple della stessa dimensione / organizzare le informazioni rimuovendo le ridondanze / mantenere le tuple ordinate / definire le chiavi primarie)

Soluzione: organizzare le informazioni rimuovendo le ridondanze.

Esercizio 22

Dato `Product(maker,model)`, `PC(model,speed,hd,price)` (model chiave per entrambe): scrivere una query SQL che trovi tutti i `model` con `maker='Apple'`.

Soluzione:

```
SELECT model
FROM Product
WHERE maker = 'Apple';
```

Esercizio 23

`Suppliers(sid,sname,address)`, `Parts(pid,pname,color)`, `Catalog(sid,pid,cost)`.

```
SELECT C.sid
FROM Catalog C
WHERE NOT EXISTS (SELECT P.pid FROM Parts P
    WHERE NOT EXISTS (SELECT C1.sid FROM Catalog C1
        WHERE C1.sid=C.sid AND C1.pid=P.pid));
```

Descrivere in una frase (max 25 parole, senza termini tecnici) cosa fa la query.

Soluzione: è il classico pattern della **divisione** tramite doppia negazione ("per ogni... esiste"): trova i fornitori che riforniscono **ogni** parte esistente nel catalogo, senza eccezioni.

13.11 Dipendenze Funzionali e Normalizzazione (cont.)

Esercizio 24

$R(A, B, C, D, E, F)$ con $F = \{AB \to C,\ BC \to AD,\ D \to E,\ CF \to B\}$. Quanto vale $(DA)^+$? (DA / ABCDEF / ABCD / DAE)

Soluzione: DAE — $(DA)^+$ parte da $\{D, A\}$; l'unica DF applicabile è $D \to E$ (nessuna delle altre ha il lato sinistro contenuto in {D,A}); si aggiunge E: $\{D, A, E\}$, e l'algoritmo si ferma.

Esercizio 25

Uno schema $\langle R(X), F\rangle$ è in 3NF se, per ogni DF non banale $X \to Y$ in F^+: (X superchiave, oppure ogni attributo di Y-X è in una chiave candidata / X superchiave E ogni attributo di Y-X è in una chiave candidata / X superchiave, oppure ogni attributo di Y-X è in una **super**chiave / nessuna, perché la 3NF considera tutte le DF non solo quelle non banali)

Soluzione: X superchiave, **oppure** ogni attributo di Y-X è in una chiave candidata (attributo primo) — è l'"oppure", non l'"e" — esattamente come definito nella sezione Normalizzazione.

13.12 Indici e Costi

Esercizio 26 (risposta da verificare sull'originale)

Quale affermazione sugli indici **non** è corretta? (non si possono avere 2 indici sullo stesso attributo, ma sì più indici sulla stessa tabella / gli indici occupano spazio / un indice velocizza le ricerche al costo di scritture extra e più spazio / gli indici unclustered non definiscono l'ordine fisico ma quello logico / un indice clustered determina l'ordine fisico su disco, quindi ce n'è al massimo uno per tabella)

Nota mia: non sono riuscito a leggere con certezza quale opzione sia segnata. Il candidato più probabile è la quarta ("unclustered... ordine logico") — un indice unclustered non impone di per sé nessun ordinamento sui dati, logico o fisico, quindi l'affermazione suona sospetta — ma non ne sono certo: verifica sull'originale.

Esercizio 27 (identica a quella già in Argomento 10)

$R[A, B]$, $S[B, C]$, hash index su $S.B$ (lookup=2). Costo di `update R,S set R.A=10 where R.A=S.B`?

Soluzione: $P_R + P_R + |R| \cdot 2$ — identica all'Esercizio 3 di Argomento 10.

13.13 Ottimizzazione delle Query (cont.)

Esercizio 28 (identica all'Esercizio 11 sopra)

Il costo di $\sigma_{A=1}(R \bowtie S)$ è sempre uguale al costo di $\sigma_{A=1}(R) \bowtie S$?

Soluzione: No — vedi Esercizio 11 (selection pushdown).

13.14 Algebra Relazionale (cont.)

Esercizio 29

Due tabelle con un attributo omonimo. L'operazione che prende le tuple della prima che hanno match nella seconda, quelle della seconda che hanno match nella prima, **e anche** quelle senza alcun match da entrambe le parti, si chiama: (Semi-Join / prodotto cartesiano / Full Outer Join / Equi-join / Theta-join)

Soluzione: Full Outer Join — per definizione (Argomento SQL: "FULL (OUTER) JOIN: tutte le tuple di entrambe le tabelle a fronte di un match in almeno una").

Esercizio 30

Data $R(A, B, C)$: (1,NULL,2),(1,NULL,3),(NULL,NULL,NULL),(NULL,1,2),(3,4,5),(1,
Risultato di `select SUM(B),C from R group by C`?

Soluzione: $(1, 2), (NULL, 3), (2, NULL), (4, 5)$ — gruppo C=2: righe con B=NULL,1 $\Rightarrow$ SUM= 1; gruppo C=3: solo B=NULL $\Rightarrow$ SUM=NULL; gruppo C=NULL (le due righe con C nullo si raggruppano insieme, a differenza del WHERE): B=NULL,2 $\Rightarrow$ SUM= 2; gruppo C=5: B=4 $\Rightarrow$SUM= 4.

Esercizio 31

$R[\underline{A}, \underline{B}]$ e $S[\underline{B}, C]$ (B chiave in **entrambe**), B+tree su S.B (lookup=3). Piano: $\sigma_{B=10}(\pi_{BC}(R \bowtie_{\text{Index Nested Loops}} S))$. Qual è il numero **massimo** di tuple restituite? ($|R| \cdot |S|$ / 1 / $|R|$ / $|S|$ / $P_S + |S| \cdot (3+1)$)

Soluzione: 1 — dato che B è chiave (quindi unica) sia in R sia in S, il join produce al più una tupla per ciascun valore di B; la selezione finale $\sigma_{B=10}$ filtra ulteriormente su un **singolo** valore di quella stessa chiave, quindi può sopravvivere al massimo **una** tupla, indipendentemente da $|R|$ e $|S|$.

Esercizio 32

A, B stesso schema, chiave C. Quale query restituisce la parte **rossa** (solo A, non sovrapposta a B)? ($A - (A \bowtie B)$ / $B - A$ / $A - (A \cup B)$ / $B - (A \cap B)$ / nessuna corretta)

Soluzione: nessuna delle formule elencate è marcata come "nessuna corretta" in questo caso — la risposta segnata è $A - (A \cap B)$, formula non tra quelle di questa lista di opzioni ma equivalente a $A - (A \bowtie B)$ (dato lo stesso schema, join naturale = intersezione — vedi Esercizio 12 sopra).

Esercizio 33

$R[A, B]$, $S[B, C]$ entrambe non vuote. `SELECT DISTINCT A FROM R` e `SELECT DISTINCT A FROM R,S` restituiscono sempre lo stesso numero di tuple, indipendentemente dal contenuto? (Sì, sempre / No, dipende dal contenuto)

Soluzione: Sì, sempre — il prodotto cartesiano con S (non vuota) associa a ogni valore di A almeno una (e potenzialmente più) combinazione con S, ma **non introduce mai**

nuovi valori di A; il `DISTINCT` finale collassa tutte le combinazioni allo stesso insieme di valori distinti di A già presente in R da sola.

13.15 Note e Chiarimenti da Spiegazioni Tutor (2023)

Nota: cosa significa "chiusura transitiva" di un insieme di DF

La domanda "due insiemi di DF F_1, F_2 sono equivalenti se, applicando la chiusura transitiva a entrambi, si ottiene lo stesso insieme di DF" ammette **due letture**:

- in senso **stretto**: "chiusura transitiva" = applicare **solo** l'assioma di transitività;
- in senso **ampio**: applicare **tutti** gli assiomi di Armstrong fino a un punto fisso (cioè calcolare F^+ per intero).

Nel secondo senso, la frase diventa semplicemente la definizione di equivalenza vista in Argomento 7 ($F_1^+ = F_2^+$), ed è per questo che il tutor ha accettato **entrambe** le opzioni "dopo aver applicato la chiusura transitiva otteniamo lo stesso insieme di DF" e "la loro chiusura è la stessa" come corrette — sono la stessa cosa, letta nel senso ampio.

14 II Esercitazione sull'Algebra Relazionale

Esercitazione di **algebra relazionale (AR)** su uno schema *Impiegati–Dipartimenti–Progetti*:

- **Esercizio 1**: formulare in AR 14 interrogazioni, dalle più semplici (selezione e proiezione) fino a negazione e quantificazione universale;
- **Esercizio 2**: descrivere in linguaggio naturale il significato di 2 espressioni di AR.

Per ogni quesito: **testo**, **ragionamento**, **soluzione** e **fondamento teorico**.

14.1 Schema di riferimento e convenzioni

Schema relazionale (chiavi primarie sottolineate)

- **IMPIEGATO**(Matricola, Cognome, Stipendio, Dipartimento)
- **DIPARTIMENTO**(Codice, Nome, Sede, Direttore)
- **PROGETTO**(Sigla, Nome, Bilancio, Responsabile)
- **PARTECIPAZIONE**(Impiegato, Progetto)

Vincoli di riferimento (ogni chiave esterna punta alla chiave primaria della relazione indicata):

- IMPIEGATO.**Dipartimento** → DIPARTIMENTO: dove l'impiegato lavora;
- DIPARTIMENTO.**Direttore** → IMPIEGATO: chi dirige il dipartimento;

- PROGETTO.**Responsabile** → IMPIEGATO: chi è responsabile del progetto;
- PARTECIPAZIONE.**Impiegato** → IMPIEGATO: chi partecipa;
- PARTECIPAZIONE.**Progetto** → PROGETTO: a quale progetto partecipa (le due chiavi esterne realizzano l'associazione molti-a-molti tra impiegati e progetti).

Idea chiave: tra IMPIEGATO e DIPARTIMENTO esistono **due** collegamenti distinti, *lavora in* (Dipartimento → Codice) e *dirige* (Direttore → Matricola): scegliere quello giusto è metà della soluzione nelle interrogazioni 1.2–1.4, 1.7, 1.9–1.10 e nell'Es. 2.2.

Convenzioni usate nelle soluzioni

- **Ridenominazione**: $\rho_{N \leftarrow V}(E)$ cambia il nome dell'attributo V in N (notazione dell'esercitazione); equivale alla notazione delle slide $\rho(R(V \to N), E)$. Con più attributi la corrispondenza è posizionale: $\rho_{N_1,N_2 \leftarrow V_1,V_2}(E)$.
- **Join**: $R \bowtie_c S = \sigma_c(R \times S)$ richiede schemi disgiunti; $R \bowtie S$ è il **join naturale**, cioè uguaglianza su *tutti* gli attributi con lo stesso nome (senza attributi comuni degenera nel prodotto cartesiano). Dopo un equi-join non riuso mai l'attributo di destra della condizione: così il risultato non dipende dal fatto che quella colonna sia mantenuta o eliminata (nelle slide compare una sola volta).
- **Semantica a insiemi**: π e $\cup$ eliminano i duplicati; per distinguere impiegati omonimi bisogna proiettare anche Matricola.
- **Algebra di base**: solo σ, π, ρ, $\cup$, $\cap$, $-$, $\times$, $\bowtie$, senza funzioni aggregate (SUM, COUNT, MAX, ...), che le slide citano tra le «altre operazioni, che vedremo solo in parte»: massimi e conteggi si ottengono con self-join e differenza.
- **Costanti**: il testo usa unità diverse (milioni, K); le soglie sono scritte come numeri assoluti (50 000 000, 100 000, 60 000) e vanno adattate all'unità effettiva degli attributi.
- **Chiavi esterne non nulle**: ogni chiave esterna ha un valore, quindi un join dal lato della chiave esterna verso la chiave referenziata associa a ogni tupla *esattamente una* tupla.

Nota: l'esercitazione non riporta soluzioni ufficiali; quelle proposte sono svolgimenti non ufficiali, controllati eseguendoli su database di prova.

14.2 Richiami teorici sugli operatori

- **Selezione** $\sigma_c(R)$: le tuple di R che soddisfano c; lo schema non cambia e $|\sigma_c(R)| \leq |R|$. Selezioni in cascata = una selezione con la congiunzione delle condizioni.
- **Proiezione** $\pi_L(R)$: mantiene solo gli attributi in L ed **elimina i duplicati**; se L contiene una chiave di R allora $|\pi_L(R)| = |R|$.
- **Ridenominazione** ρ: cambia i nomi degli attributi, non le tuple; evita i conflitti di nomi e *crea* gli attributi comuni su cui agisce il join naturale.
- **Operatori insiemistici** $\cup, \cap, -$: richiedono relazioni **compatibili** (stesso numero di attributi, domini compatibili); $-$ non è commutativo; se i nomi differiscono il risultato usa quelli del primo operando (convenzione delle slide).

- **Prodotto cartesiano** $R \times S$: tutte le coppie di tuple, $|R \times S| = |R| \cdot |S|$, schema unione degli schemi (che devono essere disgiunti).
- **Join**: commutativo e associativo; $|R \bowtie_c S| \leq |R \times S|$.
- **Monotonia**: σ, π, $\times$, $\bowtie$, $\cup$, $\cap$ sono monotoni (aggiungere tuple in ingresso non fa sparire tuple dal risultato); solo la **differenza** è non monotona, ed è quindi l'unico operatore con cui si esprimono «non esiste», «nessuno», «tutti».
- **Ottimizzazione algebrica** (slide): anticipare le selezioni, anticipare le proiezioni, riordinare i join.

14.3 Esercizio 1: formulare le interrogazioni in algebra relazionale

14.3.1 Selezione, proiezione e join lungo le chiavi esterne (1.1–1.6)

Esercizio 1.1

Trovare matricola e cognome degli impiegati che guadagnano più di 50 milioni.

Ragionamento:

- Basta la relazione IMPIEGATO: contiene sia lo stipendio sia gli attributi richiesti.
- Prima si **selezionano** le tuple con Stipendio $> 50\,000\,000$, poi si **proiettano** Matricola e Cognome.
- L'ordine è obbligato: la proiezione elimina Stipendio, che serve alla selezione (σ prima di π).

Soluzione:

$$\pi_{\text{Matricola,Cognome}}\Big(\sigma_{\text{Stipendio}>50\,000\,000}(\text{IMPIEGATO})\Big)$$

Fondamento teorico:

- σ filtra le tuple senza cambiare lo schema; π restringe lo schema ed elimina i duplicati.
- La lista di proiezione contiene la chiave Matricola, quindi $|\pi_L(R)| = |R|$ (con R le tuple selezionate): nessuna tupla va persa. Con il solo Cognome due omonimi collasserebbero in una tupla.

Esercizio 1.2

Trovare cognome e stipendio degli impiegati che lavorano a Roma.

Ragionamento:

- La sede è un attributo di DIPARTIMENTO, non di IMPIEGATO: «lavorare a Roma» significa appartenere a un dipartimento con Sede = Roma.
- Il collegamento è la chiave esterna Dipartimento → Codice (*lavora in*): equi-join con condizione Dipartimento = Codice.

- Si seleziona la sede *prima* del join (meno tuple da combinare) e alla fine si proiettano Cognome e Stipendio.

Soluzione:

$$\pi_{\text{Cognome,Stipendio}}\Big(\text{IMPIEGATO} \bowtie_{\text{Dipartimento=Codice}} \sigma_{\text{Sede="Roma"}}(\text{DIPARTIMENTO})\Big)$$

Variante con join naturale (la ridenominazione crea l'attributo comune):

$$\pi_{\text{Cognome,Stipendio}}\Big(\text{IMPIEGATO} \bowtie \rho_{\text{Dipartimento}\leftarrow\text{Codice}}\big(\sigma_{\text{Sede="Roma"}}(\text{DIPARTIMENTO})\big)\Big)$$

Fondamento teorico:

- **Join lungo una chiave esterna**: ogni impiegato si combina con l'unico dipartimento il cui Codice coincide con il suo Dipartimento (chiave $\Rightarrow$ corrispondenza unica; integrità referenziale $\Rightarrow$ corrispondenza esistente).
- **Anticipazione della selezione** (ottimizzazione algebrica): la condizione usa solo attributi di DIPARTIMENTO, quindi $\sigma_c(R \bowtie S) = R \bowtie \sigma_c(S)$.
- Il join naturale richiede attributi omonimi: $\rho_{\text{Dipartimento}\leftarrow\text{Codice}}$ lo rende equivalente all'equi-join (la colonna di join compare una sola volta).

Esercizio 1.3

Trovare cognome degli impiegati e nome del dipartimento in cui lavorano.

Ragionamento:

- Il cognome è in IMPIEGATO, il nome del dipartimento in DIPARTIMENTO: serve il join sulla chiave esterna *lavora in*, Dipartimento = Codice.
- Nessuna selezione: si proiettano Cognome e Nome. Qui Nome è l'attributo di DIPARTIMENTO (IMPIEGATO non ha un attributo omonimo, quindi non c'è ambiguità).
- Il risultato è un insieme di coppie (cognome, nome del dipartimento): due impiegati omonimi dello stesso dipartimento danno una sola coppia.

Soluzione:

$$\pi_{\text{Cognome,Nome}}\Big(\text{IMPIEGATO} \bowtie_{\text{Dipartimento=Codice}} \text{DIPARTIMENTO}\Big)$$

Fondamento teorico:

- θ-join $R \bowtie_c S = \sigma_c(R \times S)$: i due schemi sono disgiunti, quindi non serve alcuna ridenominazione.
- Dal lato della chiave esterna il join non moltiplica le tuple (ogni impiegato ha esattamente un dipartimento):

$$|\text{IMPIEGATO} \bowtie_{\text{Dipartimento=Codice}} \text{DIPARTIMENTO}| = |\text{IMPIEGATO}|$$

Esercizio 1.4

Trovare cognome degli impiegati che sono direttori di dipartimento.

Ragionamento:

- «Essere direttore» è espresso da DIPARTIMENTO.Direttore (chiave esterna verso IMPIEGATO): è il collegamento *dirige*, non Dipartimento → Codice.
- Equi-join con Matricola = Direttore: restano gli impiegati che compaiono come direttore di almeno un dipartimento; poi π_{Cognome}.
- Chi dirige più dipartimenti compare più volte nel join ma una sola volta nel risultato (la proiezione elimina i duplicati).

Soluzione:

$$\pi_{\text{Cognome}}\big(\text{IMPIEGATO} \bowtie_{\text{Matricola}=\text{Direttore}} \text{DIPARTIMENTO}\big)$$

Variante con ridenominazione e join naturale (stessa struttura dell'Es. 2.1):

$$\pi_{\text{Cognome}}\big(\text{IMPIEGATO} \bowtie \rho_{\text{Matricola}\leftarrow\text{Direttore}}(\text{DIPARTIMENTO})\big)$$

Fondamento teorico:

- Due relazioni possono essere legate da **più chiavi esterne**: la condizione di join va scelta in base al *significato* richiesto. **Errore tipico:** usare Dipartimento = Codice, che restituisce gli impiegati che *lavorano in* un dipartimento (tutti), non quelli che lo dirigono.
- Le due formulazioni sono equivalenti: differiscono solo perché con ρ e join naturale la colonna di join compare una volta sola.

Esercizio 1.5

Trovare i nomi dei progetti e i cognomi dei responsabili.

Ragionamento:

- Il nome del progetto è in PROGETTO, il cognome in IMPIEGATO: si collegano con la chiave esterna Responsabile → Matricola.
- Equi-join Responsabile = Matricola e proiezione su Nome e Cognome.
- Nome è l'attributo di PROGETTO (IMPIEGATO non ha un attributo Nome): nessun conflitto.

Soluzione:

$$\pi_{\text{Nome,Cognome}}\big(\text{PROGETTO} \bowtie_{\text{Responsabile}=\text{Matricola}} \text{IMPIEGATO}\big)$$

Fondamento teorico:

- Join dal lato della chiave esterna: ogni progetto ha esattamente un responsabile, quindi prima della proiezione c'è una tupla per progetto.

- Nome non è chiave: due progetti con lo stesso nome e lo stesso responsabile collassano in una tupla. Per distinguerli basterebbe proiettare anche Sigla (la chiave).

Esercizio 1.6

Trovare i nomi dei progetti con bilancio maggiore di 100K e i cognomi degli impiegati che lavorano su di essi.

Ragionamento:

- Servono tre relazioni: PROGETTO (nome e bilancio), IMPIEGATO (cognome) e PARTECIPAZIONE, che lega impiegati e progetti.
- Percorso di join: PROGETTO, con Sigla = Progetto, PARTECIPAZIONE, con Impiegato = Matricola, IMPIEGATO.
- Si seleziona subito il bilancio su PROGETTO (Bilancio $> 100\,000$) e alla fine si proiettano Nome e Cognome.
- **Attenzione:** Responsabile non significa «lavora su»: l'attributo Responsabile dice chi è a capo del progetto, mentre gli impiegati che *lavorano* sul progetto sono quelli di PARTECIPAZIONE.

Soluzione:

$$\pi_{\text{Nome,Cognome}}\Big(\sigma_{\text{Bilancio}>100\,000}(\text{PROGETTO}) \bowtie_{\text{Sigla=Progetto}} \text{PARTECIPAZIONE} \bowtie_{\text{Impiegato=Matricola}} \text{IMPIEGATO}\Big)$$

Fondamento teorico:

- **Relazione ponte**: PARTECIPAZIONE ha come chiave la coppia (Impiegato, Progetto) e due chiavi esterne; per collegare IMPIEGATO e PROGETTO (associazione molti-a-molti) servono due join consecutivi.
- Il join è associativo e commutativo: l'ordine non cambia il risultato, ma cambia il costo (per questo si anticipa la selezione e si riordinano i join).
- Nome non è chiave: progetti omonimi con gli stessi partecipanti collassano in una sola coppia.

14.3.2 Confronto tra tuple e unione (1.7–1.8)

Esercizio 1.7

Trovare il cognome degli impiegati che guadagnano più del loro direttore di dipartimento.

Ragionamento:

- Per ogni impiegato servono *due* stipendi: il suo e quello del direttore del proprio dipartimento. IMPIEGATO deve quindi comparire due volte, con due ruoli (impiegato e direttore).

- **ImpDip**: ogni impiegato affiancato ai dati del proprio dipartimento, compreso Direttore (la matricola del direttore).
- **Dir**: copia ridotta di IMPIEGATO con solo matricola e stipendio, ridenominati MatDir e StipDir (evita il conflitto di nomi con la prima copia).
- Join Direttore = MatDir: ogni impiegato è affiancato allo stipendio del proprio direttore; poi $\sigma_{\text{Stipendio}>\text{StipDir}}$ e π_{Cognome}.
- Se l'impiegato è lui stesso il direttore, StipDir = Stipendio e la condizione $>$ è falsa: non compare.

Soluzione:

$$\text{Dir} = \rho_{\text{MatDir,StipDir}\leftarrow\text{Matricola,Stipendio}}\Big(\pi_{\text{Matricola,Stipendio}}(\text{IMPIEGATO})\Big)$$

$$\text{ImpDip} = \text{IMPIEGATO} \bowtie_{\text{Dipartimento=Codice}} \text{DIPARTIMENTO}$$

$$\pi_{\text{Cognome}}\Big(\sigma_{\text{Stipendio}>\text{StipDir}}\Big(\text{ImpDip} \bowtie_{\text{Direttore=MatDir}} \text{Dir}\Big)\Big)$$

Fondamento teorico:

- **Ridenominazione**: se la stessa relazione compare due volte servono nomi diversi, altrimenti prodotto cartesiano e θ-join non sono definiti (schemi non disgiunti).
- Confrontare valori di due tuple (stipendio $>$ stipendio del direttore) richiede una condizione di disuguaglianza in una selezione o in un θ-join, non un equi-join.
- Cammino di chiavi esterne IMPIEGATO $\rightarrow$ DIPARTIMENTO $\rightarrow$ IMPIEGATO (prima *lavora in*, poi *dirige*): due join con IMPIEGATO in due ruoli.

Esercizio 1.8

Trovare cognome dei direttori di dipartimento e dei responsabili di progetto.

Ragionamento:

- Due insiemi di impiegati: i direttori (come in 1.4) e i responsabili di progetto (come nell'Es. 2.1). La congiunzione «e» va letta come **unione**: i cognomi di chi ricopre almeno uno dei due ruoli. L'intersezione (chi li ricopre entrambi) sarebbe un'altra interrogazione.
- Si costruisce ciascun insieme come π_{Cognome}(join) e si fa l'unione: i due operandi hanno lo stesso schema (il solo attributo Cognome), quindi sono compatibili.

Soluzione:

$$\pi_{\text{Cognome}}\Big(\text{IMPIEGATO} \bowtie_{\text{Matricola=Direttore}} \text{DIPARTIMENTO}\Big)$$
$$\cup\ \pi_{\text{Cognome}}\Big(\text{IMPIEGATO} \bowtie_{\text{Matricola=Responsabile}} \text{PROGETTO}\Big)$$

Variante: unione delle matricole e un solo join (le matricole vanno ridenominate nello stesso attributo prima dell'unione):

$$\pi_{\text{Cognome}}\Big(\text{IMPIEGATO} \bowtie \Big(\rho_{\text{Matricola}\leftarrow\text{Direttore}}(\pi_{\text{Direttore}}(\text{DIPARTIMENTO}))$$
$$\cup\ \rho_{\text{Matricola}\leftarrow\text{Responsabile}}(\pi_{\text{Responsabile}}(\text{PROGETTO}))\Big)\Big)$$

Fondamento teorico:

- $\cup$ richiede relazioni **compatibili** (stesso numero di attributi, domini compatibili) ed elimina i duplicati: chi è sia direttore sia responsabile compare una volta sola.
- π e join si distribuiscono sull'unione: $\pi_L(A \bowtie X) \cup \pi_L(A \bowtie Y) = \pi_L\Big(A \bowtie (X \cup Y)\Big)$; per questo la variante con un solo join è equivalente.
- **Errore tipico:** scrivere $\cap$ al posto di $\cup$: si otterrebbero solo i cognomi di chi è contemporaneamente direttore e responsabile.

14.3.3 Quantificazioni e negazione (1.9–1.14)

L'AR di base non ha COUNT, MAX né «non esiste»: si usano **differenza** e **self-join** (tabella dei pattern a fine sezione).

Esercizio 1.9

Trovare nomi dei dipartimenti in cui lavorano impiegati che guadagnano più di 60K.

Ragionamento:

- Condizione **esistenziale**: basta che nel dipartimento esista un solo impiegato sopra soglia (Stipendio $> 60\,000$); si filtrano gli impiegati e si fa il join.
- Selezione su IMPIEGATO, join con DIPARTIMENTO su Dipartimento = Codice, proiezione su Nome.
- Un dipartimento con più impiegati ben pagati compare più volte nel join ma una sola volta nel risultato.

Soluzione:

$$\pi_{\text{Nome}}\Big(\sigma_{\text{Stipendio}>60\,000}(\text{IMPIEGATO}) \bowtie_{\text{Dipartimento}=\text{Codice}} \text{DIPARTIMENTO}\Big)$$

Fondamento teorico:

- Quantificatore **esistenziale** = join + proiezione: la proiezione «dimentica» quale impiegato soddisfa la condizione ed elimina i duplicati.
- Da confrontare con 1.10: stessa relazione e stessa soglia, ma con il quantificatore universale serve la differenza.

Esercizio 1.10

Trovare nomi dei dipartimenti in cui tutti gli impiegati guadagnano più di 60K.

Ragionamento:

- «Tutti guadagnano più di 60K» equivale a «**non esiste** un impiegato del dipartimento con stipendio $\leq 60\,000$»: il quantificatore universale è la negazione di un'esistenziale sul controesempio.
- **Bassi**: gli impiegati con Stipendio $\leq 60\,000$ (i controesempi).

- **DipBassi**: i codici dei dipartimenti con almeno un controesempio, ridenominati in Codice per poter fare la differenza.
- **DipOk** = tutti i codici − DipBassi; infine si recupera il Nome con un join naturale su Codice.
- Si sottrae sul **Codice** (chiave), non sul Nome: due dipartimenti possono avere lo stesso nome, e sottrarre i nomi eliminerebbe per errore anche un dipartimento «buono» omonimo di uno «cattivo».
- I dipartimenti *senza impiegati* soddisfano «tutti» in modo vacuo e compaiono nel risultato; la variante qui sotto li esclude.

Soluzione:

$$\begin{aligned} \text{Bassi} &= \sigma_{\text{Stipendio} \leq 60\,000}(\text{IMPIEGATO}) \\ \text{DipBassi} &= \rho_{\text{Codice} \leftarrow \text{Dipartimento}}\Big(\pi_{\text{Dipartimento}}(\text{Bassi})\Big) \\ \text{DipOk} &= \pi_{\text{Codice}}(\text{DIPARTIMENTO}) - \text{DipBassi} \end{aligned}$$

$$\pi_{\text{Nome}}(\text{DIPARTIMENTO} \bowtie \text{DipOk})$$

Variante: solo dipartimenti con almeno un impiegato.

$$\text{DipConImp} = \rho_{\text{Codice} \leftarrow \text{Dipartimento}}\Big(\pi_{\text{Dipartimento}}(\text{IMPIEGATO})\Big)$$

$$\pi_{\text{Nome}}\Big(\text{DIPARTIMENTO} \bowtie (\text{DipConImp} - \text{DipBassi})\Big)$$

Fondamento teorico:

- **Quantificatore universale**: $\forall x\, P(x) \equiv \neg\exists x\, \neg P(x)$. In AR: insieme universo − insieme dei controesempi (differenza insiemistica).
- $R - S$ richiede relazioni compatibili: per questo Dipartimento viene ridenominato in Codice (entrambi gli operandi hanno un solo attributo, Codice).
- **Non è una divisione**: $R \div S$ esprime «per tutti gli elementi di un insieme S *dato*» (es. i marinai che hanno prenotato tutte le barche); qui «tutti» riguarda un predicato (stipendio $> 60\,000$) sulle tuple di ciascun dipartimento.
- **Errori tipici:** usare la soluzione di 1.9 (esistenziale al posto di universale); sottrarre i nomi invece dei codici.

Esercizio 1.11

Trovare cognome degli impiegati di stipendio massimo.

Ragionamento:

- L'AR di base non ha MAX: il massimo si ottiene *per esclusione*, come l'unico stipendio per cui non esiste uno stipendio più alto.
- **Stip**: l'insieme degli stipendi presenti.

- **NonMax**: gli stipendi per cui ne esiste uno più grande. Si fa il prodotto cartesiano di Stip con una sua copia ridenominata (tutte le coppie di stipendi), si seleziona Stipendio $<$ Stipendio2 e si proietta su Stipendio.
- **Max** $=$ Stip $-$ NonMax: un solo valore (se IMPIEGATO non è vuota).
- Join naturale con IMPIEGATO sullo stipendio: recupera *tutti* gli impiegati con lo stipendio massimo, anche a pari merito; infine π_{Cognome}.

Soluzione:

$$\begin{aligned}
\text{Stip} &= \pi_{\text{Stipendio}}(\text{IMPIEGATO}) \\
\text{NonMax} &= \pi_{\text{Stipendio}}\Big(\sigma_{\text{Stipendio}<\text{Stipendio2}} \\
&\qquad \big(\text{Stip} \times \rho_{\text{Stipendio2}\leftarrow\text{Stipendio}}(\text{Stip})\big)\Big) \\
\text{Max} &= \text{Stip} - \text{NonMax}
\end{aligned}$$

$$\pi_{\text{Cognome}}(\text{IMPIEGATO} \bowtie \text{Max})$$

Variante diretta, con la differenza sugli impiegati (matricola e cognome) e non sui soli cognomi:

$$\begin{aligned}
\text{Dominati} = \pi_{\text{Matricola,Cognome}}\Big(&\sigma_{\text{Stipendio}<\text{Stipendio2}} \\
&\big(\text{IMPIEGATO} \times \rho_{\text{Stipendio2}\leftarrow\text{Stipendio}}(\text{Stip})\big)\Big)
\end{aligned}$$

$$\pi_{\text{Cognome}}\Big(\pi_{\text{Matricola,Cognome}}(\text{IMPIEGATO}) - \text{Dominati}\Big)$$

Fondamento teorico:

- **Massimo come complemento dei dominati**: $\max X = X - \{x \in X \mid \exists y \in X,\ y > x\}$.
- Prodotto cartesiano $+$ ρ permettono di confrontare tutte le coppie di valori della stessa relazione (schemi disgiunti grazie alla ridenominazione).
- **Errore tipico:** sottrarre i cognomi. Se un omonimo del massimo non è massimo, il cognome viene scartato anche per il massimo: si sottrae su uno stipendio o sulla chiave (Matricola) e il cognome si recupera alla fine.

Esercizio 1.12

Trovare matricola e cognome degli impiegati che non lavorano a nessun progetto.

Ragionamento:

- «Nessun progetto» è la negazione di «almeno un progetto»: tutti gli impiegati $-$ quelli che compaiono in PARTECIPAZIONE.
- **Lavoranti**: matricola e cognome degli impiegati con almeno una tupla in PARTECIPAZIONE (join Matricola $=$ Impiegato).

- Risultato: matricola e cognome di tutti gli impiegati − Lavoranti. I due operandi hanno lo stesso schema (Matricola, Cognome) e, includendo la chiave, gli omonimi non danno problemi.

Soluzione:

$$\text{Lavoranti} = \pi_{\text{Matricola,Cognome}}\Big(\text{IMPIEGATO} \bowtie_{\text{Matricola=Impiegato}} \text{PARTECIPAZIONE}\Big)$$

$$\pi_{\text{Matricola,Cognome}}(\text{IMPIEGATO}) - \text{Lavoranti}$$

Fondamento teorico:

- **Negazione = differenza**: σ, π, $\times$, $\bowtie$, $\cup$ sono monotoni, ma «non lavora a nessun progetto» no (aggiungere una partecipazione fa sparire l'impiegato dal risultato), quindi serve $-$.
- Si sottrae all'*universo* (tutti gli impiegati) l'insieme di chi soddisfa la condizione positiva.
- **Errore tipico:** esprimere la negazione con un join con $\neq$ (Matricola $\neq$ Impiegato): restituisce quasi tutti gli impiegati, perché basta che in PARTECIPAZIONE ci sia una tupla di un altro impiegato.

Esercizio 1.13

Trovare matricola e cognome degli impiegati che lavorano a più di un progetto.

Ragionamento:

- Senza COUNT: «più di un progetto» significa che esistono *due* tuple di PARTECIPAZIONE con lo stesso impiegato e progetti diversi. Serve un confronto tra due copie della stessa relazione (self-join).
- **PA2**: copia ridenominata di PARTECIPAZIONE, con schema (Impiegato2, Progetto2); **Coppie**: tutte le coppie di partecipazioni.
- **Multi**: matricole di chi compare in due tuple con lo stesso impiegato e progetti diversi (condizione Impiegato = Impiegato2 $\wedge$ Progetto $\neq$ Progetto2).
- Join con IMPIEGATO su Matricola = Impiegato per recuperare il cognome.
- **Attenzione:** la condizione Progetto $\neq$ Progetto2 è essenziale: senza di essa ogni tupla si accoppia con se stessa e si ottengono tutti gli impiegati con almeno un progetto.

Soluzione:

$$\begin{aligned}
\text{PA2} &= \rho_{\text{Impiegato2,Progetto2}\leftarrow\text{Impiegato,Progetto}}(\text{PARTECIPAZIONE}) \\
\text{Coppie} &= \text{PARTECIPAZIONE} \times \text{PA2} \\
\text{Multi} &= \pi_{\text{Impiegato}}\Big(\sigma_{\text{Impiegato=Impiegato2} \wedge \text{Progetto}\neq\text{Progetto2}}(\text{Coppie})\Big)
\end{aligned}$$

$$\pi_{\text{Matricola,Cognome}}\Big(\text{IMPIEGATO} \bowtie_{\text{Matricola=Impiegato}} \text{Multi}\Big)$$

Fondamento teorico:

- **Self-join**: ridenominazione + prodotto cartesiano + selezione (equivalente a un θ-join con la stessa condizione, perché $R \bowtie_c S = \sigma_c(R \times S)$).
- L'AR di base non sa contare: «almeno k» si esprime con k copie della relazione e disuguaglianze (qui $k = 2$).
- Il risultato è un insieme: chi lavora a tre progetti compare in più coppie ma una sola volta in Multi.

Esercizio 1.14

Trovare matricola e cognome degli impiegati che lavorano a un solo progetto.

Ragionamento:

- «Esattamente uno» = «almeno uno» e non «almeno due»: **AlmenoUno** − **Multi** (con Multi definita in 1.13).
- **AlmenoUno**: le matricole che compaiono in PARTECIPAZIONE; **UnSolo** = AlmenoUno − Multi (entrambe hanno l'unico attributo Impiegato).
- Join con IMPIEGATO per recuperare matricola e cognome.
- Errore tipico: calcolare IMPIEGATO − (risultato di 1.13): include anche chi non lavora a nessun progetto, che non ne ha *uno*, ma zero.

Soluzione:

$$\text{AlmenoUno} = \pi_{\text{Impiegato}}(\text{PARTECIPAZIONE})$$
$$\text{UnSolo} = \text{AlmenoUno} - \text{Multi}$$

$$\pi_{\text{Matricola,Cognome}}\Big(\text{IMPIEGATO} \bowtie_{\text{Matricola=Impiegato}} \text{UnSolo}\Big)$$

Variante: sottrarre i risultati finali (Lavoranti è definita in 1.12).

$$\text{Lavoranti} - \pi_{\text{Matricola,Cognome}}\Big(\text{IMPIEGATO} \bowtie_{\text{Matricola=Impiegato}} \text{Multi}\Big)$$

Fondamento teorico:

- **Composizione di tecniche**: «almeno uno» (proiezione) − «almeno due» (self-join con $\neq$).
- In generale «esattamente k» = «almeno k» − «almeno $k+1$».

14.4 Esercizio 2: descrivere in linguaggio naturale

Metodo: si legge l'espressione dall'*interno verso l'esterno*, scrivendo a ogni passo lo schema del risultato; gli attributi con lo stesso nome sono le condizioni di join dei join naturali.

Esercizio 2.1

Con riferimento allo schema dell'esercizio precedente, descrivere in linguaggio naturale il significato della seguente interrogazione espressa in algebra relazionale.

$$\pi_{\text{Cognome}}\Big(\rho_{\text{Matricola}\leftarrow\text{Responsabile}}(\text{PROGETTO}) \bowtie \text{IMPIEGATO}\Big)$$

Ragionamento:

- $\rho_{\text{Matricola}\leftarrow\text{Responsabile}}(\text{PROGETTO})$ ha schema (Sigla, Nome, Bilancio, Matricola), dove Matricola è la matricola del responsabile del progetto.
- Join naturale con IMPIEGATO (Matricola, Cognome, Stipendio, Dipartimento): l'unico attributo comune è Matricola, quindi ogni progetto è affiancato ai dati del suo responsabile.
- π_{Cognome}: si tengono solo i cognomi.

Soluzione: i cognomi degli impiegati che sono responsabili di (almeno) un progetto. Un impiegato responsabile di più progetti compare una volta sola.

Formulazione equivalente con equi-join (stessa struttura di 1.4, con PROGETTO al posto di DIPARTIMENTO): $\pi_{\text{Cognome}}(\text{IMPIEGATO} \bowtie_{\text{Matricola=Responsabile}} \text{PROGETTO})$.

Fondamento teorico:

- Il **join naturale** è un equi-join su tutti gli attributi con lo stesso nome: ρ serve a creare (o a evitare) attributi comuni, e $\rho+ \bowtie$ equivale a un equi-join.
- Il risultato è un insieme: nessun cognome compare due volte, anche se l'impiegato ha più progetti.

Esercizio 2.2

Con riferimento allo schema dell'esercizio precedente, descrivere in linguaggio naturale il significato della seguente interrogazione espressa in algebra relazionale.

$$\pi_{\text{Cognome}}\Big(\rho_{\text{Matricola}\leftarrow\text{Direttore}}(\text{DIPARTIMENTO}) \bowtie \big(\text{IMPIEGATO} \bowtie \rho_{\text{Dipartimento}\leftarrow\text{Codice}}(\text{DIPARTIMENTO})\big)\Big)$$

Ragionamento (schemi dei risultati intermedi):

Sottoespressione	**Schema del risultato**
$A = \rho_{\text{Matricola}\leftarrow\text{Direttore}}(\text{DIPARTIMENTO})$	Codice, Nome, Sede, Matricola
$B = \rho_{\text{Dipartimento}\leftarrow\text{Codice}}(\text{DIPARTIMENTO})$	Dipartimento, Nome, Sede, Direttore
$C = \text{IMPIEGATO} \bowtie B$, comune: **Dipartimento**	Matricola, Cognome, Stipendio, Dipartimento, Nome, Sede, Direttore
$A \bowtie C$, comuni: **Matricola, Nome, Sede**	Codice, Nome, Sede, Matricola, Cognome, Stipendio, Dipartimento, Direttore

- In A la Matricola è quella del *direttore* del dipartimento; in C è quella dell'*impiegato*, affiancato ai dati del dipartimento in cui lavora.
- Il join $A \bowtie C$ impone l'uguaglianza su **tre** attributi comuni, non solo su Matricola:
  - Matricola = Matricola: l'impiegato è il direttore del dipartimento di A;
  - Nome = Nome e Sede = Sede: il dipartimento in cui l'impiegato lavora ha lo stesso nome e la stessa sede del dipartimento di A.
- Codice (in A) e Dipartimento (in C) hanno nomi diversi e **non** vengono confrontati.

Soluzione: i cognomi dei direttori di dipartimento che lavorano in un dipartimento con lo stesso nome e la stessa sede del dipartimento che dirigono. Se la coppia (Nome, Sede) identifica un dipartimento (assunzione plausibile ma *non* garantita dallo schema, dove l'unica chiave è Codice), significa: **i cognomi dei direttori che lavorano nel dipartimento che dirigono**.

Formulazione equivalente, che usa la chiave invece di Nome e Sede (equivalente se (Nome, Sede) è univoco):

$$\pi_{\text{Cognome}}\Big(\text{IMPIEGATO} \bowtie_{\text{Matricola=Direttore} \wedge \text{ Dipartimento=Codice}} \text{DIPARTIMENTO}\Big)$$

Fondamento teorico:

- **Il join naturale uguaglia tutti gli attributi con lo stesso nome**, anche quelli non voluti: Nome e Sede compaiono sia in A sia in C perché provengono entrambi da DIPARTIMENTO, e quindi vengono confrontati.
- Le ridenominazioni cambiano le condizioni di join implicite: Codice e Dipartimento restano diversi, quindi nulla impone che il dipartimento in cui l'impiegato lavora sia *proprio* quello che dirige, se non l'uguaglianza di Nome e Sede.
- Per imporre l'identità del dipartimento si usa un θ-join sulla chiave (formulazione sopra): è più robusta di un join naturale che dipende dai nomi degli attributi.

14.5 Riepilogo dei pattern ricorrenti

Tipo di richiesta	**Schema risolutivo in AR**	**Quesiti**
Filtro su una relazione	σ poi π	1.1
Dati in relazioni legate da chiavi esterne	join lungo la chiave esterna con selezione anticipata; due join se c'è una relazione ponte	1.2–1.6, 1.9
Confronto tra tuple, anche della stessa relazione	ρ + $\times$ (o θ-join) + σ con $<, >, \neq$	1.7, 1.11, 1.13
«A oppure B»	$\cup$ di relazioni compatibili	1.8
«nessuno», «non esiste»	universo $-$ chi soddisfa la condizione	1.12
«tutti» ($\forall$)	universo $-$ chi ha un controesempio	1.10
massimo, minimo	valori $-$ valori «dominati» da uno migliore	1.11
«più di uno»	self-join: stesso impiegato, progetti diversi ($\neq$)	1.13
«uno solo»	«almeno uno» $-$ «più di uno»	1.14

Controlli prima di consegnare una soluzione:

- **Schemi**: per $\cup$, $\cap$, $-$ gli operandi hanno gli stessi attributi; per $\times$ e θ-join gli attributi sono disgiunti; nei join naturali controllare *tutti* gli attributi omonimi (Es. 2.2).
- **Chiave esterna giusta**: *lavora in* (Dipartimento $\to$ Codice) $\neq$ *dirige* (Direttore $\to$ Matricola); *lavora su* (PARTECIPAZIONE) $\neq$ *è responsabile di* (Responsabile).
- **Differenze su chiavi**: sottrarre insiemi di chiavi (Codice, Matricola) o di valori che identificano la condizione, e recuperare gli attributi descrittivi (Nome, Cognome) con un join finale.

- **Duplicati e omonimi**: π elimina i duplicati; includere la chiave se serve distinguerli.
- **Casi limite**: dipartimenti senza impiegati (1.10), impiegati senza progetti (1.12, 1.14), stipendi a pari merito (1.11).

15 Esercizi SQL: GROUP BY e HAVING su Marinai, Barche e Prenotazioni

Esercitazione di **SQL** sul database *Sailors* (marinai, barche, prenotazioni), cioè lo schema usato dal corso nelle slide di SQL e nel libro di testo. Consegna: *progettare le query SQL che rispondono a queste domande*. I 5 quesiti combinano tre ingredienti:

- **raggruppamento** (`group by`) e **funzioni di aggregazione** (`sum`, `avg`, `count`, `max`, `min`);
- **filtri sulle righe** (`where`) e **filtri sui gruppi** (`having`);
- **join** lungo le chiavi esterne (quesiti 1, 2 e 5).

Per ogni quesito: **testo**, **ragionamento**, **soluzione** e **fondamento teorico**.

15.1 Schema di riferimento, istanza e convenzioni

Schema relazionale (chiavi primarie sottolineate)

- **Sailors**(sid, sname, rating, age): i marinai (`sid` e `rating` interi, `age` reale);
- **Boats**(bid, bname, color): le barche;
- **Reserves**(sid, bid, day): le prenotazioni, una riga per ogni marinaio che prenota una barca in un giorno (`day` è una data).

Vincoli di riferimento:

- Reserves.**sid** $\rightarrow$ Sailors: il marinaio che prenota;
- Reserves.**bid** $\rightarrow$ Boats: la barca prenotata. Reserves è la **relazione ponte** tra marinai e barche: le sue due chiavi esterne realizzano l'associazione molti-a-molti.

Nota: il testo dei quesiti non riporta lo schema; si assume quello del corso, cioè di Argomento04 (p. 8, istanza) e del libro di testo (cap. 5, «About the Examples»), i cui nomi (`bid`, `bname`, `day`, `rating`) coincidono con quelli del testo. La chiave composta di Reserves è quella sottolineata nel libro.

Istanza usata nelle slide (Argomento04, p. 8; Figure 5.1–5.3 del libro; date come scritte nella Figura 5.2): serve per controllare a mano i risultati.

sid	sname	rating	age
22	Dustin	7	45.0
29	Brutus	1	33.0
31	Lubber	8	55.5
32	Andy	8	25.5
58	Rusty	10	35.0
64	Horatio	7	35.0
71	Zorba	10	16.0
74	Horatio	9	35.0
85	Art	3	25.5
95	Bob	3	63.5

sid	bid	day
22	101	10/10/98
22	102	10/10/98
22	103	10/8/98
22	104	10/7/98
31	102	11/10/98
31	103	11/6/98
31	104	11/12/98
64	101	9/5/98
64	102	9/8/98
74	103	9/8/98

bid	bname	color
101	Interlake	blue
102	Interlake	red
103	Clipper	green
104	Marine	red

Convenzioni usate nelle soluzioni

- **DBMS**: MariaDB/MySQL, come nelle slide; parole chiave in minuscolo e alias di tabella (`s`, `r`, `b`) come negli esempi del corso.
- **Join**: `join ... on` per le condizioni di join (forma raccomandata nelle slide) e `where` per i filtri; la forma con relation-list e condizioni di join nel `where` è equivalente (variante del quesito 1).
- **Select-list**: solo i campi richiesti dal testo; le colonne non aggregate compaiono sempre nel `group by`. Se la regola è violata, MariaDB (senza `ONLY_FULL_GROUP_BY`) esegue comunque la query con un valore arbitrario per quella colonna: è l'«Errore!!» mostrato nelle slide.
- **having**: l'aggregato è ripetuto nella condizione. Le slide usano anche l'alias della select-list nel `having`: MySQL/MariaDB lo accettano, ma non è SQL standard.
- **Soglie**: «superiore a 7» = `> 7`; «almeno 2» = `>= 2`; «più di uno» = `> 1`.
- **Conteggi**: Reserves ha chiave (sid, bid, day), quindi ogni riga è una prenotazione distinta; dopo un join le aggregazioni lavorano su una riga per prenotazione.

Nota: l'esercitazione non riporta soluzioni ufficiali; quelle proposte sono svolgimenti non ufficiali, controllati eseguendoli su database di prova.

15.2 Richiami teorici: aggregazione, GROUP BY e HAVING

Struttura estesa di una query (Argomento04):

```
select [distinct] select-list
from   from-list
where  qualification
group by grouping-list
having group-qualification
```

- **Funzioni di aggregazione**: `count(*)` (numero di righe), `count`, `sum` e `avg` (anche con `distinct`: i valori uguali contano una volta sola), `max`, `min`. Riassumono un insieme di righe in un solo valore.

- **group by**: partiziona le righe in gruppi con gli stessi valori degli attributi di raggruppamento e produce *una riga di risultato per gruppo*. Senza `group by` l'intera tabella è un unico gruppo.
- **Regola della select-list**: ogni colonna non aggregata della select-list deve comparire anche nel `group by` (altrimenti nel gruppo avrebbe valori diversi); le espressioni del `having` devono avere un solo valore per gruppo.
- **where e having**: il `where` si applica riga per riga *prima* del raggruppamento e non può contenere funzioni di aggregazione; il `having` si applica ai gruppi *dopo* l'aggregazione. Una condizione su un attributo di raggruppamento può stare in entrambi, ma nel `where` scarta le righe prima di raggruppare.
- **Ordine logico di valutazione** (slide): (1) join o prodotto cartesiano delle tabelle del `from`; (2) `where`; (3) eliminazione delle colonne non necessarie; (4) `group by`; (5) `having`; (6) generazione delle righe di risposta (select-list).
- **Duplicati**: SQL non li elimina per default. Un join produce una riga per ogni combinazione di tuple corrispondenti, quindi le aggregazioni contano *righe*, non entità (marinai, barche).
- **Linee guida delle slide**: elencare gli attributi richiesti (R2); riconoscere le parole che suggeriscono un'aggregazione, ad esempio «media» → `avg`, «totale» → `sum`, «numero» → `count`, «per ogni» → `group by`, «solo i ... con ...» → `having` (R3); il `where` è una funzione che si applica tupla per tupla (R4); verificare la coerenza dei risultati e l'uso di `distinct` (R6).

15.3 Quesiti

Esercizio SQL 1

Per ogni colore di barca, trova la somma totale dei rating dei marinai che l'hanno prenotata, ma considera solo i colori con un rating medio superiore a 7.

Ragionamento:

- **Raggruppamento**: «per ogni colore» → `group by b.color`; il risultato ha una riga per colore.
- **Tabelle**: il colore è in Boats, il rating in Sailors; Reserves collega marinai e barche (relazione ponte). Servono due join lungo le chiavi esterne: `r.sid = s.sid` e `r.bid = b.bid`.
- **Select-list**: il colore (attributo di raggruppamento) e `sum(s.rating)`, la «somma totale dei rating».
- **Filtro sui gruppi**: «solo i colori con rating medio superiore a 7» è una condizione su un aggregato, quindi va nel `having`: `avg(s.rating) > 7`, con disuguaglianza stretta. L'aggregato del `having` (`avg`) è diverso da quello della select-list (`sum`): è lecito.

- **Cosa è una riga**: dopo il join c'è una riga per *prenotazione*; un marinaio che ha prenotato due barche dello stesso colore contribuisce due volte alla somma e alla media. Il testo non chiarisce se ogni marinaio debba contare una sola volta per colore: la soluzione principale usa una riga per prenotazione (lettura più diretta), l'ultima variante conta ogni marinaio una volta per colore.

Soluzione:

```
select b.color, sum(s.rating) as SommaRating
from sailors s
     join reserves r on r.sid = s.sid
     join boats b on b.bid = r.bid
group by b.color
having avg(s.rating) > 7;
```

Variante con relation-list e condizioni di join nel `where` (equivalente):

```
select b.color, sum(s.rating) as SommaRating
from sailors s, reserves r, boats b
where s.sid = r.sid and r.bid = b.bid
group by b.color
having avg(s.rating) > 7;
```

Variante: ogni marinaio conta una sola volta per colore (sottoquery con `distinct`):

```
select t.color, sum(t.rating) as SommaRating
from (select distinct b.color, s.sid, s.rating
      from sailors s
           join reserves r on r.sid = s.sid
           join boats b on b.bid = r.bid) t
group by t.color
having avg(t.rating) > 7;
```

Controllo sull'istanza delle slide: la soluzione principale restituisce $(\text{green}, 24)$ e $(\text{red}, 37)$. Il colore blue è escluso: le sue prenotazioni (marinai 22 e 64, entrambi con rating 7) hanno media esattamente 7, non > 7. La variante con `distinct` restituisce $(\text{green}, 24)$ e $(\text{red}, 22)$: i marinai 22 e 31 hanno prenotato due barche rosse ciascuno e contano una volta sola.

Fondamento teorico:

- **Ordine logico di valutazione**: join del `from`, poi `where` (qui assente), poi `group by`, poi `having`, infine la select-list. `sum` e `avg` sono calcolati sulle righe di ciascun gruppo.
- Le condizioni sui gruppi possono usare aggregati che *non* compaiono nella select-list: devono solo avere un valore per gruppo.
- **Righe, non entità**: un join produce una riga per combinazione di tuple e le funzioni di aggregazione lavorano su queste righe. Per contare ogni marinaio una volta occorre eliminare i duplicati *prima* di aggregare (sottoquery con `distinct` su colore e marinaio).

- **Errore tipico:** `sum(distinct s.rating)` elimina i *valori* uguali del rating, non i marinai ripetuti. Sull'istanza delle slide darebbe $(\text{red}, 15)$ invece di $(\text{red}, 37)$: i marinai 22 e 64 hanno entrambi rating 7, che verrebbe sommato una volta sola.
- **Errore tipico:** `where avg(s.rating) > 7` è un errore: le funzioni di aggregazione non sono ammesse nel `where`, che si applica riga per riga prima del raggruppamento.
- La soglia è stretta: con `>= 7` comparirebbe anche blue (media 7).

Esercizio SQL 2

Trova i giorni (day) in cui sono state effettuate almeno 2 prenotazioni complessive, considerando solo marinai con almeno 18 anni.

Ragionamento:

- **Raggruppamento**: il risultato è un insieme di giorni, quindi `group by r.day`: una riga per giorno.
- **Tabelle**: il giorno è in Reserves, l'età in Sailors: serve il join `r.sid = s.sid`.
- **Filtro sulle righe**: «considerando solo marinai con almeno 18 anni» seleziona le prenotazioni dei marinai adulti ed è applicato *prima* di contare: va nel `where` (`s.age >= 18`), non nel `having`.
- **Filtro sui gruppi**: «almeno 2 prenotazioni complessive» $\rightarrow$ `having count(*) >= 2`. `count(*)` conta le righe del gruppo, cioè le prenotazioni di quel giorno fatte da marinai adulti; «complessive» significa tutte insieme, senza distinguere marinaio o barca.
- **Select-list**: solo `r.day`, il campo richiesto; il conteggio serve solo nel `having`.

Soluzione:

```
select r.day
from reserves r
     join sailors s on s.sid = r.sid
where s.age >= 18
group by r.day
having count(*) >= 2;
```

Variante senza join (sottoquery con `in`):

```
select r.day
from reserves r
where r.sid in (select s.sid from sailors s where s.age >= 18)
group by r.day
having count(*) >= 2;
```

Controllo sull'istanza delle slide: i giorni 10/10/98 e 9/8/98. Il 10/10/98 il marinaio 22 ha prenotato due barche (101 e 102); il 9/8/98 hanno prenotato 64 e 74. Gli altri giorni hanno una sola prenotazione.

Fondamento teorico:

- **where e having**: il `where` agisce sulle singole righe prima del raggruppamento, il `having` sui gruppi dopo l'aggregazione. Il filtro sull'età riguarda la singola prenotazione, quindi va nel `where`; il conteggio riguarda il gruppo, quindi va nel `having`.
- **Errore tipico:** `having count(*) >= 2 and s.age >= 18`. L'età non è un attributo di raggruppamento né un aggregato: nel gruppo ha valori diversi, quindi la condizione non ha un valore per gruppo. MariaDB la rifiuta (`Unknown column 's.age' in 'HAVING'`); un DBMS permissivo come SQLite la esegue prendendo l'età di una riga qualsiasi del gruppo e restituisce un risultato sbagliato.
- **Errore tipico:** `where count(*) >= 2`: errore, aggregato nel `where`.
- `count(*)` conta *prenotazioni* (righe), non marinai: lo stesso marinaio con due barche nello stesso giorno vale 2. Per contare i marinai diversi servirebbe `count(distinct r.sid)`.
- «Almeno 18 anni» include chi ha esattamente 18 anni: `>= 18`, non `> 18`.

Esercizio SQL 3

Trova i nomi dei marinai condivisi da più persone (omonimi) il cui divario tra età massima ed età minima sia inferiore a 10 anni.

Ragionamento:

- **Raggruppamento**: gli omonimi si trovano raggruppando per nome, `group by s.sname`. `sname` non è una chiave: più marinai possono avere lo stesso nome ed è proprio ciò che si cerca. Basta la tabella Sailors, senza join.
- **Nome condiviso da più persone**: il gruppo ha più di un marinaio, cioè `count(*) > 1` (`sid` è la chiave: una riga è una persona).
- **Divario tra età massima e minima inferiore a 10**: `max(s.age) - min(s.age) < 10`. È un'espressione di aggregati, quindi sta nel `having`, in `and` con la condizione precedente.
- **Select-list**: solo il nome, che è l'attributo di raggruppamento.

Soluzione:

```
select s.sname
from sailors s
group by s.sname
having count(*) > 1
   and max(s.age) - min(s.age) < 10;
```

Controllo sull'istanza delle slide: solo Horatio (marinai 64 e 74, entrambi di 35.0 anni: divario 0). Tutti gli altri nomi compaiono una volta sola.

Fondamento teorico:

- Raggruppare per un attributo *non chiave* crea gruppi di tuple con lo stesso valore: `count(*) > 1` seleziona i gruppi con più di una tupla (pattern «omonimi» o «duplicati»). È la stessa differenza della slide «Che differenza c'è?»: sull'istanza `count(s.sname)` vale 10 e `count(distinct s.sname)` vale 9, perché Horatio compare due volte.
- Nel `having` si possono combinare più aggregati ed espressioni aritmetiche tra aggregati, purché abbiano un solo valore per gruppo.
- **Errore tipico:** dimenticare `count(*) > 1`: un nome unico ha divario $0 < 10$, quindi comparirebbero tutti i nomi.
- **Errore tipico:** scrivere il divario nel `where`: `max` e `min` sono aggregati sul gruppo, non proprietà della singola riga.
- «Inferiore a 10» è stretto: un divario esattamente uguale a 10 non soddisfa la condizione.

Esercizio SQL 4

Trova l'età media e il numero di marinai per ogni livello di rating, considerando solo i marinai con rating superiore a 5, ma mostra solo i livelli di rating che contengono almeno 2 marinai.

Ragionamento:

- **Raggruppamento**: «per ogni livello di rating» → `group by s.rating`. Basta la tabella Sailors.
- **Select-list**: l'attributo di raggruppamento `s.rating` e due aggregati: `avg(s.age)` (età media) e `count(*)` (numero di marinai: `sid` è la chiave, quindi una riga è un marinaio).
- **Filtro sulle righe**: «considerando solo i marinai con rating superiore a 5» → `where s.rating > 5`, applicato prima di raggruppare.
- **Filtro sui gruppi**: «solo i livelli che contengono almeno 2 marinai» → `having count(*) >= 2`; il conteggio riguarda i marinai che hanno superato il `where`.
- La condizione su `rating` riguarda l'attributo di raggruppamento, quindi potrebbe stare anche nel `having` (variante), ma nel `where` scarta le righe prima di raggruppare ed è più chiara.

Soluzione:

```
select s.rating, avg(s.age) as EtaMedia, count(*) as NumMarinai
from sailors s
where s.rating > 5
group by s.rating
having count(*) >= 2;
```

Variante equivalente, con il filtro sul rating nel `having` (meno efficiente: raggruppa anche i livelli da scartare):

```
select s.rating, avg(s.age) as EtaMedia, count(*) as NumMarinai
from sailors s
group by s.rating
having s.rating > 5 and count(*) >= 2;
```

Controllo sull'istanza delle slide: i livelli 7, 8 e 10, rispettivamente con età media 40, 40.5 e 25.5 e 2 marinai ciascuno. Il livello 9 ha un solo marinaio (74); il livello 3 ha 2 marinai (85 e 95) ma è scartato dal `where`.

Fondamento teorico:

- **Più aggregati** nella stessa select-list (`avg` e `count`) sono calcolati sullo stesso gruppo.
- **where, group by, having** sono applicati in quest'ordine (slide): le condizioni su un aggregato (`count`) possono stare solo nel `having`; quelle su attributi di raggruppamento possono stare in entrambi.
- **Modello**: l'esempio «GROUP BY e HAVING: esempio» delle slide (Q32 del libro) ha la stessa struttura, con `where S.age >= 18`, poi `group by S.rating` e `having count(*) > 1`.
- **Errore tipico:** dimenticare il `where`: compaiono anche i livelli con rating ≤ 5 (sull'istanza, il livello 3).
- **Errore tipico:** contare con `count(distinct s.age)`: conta le età distinte, non i marinai (due marinai coetanei darebbero 1).
- «Almeno 2» è `>= 2` (non `> 2`); «superiore a 5» è `> 5` (non `>= 5`).

Esercizio SQL 5

Identifica le barche (con bid e bname) prenotate da marinai la cui età media è superiore a 40 anni, considerando solo barche prenotate almeno 2 volte.

Ragionamento:

- **Raggruppamento**: una riga per barca, con bid e bname, quindi `group by b.bid, b.bname`: le due colonne non aggregate della select-list devono comparire entrambe nel `group by`.
- **Chiave e nome**: `bid` è la chiave e distingue le barche; `bname` no. Sull'istanza 101 e 102 si chiamano entrambe Interlake, quindi raggruppare solo per nome fonderebbe due barche.
- **Tabelle**: Boats (bid, bname), Reserves (le prenotazioni) e Sailors (age); join `b.bid = r.bid` e `r.sid = s.sid`.
- **Filtri sui gruppi**: «prenotate almeno 2 volte» $\rightarrow$ `count(*) >= 2` (numero di prenotazioni della barca); «età media dei marinai superiore a 40» $\rightarrow$ `avg(s.age) > 40`. Entrambe sono condizioni su aggregati: stanno nel `having`, in `and`.
- **Select-list**: solo `b.bid, b.bname`; gli aggregati servono solo nel `having`.

- **Cosa è una riga**: media e conteggio sono per prenotazione. Se lo stesso marinaio ha prenotato la stessa barca in due giorni diversi, la sua età pesa due volte nella media.

Soluzione:

```
select b.bid, b.bname
from boats b
     join reserves r on r.bid = b.bid
     join sailors s on s.sid = r.sid
group by b.bid, b.bname
having count(*) >= 2
   and avg(s.age) > 40;
```

Controllo sull'istanza delle slide: le barche 102 (Interlake), 103 (Clipper) e 104 (Marine). La barca 101, prenotata da 22 e 64 (età 45 e 35), ha età media esattamente 40 ed è esclusa dalla disuguaglianza stretta.

Fondamento teorico:

- **Regola della select-list**: ogni colonna non aggregata (`b.bid`, `b.bname`) compare nel `group by`. Si raggruppa per la chiave, e `bname` si aggiunge solo per poterlo mostrare (è determinato da `bid`, quindi non cambia i gruppi).
- Il `having` può combinare condizioni su aggregati diversi (`count`, `avg`) con `and`/`or`.
- **Relazione ponte**: Reserves collega Boats e Sailors; le aggregazioni lavorano sulle righe del join (una per prenotazione).
- Errore tipico: `group by b.bname`: `bname` non è una chiave e fonde barche omonime. Con `ONLY_FULL_GROUP_BY` la query è rifiutata (`b.bid` non è nel `group by`); senza, viene eseguita con un `bid` arbitrario per gruppo. Sull'istanza, Interlake (101 e 102) formerebbe un unico gruppo con 5 prenotazioni ed età media 43.1, quindi soddisferebbe le condizioni, mentre la barca 101 da sola no (media 40).
- Errore tipico: `where avg(s.age) > 40`: aggregato nel `where`, errore.
- Soglie: «almeno 2 volte» è `>= 2`, «superiore a 40» è `> 40`.

15.4 Riepilogo dei pattern ricorrenti

Tipo di richiesta	Schema risolutivo in SQL	Quesiti
Un risultato per ogni gruppo («per ogni ...», «i giorni», «le barche»)	`group by` sull'attributo che identifica il gruppo: una riga per gruppo	1–5
Filtro sulle righe («considerando solo ...»)	`where`, applicato prima del raggruppamento	2, 4
Filtro sui gruppi («solo i ... con ...»)	`having` con funzioni di aggregazione	1–5
Somma, media, numero, massimo, minimo	`sum`, `avg`, `count(*)`, `max`, `min`	1–5
«almeno 2», «più di uno»	`count(*) >= 2`, `count(*) > 1`	2–5
Omonimi, duplicati	`group by` su un attributo non chiave e `count(*) > 1`	3
Dati in più tabelle (relazione ponte)	`join ... on` lungo le chiavi esterne	1, 2, 5
Contare entità e non righe del join	`distinct` in una sottoquery, prima di aggregare	1 (variante)

Controlli prima di consegnare una query:

- **Raggruppamento**: gli attributi del `group by` sono quelli dei «per ogni»; preferire la chiave (`bid`) all'attributo descrittivo (`bname`), che si aggiunge solo per mostrarlo.
- **where o having**: filtro sulle righe → `where`; condizione su un aggregato → `having`; mai aggregati nel `where`.
- **Select-list**: tutti e soli i campi richiesti; ogni colonna non aggregata compare nel `group by`.
- **Cosa conta ogni riga**: dopo un join una riga è una prenotazione, non un marinaio o una barca; usare `distinct` (in una sottoquery) solo se il testo parla di entità distinte.
- **Soglie**: «superiore a» → `>`, «almeno» → `>=`; provare i casi limite sull'istanza delle slide (media esattamente 7 o 40).
- **Controllo finale**: eseguire la query sull'istanza delle slide e confrontare il risultato a mano.